



جامعة أم القرى
الكلية الجامعية بالجموم
قسم علوم الحاسب

XPlain

An Explainable Deep Learning Framework for Detecting AI-Generated Images

Graduation Project

Computer Science Department

Jamoum University College

Umm Alqura University

Submitted by

Jood Musaad Albishri- 444007983

Saja Naif Almalki-444000320

Ghadi hamzh allhyanie- 444008999

Rahaf alradadi – 444001602

Afnan Alsubhi -444008385

Project Advisor:

Dr : Mashael Alluhaybi

Date submitted: 7 Jun 2026

Table of Contents

Table of Contents	2
Abstract	6
List of Figures.....	7
List of Tables.....	8
Chapter 1.....	7
Introduction.....	7
1.1 Background and Motivation	7
1.2 Problem Statement.....	7
1.3 Objectives.....	8
1.4 Project Scope and Limitations	9
Chapter 2.....	10
Literature Review	10
2.1 Introduction	10
2.2 Review of Related Studies	10
2.2.1 Deepfake Detection	10
2.2.2 Applications in Image-Based Deepfake Detection	12
2.2.3 Applications in Video-Based Deepfake Detection	13
2.3 Research Gap and Justification.....	16
2.4 Feasibility Study	16
2.4.1 Project Description	16
2.4.2 Technical Feasibility	16
2.4.3 Operational feasibility.....	17
2.4.4 Financial feasibility.....	17
2.4.5 Risk & mitigation analysis.....	18
2.5 Summary	18
Chapter 3.....	19
Literature Review	19
3.1 Research Methodology.....	19
3.1.1 Development Approach	19
3.1.2 Data Collection and Preprocessing	20

3.1.3 Model architecture and training	20
3.1.4 Integration of Explainable AI (Grad-CAM Explainability Technique)	21
3.1.5 User workflow and streamlit Deployment	22
3.1.6 Model evaluation and validation	22
3.2 Tools and Technologies / Data Collection Methods	22
3.2.1 Python	22
3.2.2 TensorFlow / Keras	22
3.2.3 MobileNetV2	23
3.2.4 OpenCV (Image-Preprocessing)	23
3.2.5 Grad-CAM	23
3.2.6 Streamlit	23
3.2.7 Google Colab	23
3.2.8 “140K Real and Fake Faces” Dataset	23
3.3 Project Timeline	25
3.4 Risk Management	25
3.4.1 Project Management Risks	25
3.4.2 Technical Risks	25
3.4.3 Data and Model Risks	26
3.4.4 Ethical and Security Risks	26
3.4.5 Maintenance and Scalability Risks	26
3.5 Quality Assurance Plan / Validation and Reliability	27
3.6 Summary	28
4 Chapter	29
Project Requirements & Analysis	29
4.1 Stakeholder Identification	29
4.1.1 Assumptions	30
4.1.2 Constraints	30
4.2 Methods of Requirements Gathering / Data Needs and Sources	30
4.2.1 Data Needs and Sources	30
4.2.2 Technical Requirements and Image Processing	31
4.3 Functional Requirements	32
4.4 Non-Functional Requirements	33
4.5 Constraints	33
4.6 Summary	34
Chapter 5	35

System Design	35
5.1 Overview of Design Approach	35
5.2 Data Flow Analysis.....	36
5.3 XPlain System Architecture	37
5.4 Use Case Analysis	38
5.5 Entity-Relationship (ER) Analysis	38
5.6 System Use Case Diagram.....	40
5.7 System Use Case Description	40
5.8 System Class Diagram	42
5.9 XPlain System Interaction Diagram	43
5.10 System Activity Diagram.....	44
5.11 Front-End / User Interface Design.....	45
5.12 Database / Data Model Design	46
5.13 Algorithm / Process Design.....	47
5.14 Summary.....	48
Chapter 6.....	50
Implementation / Experimentation	50
6.1 Implementation Plan	50
6.2 Module / Process Development	51
6.2.1 Deterministic Forensic Noise Extraction (SRM Filters)	52
6.2.2 Dual-Topology Processing and Cross-Modal Fusion	53
6.3 Integration of Components	54
6.5 Deployment or Execution	54
6.5.1 ROC Curve	55
6.6 User Interface.....	56
6.6.1 Interface Overview.....	56
6.6.2 Dashboard Features.....	56
6.6.3 Image Analysis Interface	57
6.6.4 Model Dashboard Overview.....	59
6.6.5 Model Architecture	62
6.6 Summary	63
Chapter 7.....	64
Results and Evaluation	64
7.1 Results Presentation.....	64
7.1.1 Introduction	64

7.1.2 Final Model Results.....	65
7.1.3 Loss and Accuracy Plot	65
7.1.4 Confusion Matrix	67
7.1.5 Performance Evaluation on Unseen Data.....	69
7.1.7 Image Classification Results Using the Trained XPlain Model	69
7.2 Functional Testing Results.....	71
7.2.1 Test Environment and Scope.....	72
7.3 Performance Evaluation	72
7.4 Usability Testing / User Feedback.....	74
7.5 Comparative Analysis	77
7.6 Discussion of Findings	78
7.7 Challenges Encountered and Lessons Learned	78
7.8 Limitations.....	79
7.9 Summary	80
Chapter 8.....	81
Conclusion and Future Work	81
8.1 Summary of Work	81
8.1.1 Performance Benchmarking & Comparative Analysis	82
8.2 Key Findings	83
8.3 Limitations and Future Work.....	83

Abstract

Deepfakes have emerged as a critical challenge in recent years, driven by rapid advancements in artificial intelligence—particularly deep learning models capable of synthesizing highly realistic synthetic human faces that are virtually indistinguishable from authentic ones. The proliferation of such content has exacerbated digital skepticism, amplified visual deception, and accelerated the spread of misinformation online. Consequently, this poses severe threats to sensitive domains, including digital journalism, forensic investigations, cybersecurity, and media verification. However, most existing deepfake detection systems rely on deep learning architectures that operate as "black boxes," yielding predictions without disclosing the underlying decision-making process. This lack of transparency introduces ambiguity and diminishes user trust in automated verification tools.

To address this gap, this project introduces **Xplain**, an Explainable AI (XAI) framework designed to detect AI-generated imagery, justify model decisions, and highlight manipulated facial regions. The core architecture is built upon the **MobileNetV2** model, selected for its computational efficiency, rapid inference, and robust capacity for visual feature extraction, making it highly optimal for binary classification (Real vs. AI-generated).

Prior to classification, input images undergo a structured preprocessing pipeline. Instead of relying on traditional face-bounding algorithms, the system integrates a **Spatial Rich Model (SRM) filter** to extract high-frequency noise residuals, isolating subtle artifacts left behind by generative models. This is followed by image alignment, dimensionality standardization, and color normalization to ensure uniform input quality.

To provide visual interpretability, the system leverages **Grad-CAM** (Gradient-weighted Class Activation Mapping) to generate heatmaps that project the model's localized focus. These heatmaps explicitly highlight specific facial regions—such as the eyes, nose, or mouth—where subtle, anomalous traces of manipulation typically manifest.

To further enhance interpretability, Xplain complements these visual cues with concise textual rationale detailing the potential indicators that led to the model's classification. The framework was trained on the benchmark **140k Real and Fake Faces dataset**, providing a diverse and comprehensive corpus of authentic and GAN-generated imagery, thereby optimizing the model's generalization and pattern-detection accuracy.

Targeting a diverse stakeholder base—including academic researchers, fact-checkers, digital forensics experts, and general public users—Xplain delivers an intuitive, interactive interface. The system allows seamless image uploads, rendering instantaneous classification results paired with both visual and textual explanations.

List of Figures

Figure 1: The project timeline from start to final delivery.	25
Figure 2 : The architecture of the proposed MobileNetV2 with Grad-CAM for model interpretability.	25
Figure 3: Data Flow Diagram Showing Image Processing Pipeline in XPlain.....	36
Figure 4: System Architecture	37
Figure 5: shows the ER diagram of the system.	39
Figure 6: use case diagram.....	40
Figure 7: class diagram.....	42
Figure 8: System interaction sequence diagram	43
Figure 9: System Activity Diagram.....	45
Figure 10: Interface Design.....	46
Figure 11: Database Diagram	47
Figure 12: XPlain Algorithm	48
Figure 13: Implementation of the Spatial Rich Model layer for noise extraction	52
Figure 14 : Initialization of SRM weights	53
Figure 15: Feature extraction layers for the SRM stream	53
Figure 16 : Two-stream fusion and output layers	54
Figure 17 : ROC.....	55
Figure 18 : XPlain dashboard showing system features and usage guidance.....	57
Figure 19: Image upload interface for deepfake detection	57
Figure 20: Prediction result with confidence score	58
Figure 21 : Prediction result with Grad-CAM visualization.....	58
Figure 22: XPlain overall model performance	59
Figure 23: XPlain Final evaluation loss graph	60
Figure 24: Training and validation accuracy curve	60
Figure 25: Validation loss curve	61
Figure 26 : Confusion matrix for XPlain classification results	61
Figure 27: Class-level performance metrics for real and AI-generated images	62
Figure 28: system architecture of the XPlain model	63
Figure 29 : Final Model Results	65
Figure 30 : Loss Plot.....	66
Figure 31: Accuracy Plot	67
Figure 32: Xplain Confusion Matrix	68
Figure 33 : Xplain Test Results.....	69
Figure 34: Testing Result(REAL)	70
Figure 35: Testing Result(AI GENERATED)	71
Figure 36: Model Evaluation on Validation Data	73
Figure 37: Image analysis and prediction output.....	74
Figure 38: Prediction confidence visualization	75
Figure 39: Visual explainability outputs	75
Figure 40: Result interpretation and confidence summary.....	76
Figure 41: Comparison of XPlain Accuracy Against State-of-the-Art Deepfake Detection Models...	82

List of Tables

Table 1 : Comparison of Machine Learning Models Used in Deepfake Detection	12
Table 2 : Comparison of Image-Based Deepfake Detection Models and Techniques	13
Table 3 : Comparison of Machine Learning Models Used in Applications Video- Based Deepfake Detection.....	15
Table 4 : Summary of Identified Risks and Mitigation Strategies for the XPlain Framework.....	18
Table 5 : Sprint Plan and Deliverables.....	20
Table 6 : Tools and Technologies / Data Collection Methods.....	24
Table 7 : Summary of Risks and Mitigation Strategies in the XPlain System.....	27
Table 8: Comparison between the Xplain system and existing deepfake detection models	77

Chapter 1

Introduction

1.1 Background and Motivation

With the rapid advancements in artificial intelligence, deep fakes—synthetically generated media in which a person's likeness is manipulated—have become increasingly sophisticated. Detecting DeepFakes has become a crucial research area as the widespread use of AI image generators enables the effortless creation of face manipulated and fully synthetic content, while existing methods are often limited to binary classification (real vs. AI-generated) and lack interpretability[1].

Deepfake combines two literals, Deep and fake, related to fake material generated by Deep Neural Network (DNN)s [2]. The AI image generated data via the deepfake technique is genuine and realistic and not easily recognizable by the human being[3].

Several detection methods have been proposed, mainly relying on Convolutional Neural Networks (CNNs) such as ResNet-50, VGG-16, and Inception V3, which have demonstrated promising results in distinguishing authentic from manipulated media. Interestingly, when examining the output of generative models—like GANs or modern diffusion architectures—one can consistently observe high-frequency artifacts left behind. These traces typically manifest at the pixel level as checkerboard patterns, essentially a byproduct of upsampling layers struggling to maintain consistent sub-pixel alignment. Standard CNNs almost always miss these subtle anomalies. Their underlying architecture is designed to do the exact opposite: they pool spatial information, compressing it into broader semantic representations rather than preserving microscopic noise. However, a major limitation of these approaches lies in their lack of interpretability— they often act as “black boxes,” providing predictions without clear explanations of their decision-making processes. This lack of transparency raises concerns about trust, accountability, and adoption in real-world scenarios[4].

The motivation for this project is related to the growing need to increase both accuracy and interpretation in the deep properties of system detection . The manipulated media is increasingly refined and is not sufficient to rely on high classification performance. There is a critical demand for a system that not only detects incorrect content, but also provides a clear explanation of the data from decisions[4][5]. Such interpretations are important to strengthen trust in artificial intelligence systems. By making the decision process more transparent and understandable, the system is more reliable, responsible, and adapts to a wider implementation in sensitive applications[5].

To solve this problem, the project combines the described AI method (XAI) methods to analyze specific, networks and combine them with a compelling model of neural networks. This dual approach aims to improve detection performance and provide transparent inference behind each classification.

1.2 Problem Statement

The use of deepfake technology to manipulate digital photographs has grown to be a major worldwide problem in recent years. Using advanced deep learning techniques, fake images can now be generated to appear almost identical to real photographs, making it difficult for ordinary users to distinguish between authentic and fabricated visuals [6].

This growing ability to create realistic synthetic images has raised serious ethical, social, and security concerns. Such images can be used to spread misinformation, harm reputations, or manipulate public opinion for political or personal gain [7].

The widespread circulation of deepfake images on social media and digital platforms has contributed to an erosion of public trust in visual information. When people can no longer be sure whether an image is real or AI-generated, confidence in news media, online communication, and even evidence-based systems such as law enforcement or journalism begins to weaken [7]. These effects extend beyond technology, influencing psychological and social behavior by amplifying confusion and distrust.

Although many researchers have developed AI-based systems to detect manipulated images, most existing models still function as “black boxes.” They provide only a binary answer real or fake without explaining the reasoning behind their decision [8]. In practice, a significant limitation observed in existing single-path networks is their inability to simultaneously optimize for macro-level appearance and micro-level noise. During the training phase, these two distinct objectives inherently compete. Consequently, the network is forced into a compromise, which ultimately restricts the overall precision of the detection.

This lack of transparency reduces user trust and limits the adoption of these systems in critical domains that require accountability and explainability. Therefore, there is an urgent need for Explainable Artificial Intelligence (XAI) methods that can both detect deepfake images accurately and visually demonstrate *why* the system believes an image is manipulated. Providing such interpretability helps ensure transparency, supports ethical AI use, and strengthens public trust in automated detection systems [6][8].

1.3 Objectives

1. Develop an Explainable Detection System:

The main technical objective of this project is to develop an explainable deep-learning-based system that splits processing into two parallel streams; Stream A uses MobileNetV2 to encode structural appearance, while Stream B utilizes a custom Spatial Rich Model (SRM) filter layer to extract deterministic forensic noise.

2. Achieve Efficient and Reliable Detection Performance:

Another objective is to ensure that the detection system works efficiently and reliably by using pre-trained models. It focuses on maintaining stable classification performance and computing efficiency suitable for practical use in resource-limited environments

3. Integrate Explainability for Model Transparency:

The main objective is to increase transparency by integrating Explainable AI (XAI) technologies, specifically Grad-CAM, to visualize the image regions that affect model predictions. This helps users understand the rationale behind each classification decision.

4. Support user trust:

The ultimate objective is to present both prediction results and visual explanations in a clear and unambiguous way through a user-friendly interface. This method aims to improve user confidence and facilitate meaningful interpretations of system detection result

1.4 Project Scope and Limitations

The scope of the Xplain project is to develop a robust framework for detecting AI_Generated manipulations in images, with a strong emphasis on integrating accuracy and interpretability to foster user trust. Unlike conventional AI_Generated detection systems that function as "black boxes" and provide only a binary classification, our project introduces a Two-Stream Architecture—combining MobileNetV2 for RGB with SRM for forensic noise—that explicitly explains the reasoning behind its decisions.

The core focus is on building a data-driven, explainable AI (XAI) system that can clearly highlight the specific features and visual artifacts within images that led to a particular classification. This is through the integration of advanced deep learning models with explainability techniques to generate human understandable justifications, such as visual heatmaps and textual descriptions. The project will concentrate on analysing and detecting manipulations in images.

The goal is to create a reliable and interpretable tool that can be critically evaluated for use in real-world scenarios where understanding the "why" behind a decision is as important as the decision itself. The framework aims to make results more trustworthy and easier for users to understand.

Limitations Computational and Temporal Overhead: Merging sophisticated convolutional neural networks (CNNs) with explainable AI techniques is notoriously resource-intensive.

Generalization and Data Dependency: The system's ability to both detect fakes and explain its reasoning is only as good as the data it was trained on. A real concern is that it becomes a specialist on its "home turf"—the specific deepfake types in its training set—but fails to adapt to new manipulation methods it hasn't seen before.

Constrained Media Modalities: The project currently focuses only on visual data is squarely on images frames. This is a clear weakness in an era where convincing deepfakes are often multimedia productions. which is a significant shortcoming given that modern deepfakes often use multiple modes of manipulation.

Data Confidentiality: The dataset's variety and the model's accuracy may be impacted by sensitive or private data, which restricts its gathering or unrestricted usage.

Inherent Constraints of XAI Methods: We have to remember that the explainability tools, like Grad-CAM, have their own flaws. They provide a general spatial region of interest based on regional activation percentages (left, center, and right), but they provide clues based on localized anomalies rather than highlighting the exact micro-level pixel glitch or an artifact in the background that actually gives the manipulation away. They provide a clue, but not always the full picture.

Trade-off Between accuracy and speed: We're constantly forced to make a choice. Do we want a more thorough analysis by using more frames and higher-resolution images, which gives us better accuracy? Or do we prioritize speed and practicality? If we want . the system to check every detail in very high-quality images, it may take more time and require stronger hardware. This trade-off is a fundamental constraint that shapes the system's real-world usability .

Potential for Subjective Interpretation: While the system aims for objective explanations, the highlighted regions or features might still be open to subjective interpretation by the end-user. Furthermore, if the training data involves human annotation for explanations, it could lead to subjective interpretation of results.

Inherent Data Biases: The model's intelligence is a reflection of its training data. If that data isn't comprehensively diverse—covering a wide range of people, lighting conditions, and deepfake algorithms Its performance will be skewed, and its explanations might be misleading

Chapter 2

Literature Review

2.1 Introduction

Many manipulation algorithms[9] , [10] , [11] person without specific skills to produce high-quality fake faces without expert skills and special knowledge for training. As a result, it can be often challenging for the human eyes to identify the difference between actual and manipulated images. This has led to an increase in the usage of modified multimedia content in various cybercrime activities. The technology may be utilized maliciously, resulting in a major trust issue for modern society. Due to the fact that such methods may produce high-quality fake images that are even indistinguishable from human eyes. Therefore, the scientific community has shown a lot of interest in the need to develop techniques for identifying authentic faces from fraudulent images. Many methods for deepfake detection have been proposed[12][13][14] .

While detection accuracy remains a vital focus, the interpretability of deep learning models is equally important to ensure their widespread acceptance in real-world applications. Users, decision-makers, and stakeholders must understand why a particular instance is classified as AI-generated or real. Without this understanding, the trustworthiness and adoption of these systems may be limited, particularly in critical domains such as journalism, law enforcement, and content moderation. Additionally, the lack of transparency could hinder forensic investigations and legal proceedings where evidence-based reasoning is required. To address these challenges, this research incorporates explainable artificial intelligence (XAI) techniques to provide clear and interpretable explanations for the decisions made by the deepfake detection models. By doing so, this study aims to enhance trust in the system's outputs and improve its practicality in addressing the societal risks posed by deepfakes[15].

This chapter provides a comprehensive analysis of current deepfake detection methodologies. The discourse commences with a synopsis of deepfake creation and detection techniques, subsequently addressing the contribution of machine learning and deep learning methodologies in recognizing altered content. Additionally, the chapter explores explainable AI techniques that enhance model transparency and reliability. Finally, it introduces the datasets and evaluation metrics that form the basis for subsequent experimental analysis.

2.2 Review of Related Studies

2.2.1 Deepfake Detection

In recent years, the development of deepfake technology has become one of the most discussed topics in the field of Artificial Intelligence (AI). Deepfakes refer to digital media such as videos, images, or audio that have been artificially created or modified to appear real to the human eye or ear [16]. The ability of these systems to produce highly convincing fake content has made it increasingly difficult for people to differentiate between authentic and manipulated material.

This growing realism has raised significant concerns worldwide. On the positive side, deepfake technology can be used for creative applications in entertainment, filmmaking, virtual training, and education. However, on the negative side, it has also opened the door to the spread of misinformation, online fraud, and violations of privacy [17]. For example, AI-generated or manipulated image can be used to impersonate individuals, create false political statements, or manipulate public opinion. These risks highlight the need for reliable methods to verify the authenticity of digital content shared online.

As a result, detecting deepfakes has become a major research focus. Many efforts have been made to develop detection systems that can identify whether a given image, is real or AI-generated [16]. These systems aim to analyze patterns and inconsistencies that are invisible to the human eye, allowing automatic recognition of altered media. Nevertheless, the rapid progress of deepfake generation methods has made detection increasingly challenging. Each time researchers design new detection models, more advanced deepfake techniques emerge that can bypass them, creating an ongoing cycle between generation and detection [17].

Beyond the technical challenges, there are also ethical and social dimensions to this issue. Deepfakes are often described as a “double-edged sword”: while they can encourage creativity and innovation, they can also be misused for harmful or deceptive purposes. Therefore, many researchers argue that tackling the deepfake problem requires not only technological solutions but also legal frameworks, awareness campaigns, and policies that promote responsible AI use [18]. Together, these approaches can help maintain trust, protect individuals’ reputations, and preserve the integrity of digital media in an increasingly AI-driven world . Overall, while deepfake technology presents both creative opportunities and serious risks, a combination of technological, ethical, and legal strategies is required to mitigate its negative impact. Developing reliable detection systems, raising public awareness, and implementing responsible AI policies are all essential steps toward ensuring the safe and trustworthy use of AI-generated media

From a technical perspective, this phenomenon represents breakthrough in artificial intelligence technologies, particularly in generating realistic images using Generative Adversarial Networks (GANs)[19]. This advancement has led to the emergence of Deepfake technology — a technique that relies on deep learning algorithms to create synthetic visual content that is almost indistinguishable from genuine material [20]. Although such technologies offer creative opportunities in entertainment and digital media, they have also raised serious ethical and security concerns due to their misuse in spreading misinformation, identity theft, reputation damage, and privacy violations [21].

As the threats associated with Deepfake content continue to grow, the need for intelligent detection systems has become increasingly critical. Machine learning (ML) and, more specifically, deep learning (DL) techniques have proven to be highly effective in this field, as they can analyze subtle patterns within visual and temporal domains to detect synthetic inconsistencies introduced during generation [22][23].

Recent studies employ advanced architectures such as Convolutional Neural Networks (CNNs) to capture spatial patterns, achieving detection accuracy of up to 96.5% on DFDC datasets [Khan et al., 2021][22], and Long Short-Term Memory networks (LSTMs) to model temporal variations across video frames, with hybrid CNN-LSTM models reaching up to 99.7% accuracy on datasets like FaceForensics++ and Celeb-DF [Appl. Sci., 2024][24]. Vision Transformers (ViTs) are also being used for understanding long-range dependencies within visual data, achieving accuracy levels above 90% in large-scale datasets [19][21].

Additionally, Explainable AI (XAI) techniques, including Grad-CAM, LIME, and SHAP, are integrated to visualize and interpret the model’s decision-making process, thereby improving transparency and trust in detection results [23][25].

These machine learning applications play a crucial role across several domains — including cybersecurity, digital journalism verification, forensic investigations, and social media monitoring [21][22]. In the coming years, Deepfake detection is expected to evolve toward real-time detection systems with enhanced generalization capabilities, enabling them to identify manipulated content

generated by new and unseen AI models, with early real-time systems achieving accuracy above 80% under certain test conditions [23][21][25].

This table compares popular machine learning models for Deepfake detection, showing their main functions, accuracy from recent studies, and key references, based on results from standard datasets like DFDC, FaceForensics++, and Celeb-DF.

Model / Technique	Primary Function	Accuracy	Reference
Convolutional Neural Networks (CNNs)	Extract spatial patterns from images and videos	Up to 96.5% on DFDC dataset	[22]
Long Short-Term Memory (LSTMs)	Model temporal variations across video frames	Up to 99.7% with CNN + Bi-LSTM on FaceForensics++ and Celeb-DF	[24]
Vision Transformers (ViTs)	Understand long-range dependencies in visual data	Over 90% on large-scale datasets	[19][21]
Explainable AI (XAI) (e.g., Grad-CAM, LIME, SHAP)	Visualize and interpret model decisions	Not directly used for prediction	[23][25]

Table\ : Comparison of Machine Learning Models Used in Deepfake Detection

2.2.2 Applications in Image-Based Deepfake Detection

Image-based deepfake detection has emerged as a crucial research direction due to the widespread use of generative adversarial networks (GANs) and other advanced synthesis techniques in producing highly realistic fake images. Unlike video-based detection, which considers temporal inconsistencies, image-based approaches primarily focus on spatial irregularities such as unnatural facial textures, inconsistencies in lighting, or abnormal artifacts in local regions of the face. Machine learning, particularly deep learning models, has been widely applied to address these challenges, achieving significant advancements in detection performance.

One notable contribution is the Guardian-AI model, which integrates Vision Transformers (ViTs), Convolutional Neural Networks (CNNs), and Long Short-Term Memory (LSTM) networks. The model leverages ViTs to capture long-range dependencies within facial structures, while CNNs focus on local feature extraction. By combining these with LSTMs, Guardian-AI is able to analyze subtle inconsistencies in deepfake images with high accuracy, achieving up to 95.8% on the CDDb dataset. This demonstrates the effectiveness of hybrid architectures in improving robustness against various manipulation techniques [26].

Another important work is the Deepfake Predictor (DFP), a hybrid technique that combines VGG16 with additional CNN layers to improve the classification of real and manipulated facial images. The DFP model was trained and evaluated against multiple transfer learning architectures, including Xception, MobileNet, and NAS-Net. Experimental results showed that DFP outperformed these baselines, reaching 94% accuracy and 95% precision. This highlights the potential of combining traditional CNN structures with transfer learning techniques to build efficient and reliable deepfake image detectors [27].

In addition, explainability has recently been incorporated into deepfake image detection to increase trust and transparency. For example, an approach utilizing multiple CNN architectures—such as InceptionResNetV2, DenseNet201, ResNet152V2, and InceptionV3—integrated with the Local Interpretable Model-Agnostic Explanations (LIME) method, has achieved very high detection performance, with accuracies exceeding 99%. Beyond accuracy, LIME provided visual explanations by highlighting specific facial regions (e.g., eyes, mouth, and cheeks) that contributed most to the model’s decisions. Such interpretability ensures that detection outcomes are not only accurate but also understandable to users, thus improving the trustworthiness of AI-driven forensic systems [28].

Overall, applications in image-based deepfake detection demonstrate that combining advanced CNN and transformer-based architectures with explainability techniques yields highly effective systems. These models not only achieve state-of-the-art performance but also enhance transparency, which is essential for practical adoption in digital forensics and cybersecurity domains.

Table (2) presents a comparison of different image-based deepfake detection models and techniques. The table highlights the datasets used, reported accuracies, and references for each approach. It demonstrates that hybrid and explainable models, such as Guardian-AI and CNN combined with LIME, achieve high detection performance, often exceeding 95%, indicating the effectiveness of combining deep learning architectures with interpretability methods in detecting manipulated facial images.

Model / Technique	Dataset	Accuracy	Reference
Guardian-AI (ViT + CNN + LSTM)	CDDDB dataset	Up to 95.8%	[26]
Deepfake Predictor (DFP)	Multiple transfer learning datasets (e.g., Xception, MobileNet, NAS-Net)	94% accuracy, 95% precision	[27]
CNN + Explainable AI (LIME)	Multiple benchmark datasets	Over 99%	[28]

Table 2 : Comparison of Image-Based Deepfake Detection Models and Techniques

2.2.3 Applications in Video-Based Deepfake Detection

Video-based deepfake detection has become a major research focus due to the increasing sophistication of generative models capable of producing hyper-realistic manipulated videos. Unlike image-based detection, which analyzes spatial inconsistencies, video-based methods must also address temporal coherence, motion continuity, and multi-modal cues to determine the authenticity of audiovisual

content. Researchers have therefore explored a broad range of deep learning frameworks—ranging from Convolutional Neural Networks (CNNs) and Transformers to multi-modal fusion and explainable AI (XAI) approaches—to identify synthetic videos with high precision and interpretability.

One of the most prominent methodological directions is based on Convolutional Neural Networks (CNNs), which analyze spatial and temporal cues extracted from video frames. For instance, a dual-branch CNN framework proposed in AI-Generated Video Detection via Spatio-Temporal Anomaly Learning employed two ResNet50 models—one for RGB frame analysis and another for optical flow representation—achieving an accuracy of 91.4% and AUC was 97.1% on the GVD dataset, even under compression noise [29]. Similarly, ensemble CNN architectures integrating Xception, EfficientNet-B0, and ResNet50, supported by Grad-CAM for visual interpretability, demonstrated an accuracy of 92.8% and F1 was 92.9%, highlighting CNNs’ robustness in detecting fine spatial-temporal inconsistencies [30].

Building on this foundation, other studies investigated temporal consistency-based detection, emphasizing the analysis of motion smoothness and frame transitions. The DeCoF framework introduced a CNN-based approach for frame consistency evaluation, achieving an accuracy of 94.84% on Pika-generated videos and 98.46% on Gen-2-generated ones [31]. Extending this idea, ReStrAV utilized Vision Transformers (DINOv2) to represent perceptual straightness in temporal trajectories, reaching an accuracy of 97.17% and AUROC was 98.63% on the VidProM dataset, effectively capturing temporal irregularities that static CNNs often miss [32].

Another important research direction adopted Transformer-based architectures for comprehensive video-level representation learning. The GenVidBench benchmark systematically evaluated models such as VideoSwiv and UniformerV2 across 143,000 videos generated by eight AI models. Results revealed high intra-dataset accuracy of up to 88%, but reduced generalization below 70% across unseen generators, underscoring the need for adaptive and continually updated models [33].

Furthermore, a complementary line of work explored multi-modal deepfake detection, integrating visual, auditory, gestural, and metadata cues. The study Multi-Modal Deepfake Detection Using AI proposed a hybrid model that combines CNNs, LSTM networks, and attention-based fusion, achieving an accuracy of 93.6%, F1 was 93.4%, and AUC was 96.7% on datasets such as DFDC and ASVspoof. This integration improved the model’s robustness in real-world scenarios where manipulations span multiple modalities [34].

Some hybrid frameworks also utilized ensemble learning by combining deep neural networks with classical machine learning classifiers. A New Deep Learning-Based Methodology for Video Deepfake Detection Using XGBoost merged YOLOv3 for face detection, InceptionResNetV2 for feature extraction, and XGBoost for classification, achieving an accuracy of 90.73% and AUC was 90.62% on the CelebDF and FaceForensics++ datasets [35]. Similarly, DeepFake Videos Detection Using Machine Learning employed a CNN with Laplacian and Canny image transformations, reaching an accuracy of 95% and AUC was 95%, demonstrating that even lightweight CNN pipelines can achieve competitive results in video-level classification [36].

Lastly, recent advancements emphasize Explainable AI (XAI) to enhance transparency and interpretability in detection outcomes. The DeepExplain framework integrated CNNs and LSTMs with Grad-CAM and SHAP to visualize and quantify decision rationales, enabling users to understand model predictions. Additionally, ExDDV (Explainable Deepfake Detection Video Dataset) provided over 5,400 annotated videos with textual and click-based explanations, serving as a new benchmark for developing explainable deepfake detectors [37][38].

Overall, applications in video-based deepfake detection illustrate an evolution from CNN-based spatial analysis to temporal and transformer-driven systems, culminating in multi-modal and explainable architectures. These approaches collectively enhance detection accuracy, robustness, and

interpretability—key factors for practical use in digital forensics, media authentication, and cybersecurity domains.

Table 3 provides a comparative summary of the leading video-based deepfake detection methodologies. Each method is categorized by its underlying machine learning architecture, the datasets used for evaluation, and the reported accuracy. The references correspond to the research studies discussed in the preceding section. The table highlights the diversity of approaches—ranging from CNN-based frameworks and Transformer models to multi-modal and explainable AI architectures—reflecting the continuous evolution of detection techniques to handle temporal, spatial, and multi-modal challenges in synthetic video analysis.

Machine Learning Method	Dataset Used	Accuracy (%)	Reference
Dual-Branch CNN (ResNet50 for RGB & Optical Flow)	GVD Dataset	91.4	[29]
Ensemble CNN (Xception, EfficientNet-B0, ResNet50 + Grad-CAM)	Custom Benchmark	92.8	[30]
DeCoF (CNN-based Frame Consistency Evaluation)	Pika / Gen-2 Generated Videos	94.84 / 98.46	[31]
ReStraV (Vision Transformer – DINOv2)	VidProM Dataset	97.17	[32]
Transformer Models (VideoSwiv, UniformerV2)	GenVidBench (143,000 Videos)	Up to 88 (≤ 70 on unseen data)	[33]
Multi-Modal CNN + LSTM + Attention Fusion	DFDC / ASVspoof	93.6	[34]
YOLOv3 + InceptionResNetV2 + XGBoost	CelebDF / FaceForensics++	90.73	[35]
CNN with Laplacian & Canny Transformations	Custom Dataset	95.0	[36]
DeepExplain (CNN + LSTM + XAI: Grad-CAM, SHAP)	ExDDV Dataset (Explainable Deepfake Detection Video)	N/A	[37][38]

Table 3 : Comparison of Machine Learning Models Used in Applications Video- Based Deepfake Detection

2.3 Research Gap and Justification

Despite substantial progress in the domain of Deepfake detection, many of the current systems continue to face critical limitations that restrict their performance and reliability in real-world scenarios. Most state-of-the-art approaches built on **Convolutional Neural Networks (CNNs)** or **Transformers** emphasize achieving high detection accuracy; however, they frequently function as “**black-box**” **models**, offering little insight into the reasoning behind their classifications. This lack of transparency undermines user confidence and presents ethical and legal challenges in sensitive sectors such as **digital journalism, forensic analysis, and cybersecurity** [15][22].

Another key shortcoming of existing models is their limited **generalization ability**. These systems typically perform well on the specific types of manipulations used in their training datasets but tend to lose accuracy when confronted with new or unseen Deepfake techniques [16][19]. Such dependency on dataset-specific artifacts limits the adaptability of models to the rapidly evolving landscape of generative technologies.

Moreover, achieving **real-time detection** remains a significant challenge because deep learning models are computationally intensive, particularly when explainability tools like **Grad-CAM** or **LIME** are integrated. Many current frameworks fail to maintain an effective balance between **speed, accuracy, and interpretability**, which is essential for real-world applications such as **social media content verification** and **digital forensics** [23][25][30].

To overcome these limitations, this study introduces an **Explainable Deepfake Detection System (XPlain)** that leverages **Explainable Artificial Intelligence (XAI)** to combine high predictive accuracy with meaningful interpretability. The proposed framework employs **MobileNetV2** for efficient real-versus-AI-generated image classification and integrates **Grad-CAM** to visualize the specific image regions influencing model predictions [28][37]. This integration enhances transparency and accountability, helping users and researchers better understand the underlying rationale for the model’s outcomes.

Additionally, the system is trained on a combination of diverse benchmark datasets such as **DFDC**, **Celeb-DF**, and **140K Real and Fake Faces**, which strengthens the model’s ability to generalize and adapt to unseen manipulation techniques.

In summary, this research aims to **bridge the gap between accuracy and interpretability** in Deepfake detection by developing a scientifically sound and ethically transparent model that can be confidently deployed in real-world environments to improve trust in digital content.

2.4 Feasibility Study

2.4.1 Project Description

The XPlain framework is an AI-based AI-generated image detection tool designed to classify the input image as real or AI-generated using MobileNetV2 CNN model, supported by Grad-CAM for explainability and text generation to comprehend the decision of the CNN model, increasing both trust and transparency in model decision for each prediction. The platform is implemented using Python, Tensorflow, Open-CV, and Streamlit, making it accessible for research, and light weight deployment.

2.4.2 Technical Feasibility

The technologies required for this framework are widely accessible, stable, and well-structured. In addition, MobileNetV2 model uses lightweight architecture suitable for deployment, while open-cv

offers robust image processing routines. The dataset, 140K real and fake faces, is free to access, eliminating data acquisition challenge.

The system follows a modular architecture, for instance, the frontend module uses Streamlit user interface, the backend uses API for processing and model inference, Model core uses MobileNetV2 classifier, Explainability unit utilizes Grad-CAM visualization and text interpretation, and Storage for image logs, prediction results and feedback. Due to lightweight architecture, the system can be built and executed in local machines or cloud, such as AWS or GCP, with no special hardware required, making the project technically feasible due to mature and available tools.

2.4.3 Operational feasibility

The entire workflow—from uploading an image to receiving prediction results—is designed to be intuitive and accessible, with Streamlit abstracting UI complexity to provide a user-friendly interface suitable for non-experts; interpretability elements like heatmaps and text explanations further enhance user trust.

This project can be developed by a small, technically skilled team experienced in machine learning (CNNs), computer vision, Streamlit development, and basic MLOps practices, aligning well with typical academic or research-level competencies. Its modular architecture supports easy updates to model weights, explainability logic, dataset improvements, and UI modifications, while the system can scale to handle larger datasets or heavier workloads by deploying the model to a GPU-based cloud environment.

2.4.4 Financial feasibility

The cost of development tools is minimal since all key components are open-source and free to use, while the dataset requires no licensing fees, eliminating financial barriers associated with data acquisition and software licensing. Hardware needs are modest, with a mid-range laptop (8–16 GB RAM) sufficient for development and optional GPU access available to accelerate training processes, enabling flexibility depending on project scale and resource availability.

Cloud computing costs are optional and generally low, allowing the choice between local and cloud-based environments depending on budget and performance requirements. For deployment, the Streamlit app can be hosted at no cost through platforms like Streamlit Cloud, the Heroku free tier (if available), or local hosting setups, providing versatile options for going live without incurring additional expenses. Overall, the project's financial requirements are negligible, making it highly feasible even within tight student or academic research budgets while maintaining scalability and functionality.

2.4.5 Risk & mitigation analysis

Risk	Description	Mitigation
Technical	Model may fail on unseen images	Add more datasets, use augmentation
Operational	Inconsistent heatmap explanation	Validate during sprints, review activation maps
Data	Corrupted or low quality images	Add preprocessing checks
Security	Sensitive image uploaded	Apply temporary storage
Performance	Slow training	Use MobileNetv2 + scheduled training

Table 4 : Summary of Identified Risks and Mitigation Strategies for the XPlain Framework

In conclusion, the project is feasible across all domains. It utilizes cost-free resources, proven technologies, and an agile workflow that supports continuous refinement. With a clear risk-management strategy and minimal operational overhead, the XPlain project is practical, achievable, and sustainable.

2.5 Summary

This chapter offers a thorough examination of prior research and studies concerning Deepfake detection methodologies and the contemporary strategies employed to tackle this escalating issue. It commences by delineating the essential principles of Deepfakes and underscoring their primary technical, security, and societal threats. Subsequently, the chapter explores the most notable deep learning techniques, such as Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), and Transformers, along with their contributions to identifying altered content in images.

The review further highlights that, although the majority of existing research has concentrated on attaining high classification accuracy, it frequently neglects the significance of interpretability, leading to detection systems that operate as opaque entities and lack clarity. This shortcoming considerably diminishes user confidence and obstructs the implementation of such systems in vital areas like digital forensics, journalism, and cybersecurity.

In addition, the chapter contrasts the leading models utilized in both image-based Deepfake detection, detailing their advantages and disadvantages, and introduces recent developments in Explainable Artificial Intelligence (XAI) techniques—such as Grad-CAM, LIME, and SHAP—which assist in elucidating the rationale behind model decisions and improving the transparency and dependability of detection results. Ultimately, the chapter pinpoints several research deficiencies, including the imbalance between accuracy and interpretability, the restricted generalization of current models when confronted with novel types of manipulations, and the computational hurdles associated with achieving real-time detection.

Chapter 3

Literature Review

3.1 Research Methodology

This chapter outlines the approach employed for the development and assessment of the proposed XPlain system, a framework based on Explainable Artificial Intelligence (XAI) intended to identify distorted facial photos disseminated on social media platforms. It offers a comprehensive elucidation of the methodologies, instruments, and strategies utilized throughout the system's development, encompassing dataset selection and preprocessing, model architecture and training, in addition to the incorporation of explainability techniques and performance assessment [37][28].

The suggested methodology combines Deep Learning (DL) with Explainable Artificial Intelligence (XAI) techniques to improve transparency and interpretability in deepfake detection systems. The system utilizes Convolutional Neural Networks (CNNs) to extract spatial features from images[37]. Additionally, explainability tools like Grad-CAM, LIME, and SHAP are utilized to depict and elucidate the model's conclusions by emphasizing the most significant areas that influenced the classification of an image as "real" or "AI-generated" [28].

This method is an extension of a number of recent works in explainable deepfake detection, such as the DeepExplain framework that exploits CNN models with Grad-CAM to improve decision transparency [37], and hybrid CNN-based proposed in which InceptionResNetV2 model is combined with LIME to produce interpretable visual explanations for predictions [28]. These methods are the theoretical base for constructing the XPlain system presented in this paper.

In summary, this chapter establishes the foundation for the experimental work by presenting the methodology adopted to implement and evaluate the XPlain system.

3.1.1 Development Approach

The Agile development methodology was adopted in this project to ensure that detection systems are adapting to the continuous updates of deepfake media. The overall project framework was designed to balance between research and development processes of the XPlain project. Since the project involves complex stages, it was obvious that agile methodology is the way to go as it also offers flexibility, iterative progress, and continuous improving and updating throughout the system design and implementation. The main pillars to adopt agile methodology are iterative discovery attempts across data quality, model generalization, and user experience, enabling sprints of experimentations and feedback rather than a plain, rigid plan. Asking for feedback from stakeholders, consumers reduces delivery risk by validating new features early, enhancing explainability, and making user interface usable.

The Agile framework provided the structural freedom necessary to learn and adapt in real-time. Furthermore, agile iterative nature fits well with the Deep learning and experimentation nature of the project. Each sprint was treated as a small experiment, following agile process of planning, execution, and evaluation which let us quickly test our ideas. For example, we could see how well a new model architecture worked or how well the system adapted to a new sort of manipulation. Instead of discovering a critical flaw late in the process, we could identify what was working, understand what wasn't, and intelligently pivot our strategy. This iterative process proved invaluable, enabling us to respond to emerging research, integrate superior datasets, and consistently refine our approach. [58,59]. Additionally, Agile emphasis on adaptive planning proved crucial for incorporating new datasets, refining convolutional neural network (CNN) architectures, and integrating explainability frameworks such as Grad-CAM. [57]

sprint	Focus	Deliverable	Verification
1	Data & baseline	Data Pipeline, Baseline MobileNetV2 model	Data Quality checks
2	Model Refinement	Fine tune model weights	Accuracy, Precision, F1-score
3	Explainability	Grad-CAM Pipeline	Heatmap review, text correctness test
4	Deployment	Streamlit app	Latency measurements

Table 3.1 : Sprint Plan and Deliverables

3.1.2 Data Collection and Preprocessing

For initial training and validation, we focused on publicly available benchmarks in this section: 140k Real and AI_Generated Faces dataset, which offered a variety of real and AI_Generated image samples annotated for both classification and explainability tasks[47]. When working with image dataset, each image was labeled as real or AI_Generated allowing for supervised learning setup. MobileNetV2 natively is a 2D CNN model that works with images classification, making it a suitable approach. Custom Spatial Rich Model (SRM) natively is a filter layer that works with noise extraction, making it a suitable approach for our two-stream architecture.

The preprocessing pipeline relied heavily on TensorFlow for handling raw data efficiently. The pipeline is optimized for large-scale image handling. Verification of images was applied to ensure image is not corrupted or unreadable samples. TensorFlow was used to preprocess and standardize all images by applying data augmentation to ensure robust training of the model. Data augmentation was made using TensorFlow layers to ensure consistent application of transformations such as horizontal flipping, zoom, and rotation. TensorFlow also remove the need for external tools, loading images directly from directory. After loading, the image is augmented based on random transformations, applying rotation, zoom, or flip differences between frames and ensuring that these features have a varied representation in any input frame which helps CNN, later on, detect real or AI_Generated media. Next, the augmented image is resized to a fixed dimension (224 x 224 pixels) to match the input size of MobileNetV2 model and SRM layer, and kept in [0, 255] scale without normalization to preserve noise features for SRM and stabilize gradient decent updates.

3.1.3 Model architecture and training

Convolutional neural network architecture is used to extract spatial features from facial area to detect subtle anomalies in AI-generated media. The model is implemented using transfer learning based on pre-trained ImageNet architecture, chosen for efficiency, light weight structure, and representation of deepfakes detection task. Additional Convolutional and other connected layers were added and fine-tuned on the architecture to improve the model performance for the chosen datasets. For instance, The MobileNetV2 architecture employs ReLU6 as the activation function for all convolutional layers, and Softmax activation in the output layer. A dropout rate of 0.2 was applied before the classification head to mitigate overfitting. The model was trained using the Adam optimizer with a learning rate scheduler,

with binary cross-entropy serving as a loss function. Batch normalization and dropout layers were used to enhance convergence and reduce overfitting. Each sprint of model development focused on one or more experiments: adjusting learning rates, adding regularization, or unfreezing some pre-trained layers. After each sprint, model checkpoints were saved and compared using key metrics such as accuracy, F1-score, and Area Under the Curve (AUC).

MobileNetV2 was chosen for its efficiency and lightweight size, making it suitable for a quick and simple web interface (Streamlit).. In addition, it ensures low latency with fast inference time enhancing end-user experience. MobileNetV2 is based on Depthwise Separable Convolutions, which allow it to capture spatial sequences. [] In this project, MobileNetV2 architecture not only enabled fast and accurate classification but also played a crucial role in supporting explainability. The modular structure of the network made it easier to integrate with Grad-CAM, as feature maps from intermediate convolutional layers could be directly visualized to explain the reasoning behind model predictions.

3.1.4 Integration of Explainable AI (Grad-CAM Explainability Technique)

Deep learning models face the "Black Box" problem, where decisions can be accurate but can't be

explained. Grad-CAM (Gradient-weighted Class Activation Mapping) solves this by providing visual evidence of the reasoning behind the classification decision, producing heatmaps that highlights the main facial areas that lead to a specific classification. Grad-CAM computes the gradient of the predicted class score AI-generated, with respect to the facial areas of the image. These gradients act as a weights to each feature, where the highest weight is the most influential to the classification and the lowest is the lowest contributor. To sum up, it helps user and developer to make sense of the causes of this specific classification, allowing diagnostics of the model performance and reasoning. [52]. The

heatmaps are expected to focus on the regions where GANs fail to generate realistically such as boundary anomalies, eye glitches, texture manipulations and artifacts.

$$L_{Grad-CAM}^c = ReLU(\sum_k \alpha_k^c A^k)$$

Where, A^k is the activation map of the k^{th} channel in the last convolutional layer, α_k^c is the global

average of the gradient of the class score with respect to that channel. ReLU function made sure that only features with positive influence on the classification process are retained.

The text explanation feature complemented the heatmaps of Grad-CAM by generating a meaningful text that can be easily comprehended by the end-user. This process involved facing segmentation into semantic regions, computing the activation for each region and then normalize it [0,1], applying a threshold based triggers to identify potential anomalies [37], [52]. Based on the detected activation, the system generates a small, comprehensive text such as “High activation found around the right eye

region suggesting possible blending or texture anomalies.”

During each Agile sprint, Grad-CAM outputs were tested using quantitative metrics to ensure that model predictions aligned with human reasoning. If a model focused on irrelevant regions such as background artifacts, the dataset or training pipeline was adjusted to correct bias and improve

interpretability. In other words, Grad-CAM acts as quality control mechanism to detect overfitting, dataset bias, and failure to generalize the model. This iterative loop helped ensure high model performance, transparent decision pathways. Furthermore, the Grad-CAM outputs were used in the final web interface using streamlit, allowing users to visualize the reasoning behind a particular decision.

The text explanation feature complemented the heatmaps of Grad-CAM by generating a meaningful text that can be easily comprehended by the end-user. This process involved facing segmentation into semantic regions, computing the activation for each region and then normalize it [0,1], applying a threshold based triggers to identify potential anomalies. Based on the detected activation, the system generates a small, comprehensive text such as “High activation found around the right eye region suggesting possible blending or texture anomalies.”

3.1.5 User workflow and streamlit Deployment

Streamlit was used for rapid prototyping and its ease of use is the main advantage as it converts a python script in an interactive UI with minimal effort and no complex web development, accelerating user testing and feedback collection on the application which aligns perfectly with the agile nature of this project. Streamlit is an end-user feedback tool, it is mainly used in Measure and adapt phase in the agile methodology, ensuring the explainability feature is continuously refined based on end-user comprehension. The Streamlit interface provided a smooth user experience, allowing users to upload images, trigger automatic detection through the model pipeline, and view results instantly. Upon

image upload, the file passed through all the stages—preprocessing, prediction using MobileNetV2, Grad-CAM generation, and text explanation. The results show the classification results with confidence score, the Grad-CAM heatmap, and the explanation text.

3.1.6 Model evaluation and validation

The evaluation of the model is carried out in both quantitative and qualitative manners. Quantitative evaluation employs standard performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. In addition, visualizations of the ROC curve and the confusion matrix are generated to provide deeper insights into class-wise performance and the trade-off between true positive and false positive rates.

On the qualitative side, visual consistency and user comprehension rating are used as explainability metrics. At the end of each sprint, misclassified samples and misaligned heatmaps are analyzed to identify the root causes of model underperformance, enabling continuous improvement. This closed-loop feedback cycle ensures steady progress and performance refinement throughout the model development lifecycle.

3.2 Tools and Technologies / Data Collection Methods

3.2.1 Python

Python was selected as the primary programming language because of its clear syntax, flexibility, wide ecosystem, and strong support for machine learning and deep learning libraries. A recent survey emphasises that Python has become the language of choice for AI/ML development and education, thanks to its readability and extensive library ecosystem[48] . In our system, Python enabled us to integrate preprocessing, model training, and interface development in one coherent environment helping reduce development overhead and ease collaboration among team members.

3.2.2 TensorFlow / Keras

For training our convolutional neural network (CNN), we used TensorFlow together with the high-level Keras API. Keras allows rapid prototyping of deep learning models while TensorFlow provides reliability and performance in production-like settings [49] .Because these tools are well-established in research and industry, we could benefit from community-tested modules, tutorials, and documentation,

which accelerated our development and enabled smoother fine-tuning of the model for image-based AI-generated content detection.

3.2.3 MobileNetV2

We adopted the pretrained model MobileNetV2 for image classification tasks because of its efficient architecture: it uses inverted residuals and linear bottlenecks to reduce computational cost while maintaining strong accuracy[50]. This made it especially suitable for processing large datasets of images without requiring a high-end server setup, thereby making the system more feasible in practical settings where resources may be limited.

3.2.4 OpenCV (Image-Preprocessing)

In the preprocessing phase, we utilized OpenCV for image resizing, normalization, and data-augmentation operations such as flips, rotations, and color adjustments. Data augmentation, when properly applied, is known to improve generalization of vision models by increasing input diversity. [51]

By preparing the images uniformly (for example resizing to 224×224 pixels) and applying augmentations, we helped the model learn more robust features and reduced the chance of over-fitting or bias toward specific image conditions.

3.2.5 Grad-CAM

To make our CNN model more **explainable**, we integrated the technique Grad-CAM (Gradient-weighted Class Activation Mapping). This method produces heatmaps highlighting the image regions that influenced the model’s decision, thus offering visual justification for outputs and increasing user trust [52]. With Grad-CAM, a user can see *where* in a face image the model is “looking” when it decides whether it is real or manipulated which is crucial when the system may be used in sensitive domains requiring interpretability.

3.2.6 Streamlit

For deploying a user-friendly web application, we chose Streamlit as the frontend framework. Streamlit allows rapid conversion of Python scripts into interactive web apps without deep frontend coding. [53] This meant that testers and stakeholders could upload images, view model predictions and Grad-CAM heatmaps in real time, without needing to write HTML, CSS or JavaScript making the system accessible and reducing the barrier to user adoption.

3.2.7 Google Colab

We selected Google Colab as the development and training environment due to its free GPU access, notebook-style collaboration, and ease of sharing among team members. This environment made it easy to iterate quickly, log experiments, and exchange code. Using Colab helped us reduce infrastructure costs and accelerated experimentation, while keeping the project accessible regardless of individual hardware constraints[54].

3.2.8 “140K Real and Fake Faces” Dataset

For training and evaluation, we used the public dataset titled “140K Real and Fake Faces.” This dataset offers a wide variety of genuine and manipulated face images, enabling the model to learn subtle differences and generalize better. By training on a large, varied set of real vs AI-generated images, our system gained stronger robustness against diverse manipulations and improved its reliability when applied in real-world scenarios [55].

Tool / Technology	Reason for Use
Python	Used as the main programming language because it is simple, flexible, and provides strong support for machine learning and deep learning libraries.
TensorFlow / Keras	Used to train the CNN model (MobileNetV2) for classifying real and AI-generated faces efficiently and accurately.
MobileNetV2	A pretrained CNN model chosen for its lightweight architecture and high performance in image classification tasks.
OpenCV	Used for image preprocessing, resizing, and data augmentation to improve the quality and consistency of input images.
Grad-CAM	An explainability tool integrated to visualize the important image regions that influence the model's decisions.
Streamlit	Used to build a simple and interactive web interface for testing and visualizing model results.
Google Colab	Chosen as the training and development environment for its free GPU support and ease of collaboration.
140K Real and Fake Faces Dataset	A public dataset containing real and manipulated face images, used for model training and evaluation.

Table 7 : Tools and Technologies / Data Collection Methods

3.3 Project Timeline

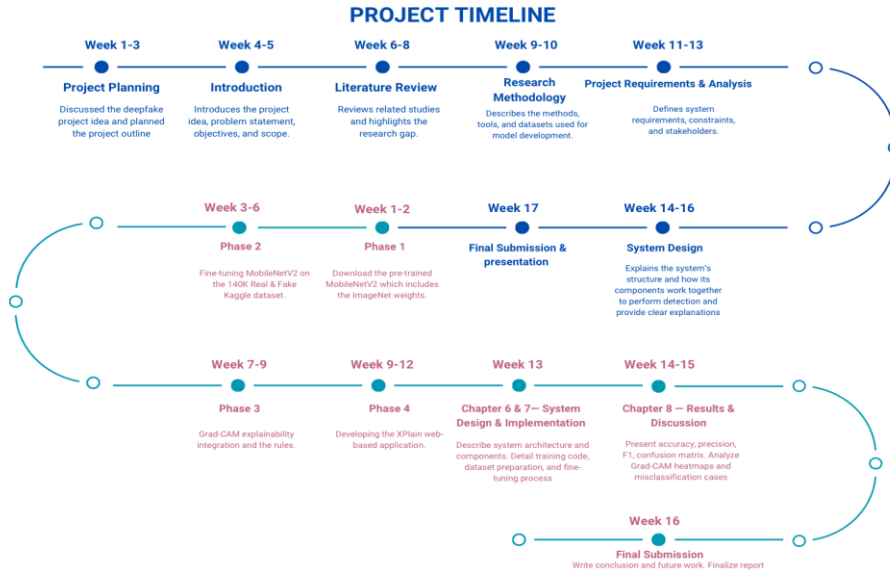


Figure 1: The project timeline from start to final delivery.

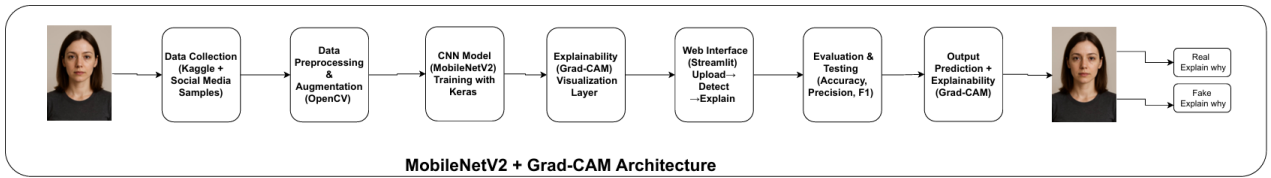


Figure 2: The architecture of the proposed MobileNetV2 with Grad-CAM for model interpretability.

3.4 Risk Management

Effective risk management is essential to ensure the stability, reliability, and ethical compliance of the XPlain system. The complexity of deepfake detection and explainability introduces a variety of technical, data-related, and organizational risks that must be proactively addressed to ensure project success. Table (6) summarizes the key risk categories and corresponding mitigation strategies applied throughout the system's development.

3.4.1 Project Management Risks

Unplanned changes in system requirements such as modifications in explainability visualization techniques, model optimization parameters, or dataset configurations—can lead to project delays and increased development costs. To mitigate these challenges, regular validation, iterative testing, and stakeholder review cycles are incorporated within the Agile methodology adopted for the XPlain system. Each sprint includes risk evaluation checkpoints to verify that project objectives remain aligned with stakeholder expectations and technical feasibility [48].

3.4.2 Technical Risks

Deep learning and explainable AI frameworks often require high computational resources, which may result in training delays or hardware limitations. Additionally, Grad-CAM explainability can occasionally fail to capture subtle artifacts, leading to misinterpretation. To minimize these risks, lightweight architectures such as MobileNetV2 and cloud-based GPU platforms (e.g., Google Colab

pro) are employed to balance performance and efficiency. Furthermore, continuous performance monitoring and code version control (GitHub) ensure reproducibility and prevent critical failures [39][40].

3.4.3 Data and Model Risks

Model performance is highly dependent on data quality and diversity. Insufficient or biased datasets can cause poor generalization and unreliable explanations. To mitigate this, the system integrates multiple benchmark datasets (e.g.,DFDC, Celeb-DF, ExDDV, and 140K Real and Fake Faces) and applies data augmentation, normalization, and cross-validation techniques. These steps ensure dataset robustness and model fairness, reducing the likelihood of overfitting and bias [38][47].

3.4.4 Ethical and Security Risks

As the project involves facial data, there are potential ethical risks regarding user privacy, consent, and data protection. Unauthorized access or dataset misuse could result in ethical or legal violations. To mitigate these risks, all datasets are processed according to FAIR data principles and anonymization standards, ensuring compliance with institutional and international AI governance frameworks [17][19]. Only publicly available or research-approved datasets are used to preserve participant confidentiality.

3.4.5 Maintenance and Scalability Risks

AI models degrade over time as new deepfake generation methods emerge. Without continual updates, detection accuracy may decline. To mitigate this, the project implements continual learning strategies, enabling the model to adapt to evolving forgery techniques. Scheduled retraining, dataset updates, and version control practices ensure that XPlain remains scalable, sustainable, and aligned with emerging AI detection standards [33][39].

Risk Category	Description of Risk	Mitigation Strategy	Reference
Project Management Risks	Unexpected changes in system requirements, visualization methods, or data analysis techniques may cause project delays and cost overruns.	Adopt Agile methodology with iterative validation, sprint reviews, and stakeholder feedback cycles.	[19]
Technical Risks	Deep learning and explainable AI frameworks demand high computational power, possibly leading to performance bottlenecks or model misinterpretation.	Use lightweight architectures (MobileNetV2), cloud-based GPUs (Google Colab), and continuous performance monitoring.	[39][40]
Data and Model Risks	Limited or biased datasets may reduce generalization and cause unreliable explanations.	Combine diverse datasets (DFDC, Celeb-DF, ExDDV, 140K Faces) and apply data augmentation and cross-validation.	[38][47]
Ethical and Security Risks	Handling facial data may raise privacy, consent, or data misuse concerns.	Follow FAIR data principles, anonymize datasets, and use only research-approved public data.	[17][19]
Maintenance and Scalability Risks	New deepfake generation techniques may reduce model effectiveness over time.	Implement continual learning and periodic retraining; maintain version control for scalability.	[33][39]

Table V : Summary of Risks and Mitigation Strategies in the XPlain System

3.5 Quality Assurance Plan / Validation and Reliability

Ensuring the quality and reliability of the XPlain research outcomes is essential in this phase of the project, which focuses on reviewing and analyzing existing explainable deepfake detection techniques. To achieve this, only credible and peer-reviewed academic sources were selected, and findings were cross-checked across multiple studies to ensure accuracy and consistency with the project objectives [6][8].

Reliability of the collected datasets and methodologies is considered by exploring widely used benchmark datasets (e.g., DFDC, Celeb-DF, ExDDV, and 140K Real and Fake Faces) which may be used in the next implementation phase depending on their suitability and performance outcomes. These datasets are common in the domain of deepfake detection and are being evaluated as potential options for future system training and validation [38][47].

All definitions, model descriptions, and evaluation metrics are clearly documented to maintain transparency and allow future replication in the implementation phase [37].

In the next stage of development, additional validation processes will be implemented to measure the performance and trustworthiness of the system. These include evaluating accuracy, precision, recall, and F1-score, as well as using explainability techniques such as Grad-CAM to verify that the model focuses on appropriate facial regions when detecting manipulations [28][52]. Continuous review following the Agile methodology will ensure incremental improvements in system stability and usability [48].

3.6 Summary

This chapter shows the research and technical methodology adopted for developing the XPlain system, a framework based on Explainable Artificial Intelligence (XAI) and Deep Learning (DL) techniques for detecting manipulated facial images distributed on social media platforms. The methodology was designed to ensure transparency, iterative development, and adaptability to emerging deepfake generation techniques [47][48].

The project adopted the Agile methodology to support incremental development and continuous improvement. This approach allowed multiple short development cycles (sprints) covering data collection, image preprocessing, model optimization, and performance evaluation [48][49]. The iterative nature of Agile facilitated early detection and rapid correction of issues, improving model quality and efficiency across all development stages.

In the data collection and preprocessing stage, the “140K Real and AI_Generated Faces” dataset was used for its balanced and diverse set of real and AI_Generated images. Tools such as TensorFlow were employed for data augmentation and resizing to 224×224 pixels, alongside a custom Spatial Rich Model (SRM) filter layer, enabling the two-stream neural network to accurately extract spatial and noise features from facial regions [47][51].

During the model development phase, a two-stream architecture integrating MobileNetV2 and SRM was used for its lightweight architecture and efficiency, based on depth wise separable convolutions and high-frequency noise extraction [50][53]. Transfer learning was employed to accelerate training and improve performance using the Adam optimizer with Binary Cross-Entropy loss, while layers such as Dropout and Batch Normalization were added to reduce overfitting and improve stability.

To address the “black-box” nature of deep learning models, Grad-CAM was integrated to provide visual explanations of decisions by producing heatmaps highlighting the most influential facial regions for classification [52][55]. This technique helped assess model performance, detect potential biases, and ensure attention was focused on relevant areas such as eyes and mouth rather than the background.

The model evaluation involved both quantitative metrics, such as accuracy and F1-score, and qualitative metrics, including the consistency of visual explanations and user comprehension [54]. Periodic analysis of misclassified samples and inaccurate heatmaps supported a closed-loop improvement process.

The chapter also discussed the risk management plan, covering technical, ethical, and data-related risks. Challenges such as limited or biased datasets were mitigated by integrating multiple benchmark datasets (DFDC, Celeb-DF, ExDDV, and 140K Faces) and applying cross-validation techniques [38][47]. Ethical risks were addressed by following FAIR data principles and anonymizing datasets to preserve participant privacy [17][19].

A quality assurance plan was presented to ensure the reliability and credibility of the project outcomes. Only peer-reviewed academic sources and publicly available datasets were used, with thorough documentation of all training and evaluation processes to guarantee reproducibility [6][8][37].

Finally, the project timeline was included to illustrate the stages from initial design to final delivery, reflecting adherence to the development plan and continuous review to ensure project objectives were achieved [33][39].

Overall, this chapter provides a comprehensive methodological foundation for the XPlain system by integrating advanced tools such as Python, TensorFlow, SRM filters, MobileNetV2, Grad-CAM, and Streamlit within a flexible, transparent, and iterative development framework.

This foundation sets the stage for Chapter 4, which focuses on project requirements and system design, enabling a systematic transition from research to practical implementation [47][48].

4 Chapter

Project Requirements & Analysis

4.1 Stakeholder Identification

Identifying stakeholders is an essential step within the XPlain mission—an explainable deepfake detection device—because it clarifies the corporations that gain from or are encouraged by the device and defines the position each performs in its lifecycle. The mission aims to increase a device based on Explainable Artificial Intelligence (XAI) to detect fake pics (deepfakes) while providing transparent reasons for classification consequences. This explainability is critical for building trust among numerous stakeholders.

1. **End Users:** These are people or general internet users who wish to verify the authenticity of pics shared across digital platforms. They interact directly with the device by uploading media to gain a classification result (real or AI-generated) accompanied by both a visual explanation (e.g., heatmap) and a textual explanation that clarifies the reasoning behind the decision. Since these users represent the most interactive group, the device's interface is designed to be simple, intuitive, and accessible, even for non-technical users.
2. **Researchers and Developers:** Researchers utilize the device as an experimental platform to improve deepfake detection algorithms or to study aspects of AI transparency and explainability. Developers, on the other hand, are responsible for maintaining the device and updating models to ensure efficient performance with new data inputs. These individuals represent the technical and scientific core of the mission, contributing to the implementation of advanced models such as MobileNetV2 and the integration of interpretability techniques like Grad-CAM and LIME.
3. **Media and Fact-Checking Organizations:** Media institutions and fact-checking groups rely on the device to verify the credibility of digital content before publication, especially given the rapid spread of misinformation. The device supports ethical journalism by providing a reliable and efficient tool that helps detect visual manipulation, thereby enhancing public trust in published media.
4. **Law Enforcement and Digital Forensics Experts:** This group benefits from the device in criminal investigations or legal cases involving blackmail, identity theft, or media forgery. The device assists in analyzing digital evidence and generates interpretable results that enhance the credibility of findings before judicial authorities. By using explainable AI, the device not only provides results but also justifies the reasoning behind each decision, which is crucial in legal contexts.
5. **System Administrators:** Platform administrators are responsible for operating, monitoring, and maintaining the device's infrastructure to ensure its stability, security, and continuity. Their duties contain performance monitoring, database management, and implementing safeguards

against hacking or data manipulation. This group plays a critical role in maintaining the integrity and reliability of the service.

4.1.1 Assumptions

The improvement of the XPlain machine became guided via way of means of numerous key assumptions that fashioned its layout and implementation. It became assumed that cease customers own a primary stage of technical literacy, allowing them to have interaction with the machine's interface and apprehend its outputs. This assumption allowed the layout crew to simplify the interface in place of such as more than one layers of complicated tutorials or guidance.

Another assumption worried the excellent of enter images, which became anticipated to be enough to appropriately hit upon facial functions and symptoms and symptoms of manipulation. This guided the selection to appoint strong preprocessing equipment which include YOLOv8 (used exclusively for face detection and cropping rather than model training) and OpenCV for picture processing.

Additionally, it became assumed that customers might have get admission to to a solid net connection, because the machine operates thru cloud-primarily based totally structures like Streamlit and Google Colab. This allowed builders to recognition efforts on optimizing cloud overall performance instead of constructing an offline model of the machine.

4.1.2 Constraints

Certain barriers motivated both the improvement method and stakeholder experience. For instance, limited data availability and quality can directly impact the accuracy of detection outcomes delivered to end users. Similarly, restricted computational resources and time constraints affected the speed of model development and experimentation, reducing opportunities for continuous optimization.

Furthermore, the system's dependence on cloud infrastructure and internet connectivity may provide difficulties for media companies or law enforcement agencies working in settings with spotty or restricted access.

Concerning the possible abuse of deepfake datasets and detection technologies, more ethical limitations emerge. Instead of being used to identify them, these databases run the risk of being used to create more lifelike synthetic media. In order to improve ethical transparency and ensure that model decisions are still understandable and responsible, the development team used XAI (Explainable AI) concepts. This transparency reduces the possibility of bias, misuse, or misinterpretation by enabling stakeholders to comprehend how and why a specific detection result was generated.

Therefore, the development team focused on achieving a balance between accuracy, usability, and transparency, ensuring that all stakeholder groups could benefit ethically and effectively from the system's outcomes.

4.2 Methods of Requirements Gathering/ Data Needs and Sources

4.2.1 Data Needs and Sources

Since this project is based on research and development, the requirements gathering phase focused on identifying the types of images and data sources necessary to develop an accurate and explainable model for detecting AI-generated images (Deepfakes). All data used in this project were still images, not video frames, to match the model input requirements and project objectives. The system requires a well-structured and diverse dataset that includes both real and AI-generated facial images to ensure effective training, testing, and validation of the model.

The dataset must consist of high-quality authentic facial images representing real human features, as well as fake images generated using Deepfake or GAN-based techniques to simulate manipulated or synthetic faces. To ensure the model's ability to generalize effectively, the dataset should also include

wide diversity in demographic attributes such as age, gender, and ethnicity, along with various facial expressions and lighting conditions that reflect real-world variations.

To meet these criteria, the project utilized the “140K Real and Fake Faces” public dataset, which contains approximately 140,000 labeled images divided into two categories: real and AI-generated. The real images originate from genuine celebrity photographs, while the AI-generated ones are produced using advanced GAN models such as StyleGAN. This dataset was selected because of its large size, balance, and diversity, which make it ideal for deep learning applications. It enables the model to capture subtle manipulation cues, enhances learning accuracy, and helps prevent overfitting during training [62].

Additionally, small sample images from social media platforms may be included in the validation phase to improve the model’s real-world relevance. These samples help test the model under practical conditions, accounting for varied lighting, camera quality, and facial occlusions commonly found outside curated datasets.

4.2.2 Technical Requirements and Image Processing

The technical requirements of the system are primarily shaped by the two main tools used in its development: MobileNetV2 and Grad-CAM. These tools define the characteristics of the images suitable for the model, as well as the preprocessing procedures required to ensure the highest possible performance.

MobileNetV2 is a lightweight Convolutional Neural Network (CNN) architecture designed for efficient image classification. It performs best with natural RGB images that exhibit clear spatial patterns, making it particularly suitable for facial analysis. To ensure compatibility and consistency, all images—whether real or AI-generated—were resized to 224×224 pixels, normalized according to the model’s input range, and augmented through transformations such as flipping, rotation, and brightness adjustment. These preprocessing steps ensure that all images are standardized and ready for model training and inference, regardless of their original quality or resolution [63].

The Grad-CAM (Gradient-weighted Class Activation Mapping) technique was employed to enhance the model’s explainability by visualizing the areas of each image that most influenced the model’s decision. Grad-CAM outputs are computed specifically from the last convolutional layer (Conv_1) of MobileNetV2, producing a heatmap that highlights image regions contributing most to the classification. This heatmap is then merged with the original image using overlay techniques, providing a visually interpretable result that clarifies the reasoning behind the model’s predictions. In the context of Deepfake detection, Grad-CAM highlights subtle irregularities and unnatural facial patterns that may not be easily perceived by humans [64].

When applied to facial images, Grad-CAM typically emphasizes areas such as the eyes, mouth, and jawline, where blurring, oversharpening, lighting inconsistencies, or irregular textures may occur. By revealing these visually significant areas, the technique contributes to the transparency and trustworthiness of the system, making it especially useful in sensitive fields like media verification and digital forensics [64].

Overall, the dataset and tools employed in this project complement each other to achieve both accuracy and explainability. The 140K Real and Fake Faces dataset provides diverse and high-quality visual data essential for training the model [62]. MobileNetV2 processes these images efficiently, ensuring accurate classification results [63], while Grad-CAM introduces a crucial layer of visual interpretability that clarifies the reasoning behind the model’s predictions [64]. Together, these components create a balanced system that not only detects Deepfakes with high precision but also maintains transparency, an essential requirement for ethical and responsible AI applications.

4.3 Functional Requirements

The main functions of Xplain framework is accurate classification of real and AI-generated images, and a comprehensive explanation of the model reasoning process. Both functions are grounded in state-of-the-art deepfake detection methodologies that emphasize robustness, interpretability, and user trust [1], [3]-[5].

The first major function is prediction, which is responsible for detecting whether an input image is AI-generated or genuine. This function relies on a fine-tuned MobileNetV2 architecture that analyzes facial regions to determine if there exist any inconsistencies introduced by GANs. MobileNetV2 model was chosen due to its accuracy and computational efficiency. Each convolutional block in MobileNetV2 consists of three stages: an initial 1×1 pointwise convolution that expands the feature depth, a depth wise convolution that performs spatial filtering independently per channel, and a final 1×1 projection layer that compresses the dimensionality back to the desired latent space [60], [47]. The model is trained on 140K real and fake faces dataset [47], which is suitable for generalizing the model to detect manipulation across lighting, pose, and compression variation.

Within each block, ReLU6 activation functions are used to prevent loss of information, improving numerical stability during training through regularization algorithm. The final layers include a normalization layer to prevent shifts and accelerate convergences of the model weights. Furthermore, a dropout layers at rate of 0.2 were added during training to reduce overfitting. Finally, 1-unit sigmoid function layer was added to calculate the likelihood of the input image is AI-generated. The resulting binary output ranges between 0,1, representing AI-generated confidence at a threshold of 0.5 to determine the categorical classification. This design enables the system to produce interpretable outputs directly usable for user-level reporting while preserving computational lightness suitable for real-time deployment on low-resource environments such as web or mobile systems [1], [5], [47].

The next main function is Explainability, which addresses the reasoning behind a particular classification done by the model. Using Grad-CAM [52], the system back-propagates the gradient of the output with respect to the feature maps of the convolutional layer. These gradients serve as importance weights, making it easier to interpret which region is responsible for the output classification. The weighted combination of these maps, followed by ReLU activation, produces a heatmap superimposed on the input image to indicate suspicious areas [4], [8]. To complement the heatmap visual explanation, XPlain generates a comprehensive text that summarizes the detected manipulations. The explanation algorithm uses the activations of semantic regions such as eyes, mouth, and hair, normalize them, and applies region-specific thresholds, outputting a structured statement about the reasoning. These thresholds are rule-based and were empirically tuned to capture region-specific activation intensities, ensuring consistent textual explanations across samples. It is important to note that the textual explanations are heuristic, derived from rule-based mapping of visual activations rather than NLP-generated language. The explainability module also functions as a feedback mechanism to researchers and developers, by examining the Grad-CAM output during iterative training cycles, certain biases such as irrelevant pixels' over-attention or background noise can be identified and corrected [4], [15]. This is essential for improving technical accuracy.

Both functions are seamlessly integrated into a Streamlit-based interface that allows users to upload an image, trigger prediction and explainability modules, and instantly receive both outputs: the classification probability and its visual-textual rationale. This interface design mirrors contemporary explainable-AI platforms that combine technical depth with usability [53].

4.4 Non-Functional Requirements

The non-functional requirements define the quality attributes that ensure the system’s effectiveness, reliability, and usability. These include the following aspects:

- **Performance:**
The system should provide fast and accurate results with minimal latency during image analysis. Model inference should run efficiently even on limited hardware resources to support real-time or near real-time detection [50].
- **Usability:**
The user interface must be simple, intuitive, and easy to navigate. Users should be able to upload images and view results — including explanations and Grad-CAM visualizations — without requiring any technical background [53].
- **Security:**
The system must ensure data privacy and integrity. Uploaded images and processed data should not be stored or shared without authorization. The application should also follow ethical AI guidelines and comply with privacy and safety regulations [56].
- **Interpretability / Explainability:**
The system must provide transparency and explainability in its decision-making process. Users should be able to understand why a specific image was classified as real or AI-generated, with clear, interpretable explanations such as visual heatmaps and textual summaries. This enhances trust in the model and ensures its accountability in real-world applications .

4.5 Constraints

Before starting the implementation of the XPlain project, several constraints were identified that might affect how the system is developed, trained, and evaluated. These limitations are common in student-level AI projects and must be considered during the planning and development phases.

1.Limited Hardware and Resources

The project depends mainly on Google Colab, which provides limited GPU and memory capacity. This restricts training large datasets or deep models, and long training sessions may fail or stop because of time or memory limits.

2.Dataset and Data Quality

The datasets used, including 140K Real and Fake Faces, provide a useful variety of samples but do not cover all AI generation methods or lighting conditions. As a result, the model may not perform the same on completely new types of AI-generated images.

3.Data Generalization Limitations

Since the XPlain system learns from the provided training data, it may perform well on familiar deepfake patterns but struggle with unfamiliar ones, limiting generalization to future manipulation techniques.

4.Explainability Accuracy Constraints

The Grad-CAM technique highlights regions of interest in the face for classification, but it may not always localize the manipulated areas precisely. Sometimes it highlights broader regions, which can reduce interpretability for end-users.

5.MobileNetV2 Receptive Field Limitation

MobileNetV2 has a relatively small receptive field, which may cause it to miss subtle details in high-resolution forgeries. Future work could explore hybrid ViT-CNN models to enhance detection of fine-grained manipulation traces.

6.Limited Experience with Advanced AI Tools

As student developers, our experience with advanced explainability tools, such as Grad-CAM, was limited, requiring extra learning and experimentation time.

7.Internet and Platform Dependence

The project relies on cloud tools like Google Colab and Streamlit, requiring stable internet connectivity. Poor connection or session timeouts may interrupt training and testing.

4.6 Summary

This chapter presents the essential requirements and analysis for the development of the XPlain system, an explainable deepfake-detection framework. It begins by identifying key stakeholders, such as end-users, researchers, media verification organisations, and digital forensic teams, each relying on the system to assess image authenticity and understand the reasoning behind predictions. The chapter outlines the data requirements and technical components used to support training and evaluation, including real and manipulated facial images, as well as tools like MobileNetV2, and Grad-CAM to enable efficient classification and interpretability. Functional requirements focus on the ability to upload an image, determine whether it is real or AI-generated, and provide visual and textual explanations. Non-functional requirements address usability, performance, security, and privacy. Finally, the chapter highlights constraints such as limited computational resources, dependence on cloud-based environments, and dataset variability.

Overall, this section establishes a clear foundation for system design, ensuring alignment with accuracy, transparency, and ethical AI considerations. The next chapter will discuss the system design and architecture in detail.

Chapter 5

System Design

5.1 Overview of Design Approach

The design of XPlain was created to be both modular and explainable, making the system organized, flexible, and easy to understand. Each part of the system has its own clear role — from handling the input images and preparing the data, to training the model, detecting AI-generated faces, and finally generating Grad-CAM visual explanations. This structure helps in improving and updating any part of the system without affecting the rest of it.

This design's primary objective is to make XPlain user-friendly and technically effective in identifying phony or altered images. Additionally, it focuses on making the decisions made by the model transparent and comprehensible so that users may not only determine whether an image is authentic or not, but also comprehend the reasoning behind the model's decision. By highlighting the key facial areas that affected the forecast using Grad-CAM, the technique boosts confidence and clarity in the AI's operation.

Preprocessing, model training, detection, and explanation were all precisely specified modules that interacted via well-defined interfaces to achieve the design approach. This guarantees scalability, ease of debugging, and the potential to incorporate future enhancements like more sophisticated DeepFake models or alternative explainability methodologies.

Overall, the system was designed to fit the main purpose of the project promoting explainable AI in DeepFake detection. By combining clear modular design with visual explanations, XPlain provides a balanced approach between technical accuracy and human understanding.

5.2 Data Flow Analysis

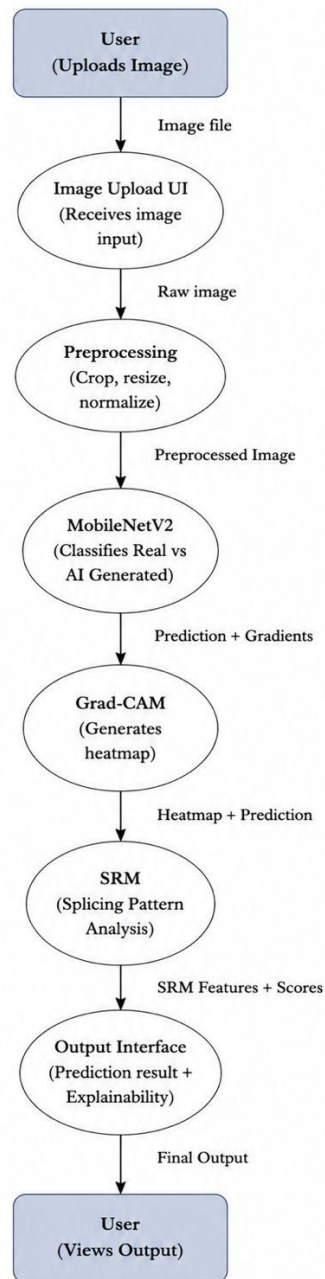


Figure 3: Data Flow Diagram Showing Image Processing Pipeline in XPlain

5.3 XPlain System Architecture

The system starts with the Streamlit Web UI, where the user interacts by uploading an image to begin the analysis. Then, the input is passed to the Application Layer, specifically the Controller, which manages the workflow and coordinates the necessary steps. After that, the Preprocessing pipeline prepares the data by resizing the image to 224x224 pixels and applying normalization. Next, the preprocessed image is passed to the Two-Stream Detection Model—which is maintained within the Application Layer and receives trained weights from the Data & Infrastructure Layer. The Model Serving unit then utilizes the model's output to predict whether the content is real or AI-generated. Meanwhile, an Explainability Engine will use the model's insights to calculate gradients and create a Grad-CAM heatmap along with textual explanations to illustrate the most influential regions for the model's decision and give transparency to the user. Finally, these insights are sent back to the Streamlit Web UI, which displays the final output including the prediction label, confidence score, heatmap, and explanation to the user.

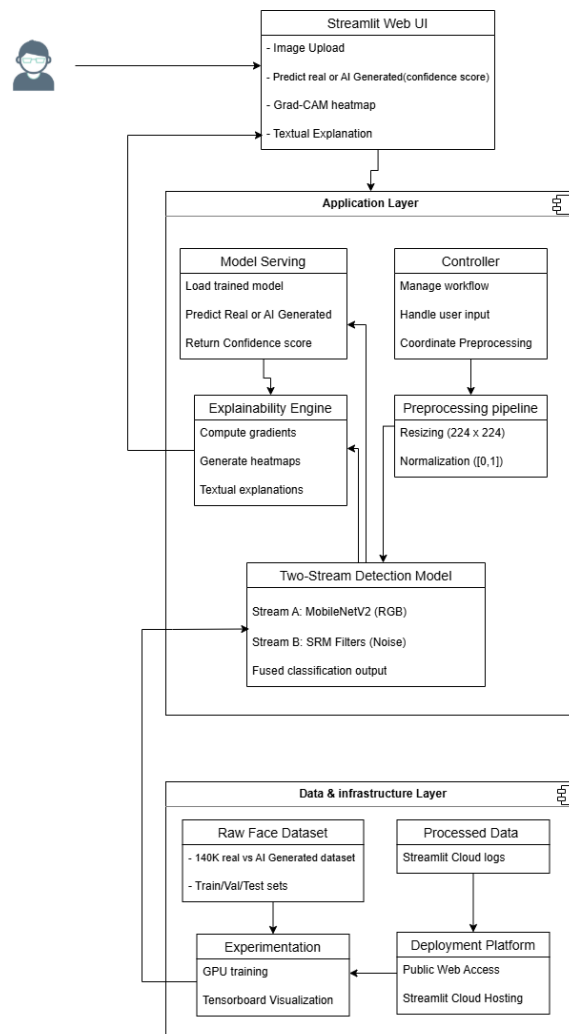


Figure 4: System Architecture

5.4 Use Case Analysis

This section describes what the XPlain system does from the user's point of view. It identifies the main actors who interact with the system and the main actions (use cases) they perform. Each use case represents a specific function that helps achieve the goal of detecting AI-generated images and providing explainable results.

Actors

1. User – Uploads an image to the system, views the classification result, and interprets the Grad-CAM explanation.
2. Developer – Tests and evaluates the model's accuracy and explanation performance, and makes necessary improvements.
3. System Administrator – Manages the system's performance, maintains datasets, and updates or retrains the model when needed.

Use Cases

1. Upload Image – The user uploads an image to the system for analysis.
2. Detect AI-generated Image – The system analyzes the uploaded image and classifies it as real or AI-generated .
3. Generate Explanation (Grad-CAM Visualization) – The system produces a heatmap showing which image regions influenced the classification result.
4. View Results – The user views both the classification result and the Grad-CAM explanation.
5. System Maintenance – The administrator performs updates, monitors logs, and ensures system stability.

5.5 Entity-Relationship (ER) Analysis

The system database is organized into four main entities: **User**, **Image**, **Result**, and **Explanation**. The **User** entity stores essential user information such as the user ID, name, and email, and it is connected to the **Image** entity through a one-to-many relationship, as each user can upload multiple images.

The **Image** entity contains details such as the image ID, the user it belongs to, the file path, and the upload date. Although the diagram shows a one-to-many structure between Image and Result, the system currently generates one result per image, where the **Result** entity stores the classification label and confidence score. Each result is associated with a single **Explanation**, which provides interpretation details such as the heatmap path and the textual explanation.

Together, these entities form a clear data flow that links users to their images, the results produced by the system, and the explanations that describe or visualize the output.

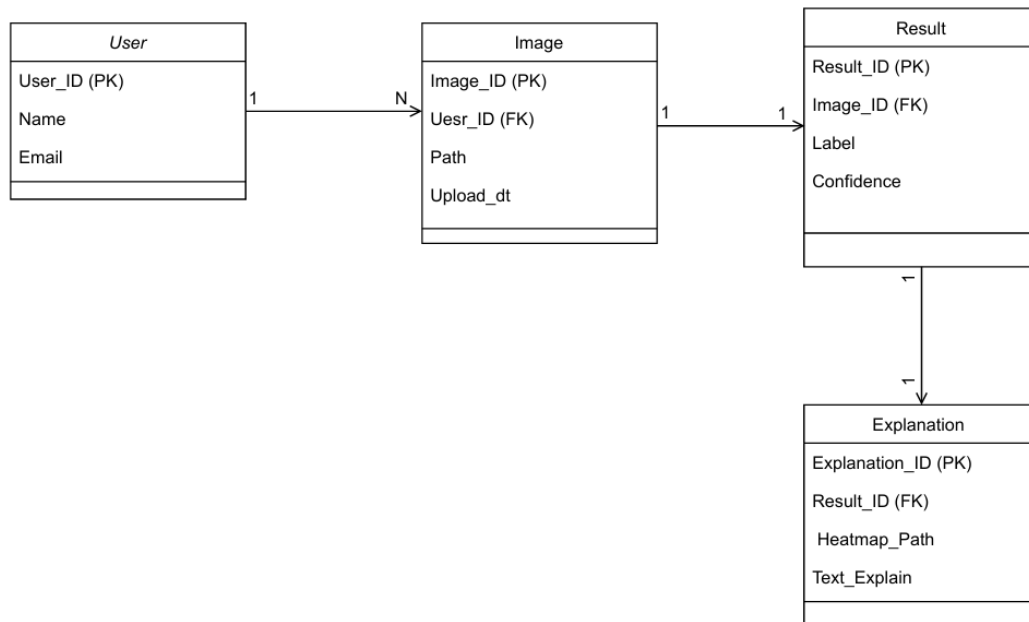


Figure 5: shows the ER diagram of the system.

5.6 System Use Case Diagram

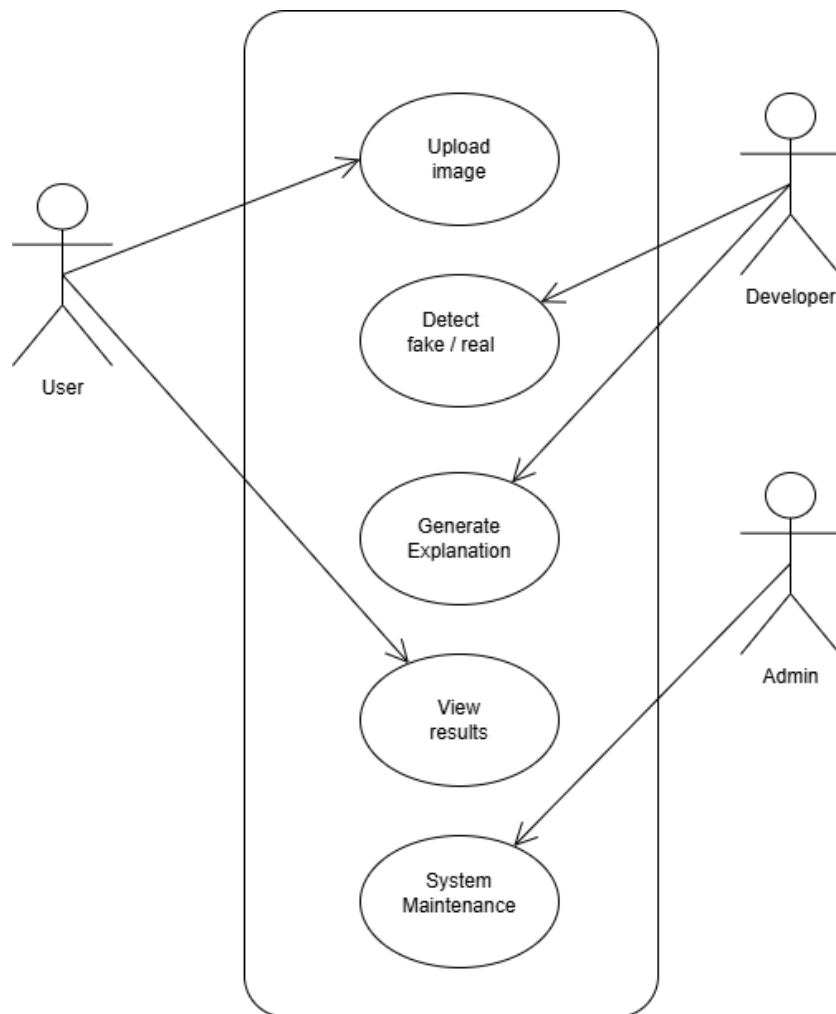


Figure 6: use case diagram

5.7 System Use Case Description

The interaction with the XPlain system begins with the “Upload Image” use case, where the user selects and submits a facial image through the interface. This step serves as the entry point to the entire detection pipeline. Once the image is uploaded, the system validates the file format, ensures that the content is readable, and stores it temporarily for further processing. The user does not need to perform any additional actions at this stage, as the system automatically transitions to the next phase once the image is received successfully.

Following the upload, the “Preprocess Image” use case is activated. In this phase, the system applies a series of automated preprocessing operations to prepare the image for model inference. Specifically, YOLOv8 is used to detect the face within the image, after which the image is cropped to remove background noise. The system then aligns the facial region to correct any tilt or rotation, ensuring consistent positioning of features such as the eyes and mouth. After alignment, the image is resized to

224×224 pixels and normalized to match MobileNetV2 input standards. This preprocessing pipeline ensures that the image is clean, standardized, and optimized for accurate prediction.

Next, the system proceeds to the “Prediction” use case, where the preprocessed image is passed through the trained MobileNetV2 classification model. During this stage, the model evaluates the spatial patterns of the face to detect inconsistencies or artifacts that may indicate manipulation. The output of this step is a probability score ranging between 0 and 1, representing the likelihood that the image is fake. The probability is then mapped to a final binary decision (real or AI-generated) based on a predefined threshold. This prediction serves as the core analytical result of the system.

Once the prediction is generated, the “Generate Grad-CAM Explanation” use case is executed. In this phase, the system computes gradient-weighted class activation maps from the last convolutional layers of MobileNetV2. These maps highlight the facial regions most influential in the model’s decision-making process. The system overlays the resulting heatmap onto the original image to visually illustrate areas showing unusual textures, lighting inconsistencies, or manipulation artifacts. Alongside the heatmap, the system also produces a short textual explanation that interprets the highlighted regions, such as noting whether the model detected suspicious activation around the eyes, mouth, or facial edges.

Finally, the “Display Results” use case presents all outputs to the user in a clear and interpretable format. The interface displays the predicted label (real or AI-generated), the confidence score, the Grad-CAM heatmap overlay, and the generated textual explanation. This final stage ensures that the user not only receives a decision but also understands the justification behind it, supporting transparency and trust in the system. The structured flow from upload to explanation reflects the system’s commitment to accuracy, clarity, and explainability in detecting manipulated facial images.

5.8 System Class Diagram

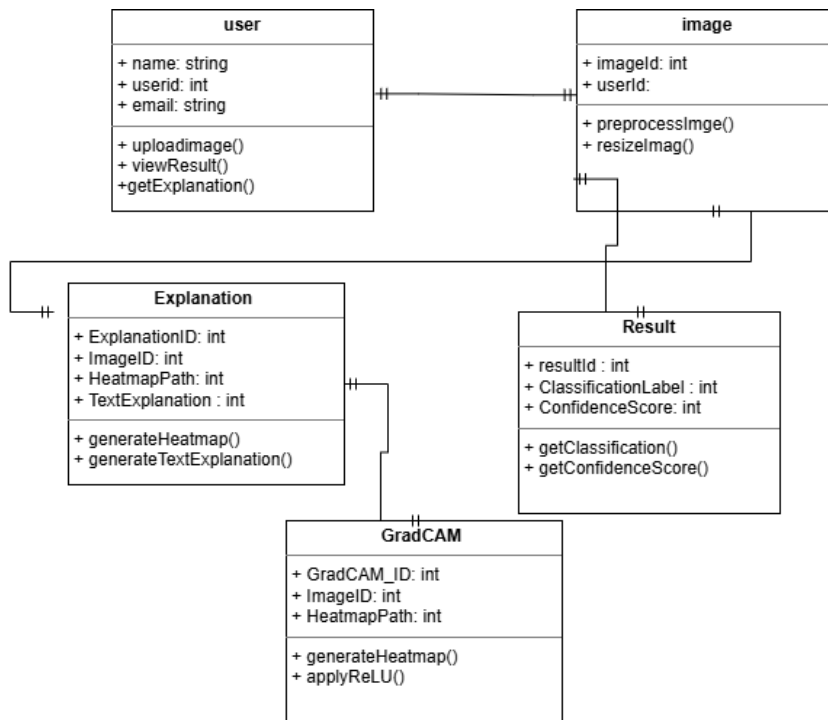


Figure 7: class diagram

The class diagram in Figure (7) outlines the main components of the XPlain system:

- **User Class:** Represents end-users with attributes like name, userid, and email. It includes methods to upload images, view results, and get explanations.
- **Image Class:** Handles image data, with methods for preprocessing and resizing images.
- **Explanation Class:** Provides classification explanations, including methods to generate heatmaps and textual summaries.
- **GradCAM Class:** Generates and applies Grad-CAM heatmaps to visualize key areas influencing classification.
- **Result Class:** Stores the classification results and confidence scores.

These classes work together to ensure accurate deepfake detection and provide transparent explanations for the system's decisions.

5.9 XPlain System Interaction Diagram

This sequence diagram illustrates the complete workflow of the XPlain Framework, starting from the user interaction and ending with the generation of an explainable prediction. When a user uploads a picture via the Streamlit UI, the image and its information are sent to the API, starting the process.

Through the Preprocessing Module, the API carries out preprocessing and validation, including 224x224 scaling, and normalization. The MobileNetV2 Model receives the resulting preprocessed tensor and generates a prediction probability that indicates whether the image is authentic or fraudulent. The method uses Grad-CAM on the last convolutional layer to produce an upscaled heatmap that emphasizes the areas affecting the model's choice in order to provide interpretability.

After analyzing the heatmap, the Text Explanation Module produces a written explanation outlining the model's logic. The Streamlit interface shows the user the final response that the API provides, which includes the label, confidence score, heatmap, and explanation. The user has the option to provide input, which is saved in the Storage module to help with future model enhancements.

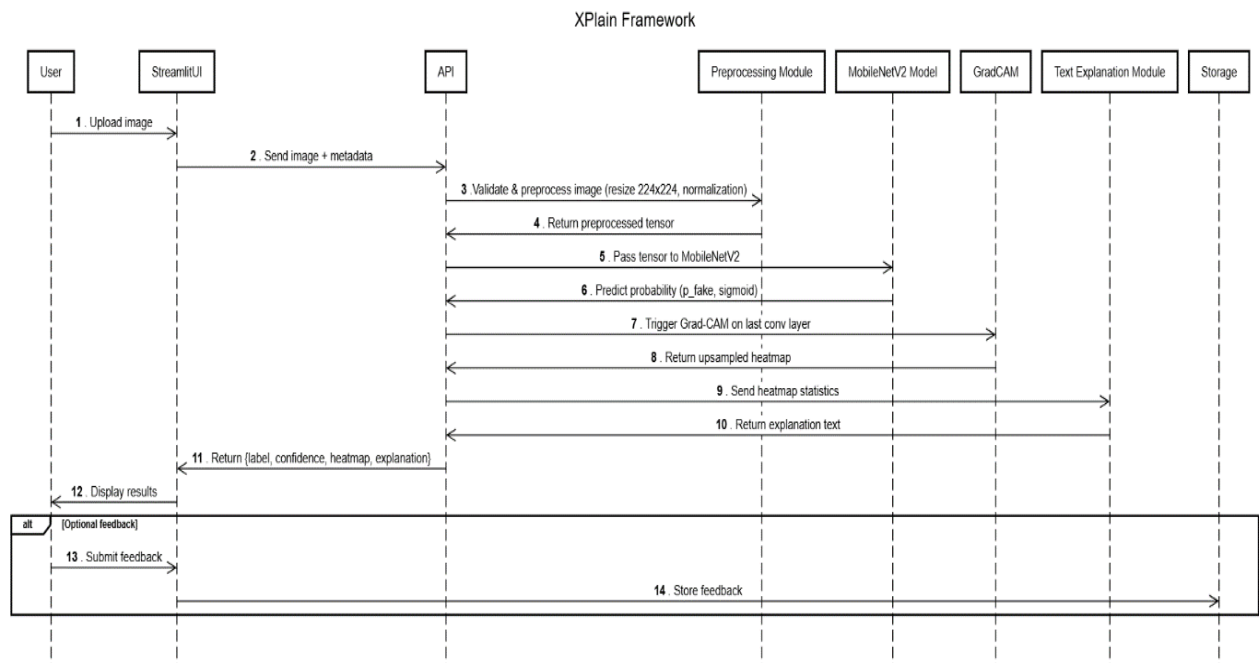


Figure 8: System interaction sequence diagram

5.10 System Activity Diagram

The system starts when the user enters the application, as shown in the flowchart with the word **“Start”**. Then, the user is asked to **“Upload Image”** to provide a face image for analysis. After that, the uploaded image goes through a preprocessing stage labeled **“Preprocessing (Face detection, cropping, resizing)”**, where the face is detected, cropped, and resized to match the model’s input requirements. Next, the processed face is sent to the deep learning classifier, which is represented by **“MobileNetV2 Prediction”**, to decide whether the face is real or AI-generated. To make the prediction explainable, the system generates a visual interpretation using **“Grad-CAM Heatmap Generation”**, highlighting the important facial regions that influenced the decision. This is followed by **“Text Explanation”**, where the system produces a short justification describing the reason behind the decision. Finally, the user can view all results under **“Display Results (Label + Accuracy + Heatmap + Explanation)”**, which is about the classification label, accuracy score, heatmap, and explanation. Based on the classification, the system will show either **“Real”** or **“AI-generated”**, and the user can click on **“Why Real?”** or **“Why AI-generated?”** to understand more details. At the end of the process, the flow concludes with **“End”**.

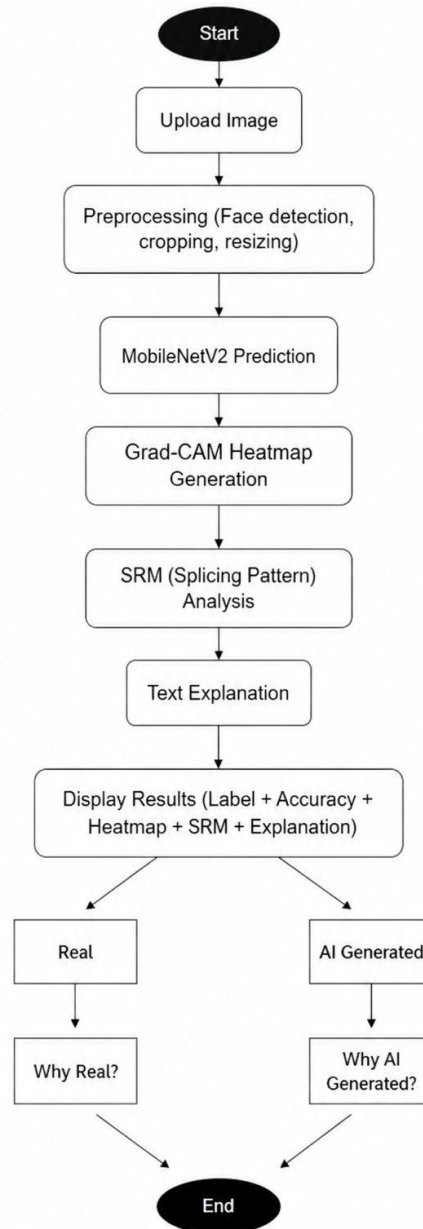


Figure 9: System Activity Diagram

5.11 Front-End / User Interface Design

The graphic depicts the user interface of Xplain Detect, a technology that uses explainable visual feedback and artificial intelligence to detect deepfakes and altered photos. Users can choose an image using the Select Image button or just drag and drop it into the upload section of the main screen. The system shows the analysis findings, including the classification result and the confidence score, on the right side of the interface after an image is submitted. The model's ability to accurately determine the possibility of manipulation is seen in the example where the algorithm identifies the image as "AI-generated " with a confidence level of 92.4%.

The original image and a heatmap visualization that highlights the regions the model concentrated on during detection are displayed on the left side of the interface. The explanation section offers thorough

insights into important variables affecting the outcome, including:

- elevated activation in the vicinity of the eyes.
- irregularities in the mouth's boundaries.

irregular lighting patterns that could indicate manipulation.

- Two primary user options are also included in the interface:
- To upload and examine a different image, select Analyze New Image.

Get a comprehensive report that summarizes the analysis and justifications by downloading the report.

All things considered, the system provides an engaging and comprehensible user experience that increases transparency and confidence in the analysis results by allowing users to not only identify whether an image is false but also comprehend the precise rationale behind the categorization.

Additionally, Canva was also used to create this mockup, which depicts the anticipated interface layout. To give actual, useful deepfake detection capabilities, the system will be constructed using Streamlit—or similar intelligent framework—integrated with Python in the upcoming semester.

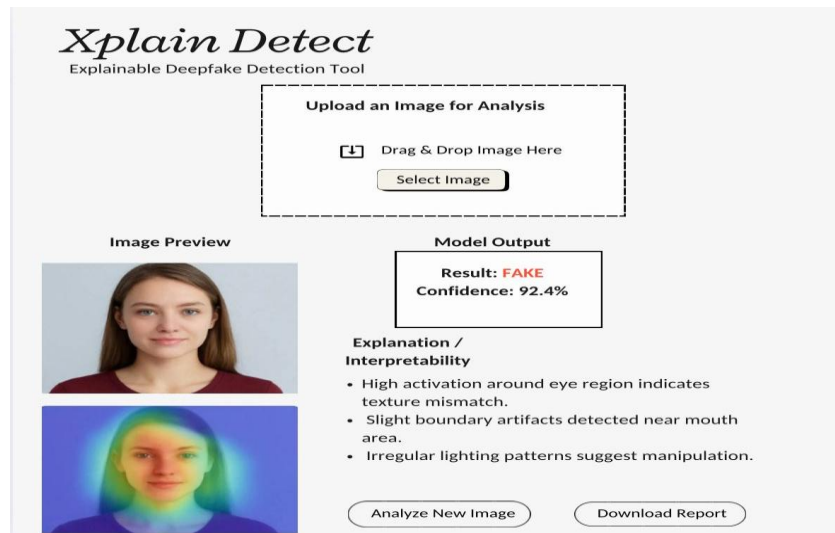


Figure 10: Interface Design

5.12 Database / Data Model Design

The four primary entities in the XPlain database are User, Image, Result, and Explanation. Each image has exactly one detection result (1:1), although each user may upload more than one image (1:Many). Each outcome is connected to a single explanation that emphasizes the factors affecting the model's choice.

The User entity stores basic user information. The Image entity provides image data such as file path and upload date. The Result entity keeps the classification output and confidence score, and the Explanation entity stores the Grad-CAM heatmap and the related written explanation.

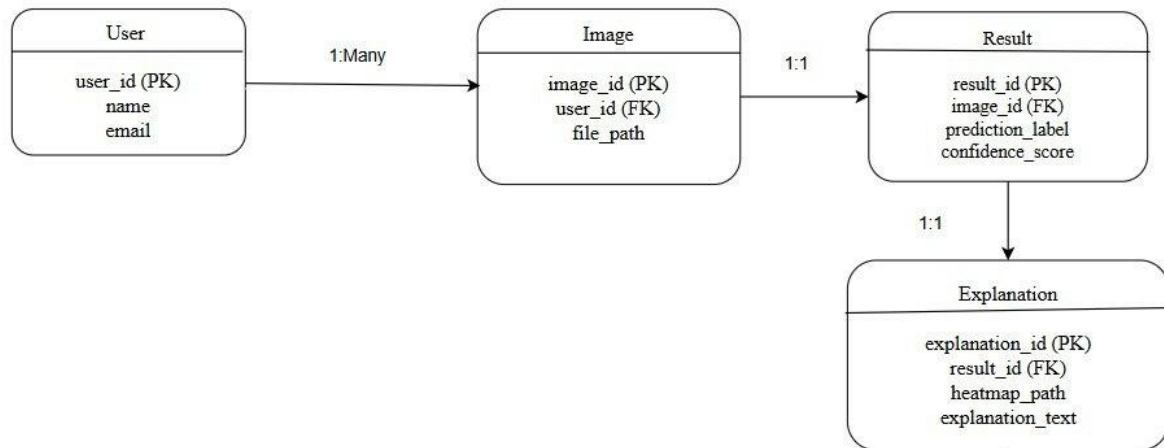


Figure 11: Database Diagram

5.13 Algorithm / Process Design

The system starts when the user enters the application, as indicated by the “**Start**” node in the flowchart. The user is first required to upload an image through the “**Upload Image**” step, after which the system checks whether the image is loaded successfully; if the load fails, the process stops. Once the image is successfully loaded, it is passed to the “**Image Preprocessing**” stage, where necessary preprocessing operations are applied. The preprocessed image is then sent to the deepfake detection model under “**Deepfake Classification Using MobileNetV2**” to classify the image as either real or AI-generated . To make the prediction explainable, the system generates a visual explanation using “**Grad-CAM Explanation**,” highlighting the important regions that influenced the decision. Based on the classification result, the system outputs either a “**Real Image**” or a “**AI-generated Image**,” and the user can select “**Why Real?**” or “**Why AI-generated ?**” to receive further clarification supported by the “**Knowledge Graph (XAI)**.” After displaying the results and explanations, the system saves the output in the “**Save Results**” step, and the workflow concludes with “**End**.”

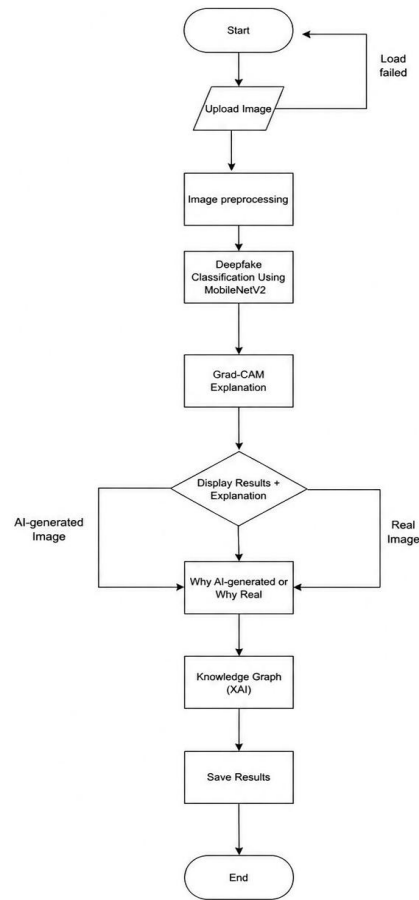


Figure 12: XPlain Algorithm

5.14 Summary

Chapter 5 discusses the general architectural and structural architecture for the XPlain system, an explainable deepfake detection framework that merges deep learning with explainable artificial intelligence (XAI) methodologies. The chapter covers how the system is built, how data flows through its components, and how each module helps to attaining both accurate categorization and straightforward interpretability.

The chapter begins by outlining the design strategy, which utilizes a modular architecture to enable flexibility, maintainability, and scalability. Image preprocessing, the Grad-CAM explainability module, the MobileNetV2 classification model, and the user interface are all designed as separate components with distinct roles. Because of its modularity, individual components can be easily improved without affecting the system as a whole.

A thorough data flow analysis is presented, describing the sequence of activities conducted on an input image: uploading through the user interface, face identification and alignment during preprocessing, classification using MobileNetV2, and creation of Grad-CAM visual explanations. This flow ensures that the user receives both the classification label (genuine or AI-generated) and the related interpretability output that indicates the influential facial regions in the decision-making

process.

The system architecture section offers a multi-tier design that includes the front-end interface, the deep learning engine, the explainability layer, and the data management component. The architecture depicts how various layers interact cohesively to allow efficient detection and interpretable outputs

A thorough Use Case Analysis is also included in the chapter, which identifies the main actors—users, developers, and system administrators—and details how they interact with the system, including uploading images, starting deepfake detection, viewing results, and carrying out system maintenance. Key data pieces (User, Image, Result, and Explanation) are organized and connected within the system's database, as further explained by the Entity-Relationship (ER) analysis.

To give a visual depiction of system behavior and the relationships between its internal components, a number of UML diagrams are presented, such as use case diagrams, class diagrams, interaction diagrams, and activity diagrams.

The chapter also examines the front-end interface design, stressing usability and clarity through a streamlined layout designed using Streamlit. The database design and algorithmic processes—particularly the usage of MobileNetV2 for classification and Grad-CAM for interpretability—are explained to demonstrate how the system achieves both performance and transparency.

Finally, the core design architecture required to implement the XPlain system is established in Chapter 5. It successfully connects the practical growth stages that will be expounded upon in Chapter 6 with the conceptual research offered in previous chapters.

Chapter 6

Implementation / Experimentation

6.1 Implementation Plan

The system was developed following a structured and phased engineering approach, starting from the initial coding stage to the final deployment and testing. The implementation was carried out using the Google Colab environment, which provides access to Tesla T4 GPU acceleration. This significantly improved the efficiency of training deep learning models, especially when working with a large-scale dataset of approximately 140,000 images.

MobileNetV2 Architecture :

MobileNetV2 is a convolutional neural network (CNN) architecture designed for efficient and lightweight image classification tasks. It introduces advanced concepts such as depthwise separable convolutions, inverted residual blocks, and linear bottlenecks, which significantly reduce computational cost while maintaining high accuracy.

Depthwise separable convolutions split the convolution process into depthwise and pointwise operations, reducing the number of parameters and improving efficiency. The inverted residual structure helps preserve important features while reducing memory usage. These properties make MobileNetV2 suitable for large-scale and real-time applications.

In this project, MobileNetV2 was initialized with pretrained ImageNet weights, meaning it already learned general visual features such as edges, textures, and shapes. The top classification layers were removed, and a custom classification head was added to adapt the model for binary classification (Real vs Ai generated).

MobileNetV2 provides a balance between computational efficiency and strong feature extraction capability, making it ideal for deepfake detection tasks.

A Transfer Learning strategy was adopted using the MobileNetV2 architecture pre-trained on the ImageNet dataset. This approach allows the model to leverage previously learned knowledge and adapt it to the deepfake detection task, reducing training time and improving performance.

ImageNet Dataset (Pretraining Knowledge)

ImageNet is a large-scale visual dataset containing millions of labeled images across thousands of categories. It is widely used for pretraining deep learning models.

When MobileNetV2 is trained on ImageNet:

- It learns general visual patterns
- It recognizes edges, textures, and object structures
- It develops strong feature extraction abilities

These learned features are transferred to the deepfake detection task in this project.

ImageNet pretraining improves model performance, reduces training time, and enhances generalization ability.

Dataset Description (Deepfake Dataset Used in the Project)

The dataset used in this project plays a crucial role in training and evaluating the deepfake detection system. It consists of approximately 140,000 facial images collected from publicly available sources such as Kaggle datasets. The dataset is specifically designed for binary classification tasks involving facial image authenticity detection. The dataset contains two main categories: Real Images, which represent authentic human faces captured from real datasets or recordings, and AI-generated images (Deepfakes), which are artificially generated or manipulated using deep learning techniques such as GANs (Generative Adversarial Networks).

The dataset was carefully prepared and organized into three main subsets. The Training Set was used to train the model and learn patterns from data. The Validation Set was used to monitor model performance during training and prevent overfitting. Finally, the Test Set was used for final evaluation and explainability analysis using Grad-CAM. Before training, all images were preprocessed to ensure consistency. Each image was resized to $(224 \times 224 \times 3)$ to match the input requirements of MobileNetV2, and pixel values were normalized to improve numerical stability during training.

Data augmentation techniques were also applied to increase dataset diversity. These included rotation, flipping, and minor transformations, which help the model generalize better to unseen data. So, The dataset is a critical component of this project because it directly influences model performance. A well-balanced and properly processed dataset improves accuracy, reduces overfitting, and ensures reliable deepfake detection.

Step-Based Implementation Workflow

Step 1: Feature Extraction using Pretrained MobileNetV2

The MobileNetV2 model was used as a feature extractor with frozen weights from ImageNet. It extracts important visual features such as edges, textures, and facial structures to reduce training time and improve feature quality.

Step 2: Fine-Tuning using Real & Fake Dataset

The model was fine-tuned using the deepfake dataset by unfreezing selected layers and training on labeled images. Training included both real images and AI-generated images. The model was then evaluated using several performance metrics, namely Accuracy, Precision, and F1-score.

Step 3: Grad-CAM Explainability

Grad-CAM was applied to visualize model decisions by highlighting important image regions using gradient-based activation mapping to provide transparency and interpretability for model predictions.

6.2 Module / Process Development

The system was implemented as modular components, each responsible for a specific function in the pipeline. The preprocessing module standardizes input images using resizing, normalization, and tensor conversion. This ensures all images follow a consistent format required by MobileNetV2.

The augmentation module increases dataset diversity using transformations such as rotation and flipping. This improves model generalization and reduces overfitting. The feature extraction module relies on MobileNetV2, which extracts hierarchical visual features from images.

A SRM (Spatial Rich Model) module was also integrated to extract forensic noise patterns, including pixel inconsistencies and compression artifacts. This creates a Two-Stream Architecture combining RGB visual features and SRM forensic features.

6.2.1 Deterministic Forensic Noise Extraction (SRM Filters)

Generative models, including both GANs and diffusion architectures, leave high-frequency artifacts in the images they produce. These show up at the pixel level as checkerboard patterns, a byproduct of upsampling layers that fail to maintain consistent sub-pixel alignment. Standard CNNs miss them because they are built to do the opposite: pool spatial information and compress it into semantic representations of shapes and textures. To address this, we implemented a custom SRMFilterLayer (Spatial Rich Model) at the input stage.

Rather than letting the network learn what to detect, we fixed the filters mathematically. The layer applies three 3×3 convolution matrices directly to the raw image tensor, discarding visual content and retaining only the pixel-level noise:

- **Filter 1 (first-order horizontal):** Computes horizontal pixel gradients, revealing unnatural transitions that occur when a generator misaligns adjacent pixel grids.
- **Filter 2 (second-order Laplacian):** A center-surround filter ($[1, -4, 1]$) that responds to high-frequency, multi-directional noise across neighboring pixels.

```
class SRMFilterLayer(tf.keras.layers.Layer):
    def __init__(self, **kwargs):
        super(SRMFilterLayer, self).__init__(**kwargs)

    def build(self, input_shape):
        #Filter 1: First-order horizontal edge detector
        filter_1 = np.array([
            [ 0,  0,  0],
            [-1,  1,  0],
            [ 0,  0,  0]
        ], dtype=np.float32)

        #Filter 2: Second-order Laplacian
        filter_2 = np.array([
            [ 0,  1,  0],
            [ 1, -4,  1],
            [ 0,  1,  0]
        ], dtype=np.float32)

        # Filter 3: Third-order SQUARE 5x5 high-pass (padded)
        # We use 3x3 for consistency this is a diagonal Laplacian variant
        filter_3 = np.array([
            [-1,  2, -1],
            [ 2, -4,  2],
            [-1,  2, -1]
        ], dtype=np.float32)
```

Figure 13: Implementation of the Spatial Rich Model layer for noise extraction

- **Filter 3 (third-order high-pass):** A diagonal variant sensitive to blending artifacts at the boundaries where a generated mask meets real facial structure.

```
self.srm_kernel = self.add_weight(
    name='srm_filters',
    shape=kernel.shape,
    initializer=tf.keras.initializers.Constant(kernel),
    trainable=False # CRITICAL: These filters must NOT be trained
```

Figure 14 : Initialization of SRM weights

The weights are frozen (`trainable=False`). This decision matters more than it might appear. Gradient-based optimizers find the easiest path to reduce loss, and color gradients are considerably easier to classify than forensic noise. Without this constraint, backpropagation would gradually push these filters back toward detecting visual features. Freezing them keeps the layer focused on noise regardless of what the training signal would otherwise prefer.

6.2.2 Dual-Topology Processing and Cross-Modal Fusion

A single-path network cannot simultaneously optimize for macro-level appearance and micro-level noise. These two objectives compete during training, and the result is always a compromise. The architecture addresses this by splitting into two parallel, independent streams.

```
#1
noise = layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='srm_conv1')(noise)
noise = layers.BatchNormalization(name='srm_bn1')(noise)
noise = layers.MaxPooling2D((2, 2), name='srm_pool1')(noise)

#2
noise = layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='srm_conv2')(noise)
noise = layers.BatchNormalization(name='srm_bn2')(noise)
noise = layers.MaxPooling2D((2, 2), name='srm_pool2')(noise)

#3
noise = layers.Conv2D(128, (3, 3), activation='relu', padding='same', name='srm_conv3')(noise)
noise = layers.BatchNormalization(name='srm_bn3')(noise)
noise = layers.GlobalAveragePooling2D(name='srm_gap')(noise)
```

Figure 15: Feature extraction layers for the SRM stream

Stream A feeds the input image through MobileNetV2, which acts as the semantic backbone. The output is a (None, 1280) feature vector encoding the structural and physical appearance of the face.

Stream B takes the SRM noise map through a dedicated CNN with three sequential convolutional blocks (32, 64, and 128 filters). Batch Normalization is applied after each convolution because SRM output has high and erratic variance. Without normalizing the distributions, gradient updates become unstable. Max Pooling reduces spatial dimensions at each stage, and a final Global Average Pooling layer produces a (None, 128) noise signature.

```
# Output: (batch, 128)
merged = layers.Concatenate(name='stream_merge')([stream_a, noise])

x = layers.Dense(256, activation='relu', name='head_dense')(merged)
x = layers.BatchNormalization(name='head_bn')(x)
x = layers.Dropout(0.4, name='head_dropout')(x)
output = layers.Dense(1, activation='sigmoid', name='head_output')(x)
```

Figure 16 : Two-stream fusion and output layers

A Concatenate layer merges both outputs into a (None, 1408) tensor. This passes through a Dense(256) bottleneck that maps visual traits against corresponding noise patterns, rather than treating the two streams as independent votes. Dropout(0.4) randomly disables 40% of neurons per batch, reducing the model's tendency to memorize co-occurrence patterns specific to the training set. A single-node Dense layer with sigmoid activation then converts the unbounded logit into the final [0.0, 1.0] classification probability.

The classification head consists of GlobalAveragePooling, Dense (512 neurons), Dropout (0.4), and BatchNormalization layers. These components improve classification accuracy and training stability.

The Grad-CAM module provides explainability by generating heatmaps that highlight the most influential regions in the image. Modular design improves system scalability, maintainability, and performance optimization

6.3 Integration of Components

All modules were integrated into a unified pipeline to ensure smooth data flow from input to output. The system starts with image preprocessing, followed by feature extraction using both RGB and SRM streams. These features are then fused and passed to the classification head, producing a probability score between 0 and 1.

A major challenge during integration was the nested structure of MobileNetV2 inside the final model, which made accessing internal layers difficult for Grad-CAM. This was solved using a dynamic layer search mechanism.

The final system flow begins with the input image, followed by preprocessing, then feature extraction using both RGB and SRM techniques, and finally classification. Grad-CAM visualization integration ensures that all components work together efficiently and produce consistent and reliable results.

6.5 Deployment or Execution

The system was developed and executed in Google Colab to simplify testing and model experimentation. The code starts by loading the input image, then applying preprocessing steps such as resizing, normalization, and tensor conversion so it becomes suitable for the trained model. After that, the image is passed through the model to perform inference and produce a prediction (real or Ai generated).

In the next step, the code applies Grad-CAM to extract the important regions that influenced the model's decision. This is implemented by computing gradients from the final convolution layers and generating a heatmap that highlights high-attention areas in the image. The heatmap is then resized to match the original image size.

Finally, the code visualizes the results by displaying three outputs: the original image, the generated heatmap, and an overlay image that combines both for clearer interpretation. This helps in understanding not only the prediction result but also the reasoning behind it in a visual form.

The same code structure can be easily adapted for deployment in a Streamlit application or backend API by wrapping the preprocessing, inference, and Grad-CAM functions into callable modules.

6.5.1 ROC Curve

To evaluate the classification performance of the XPlain framework, the Receiver Operating Characteristic (ROC) curve was employed as a key metric. Figure X illustrates the ROC curve obtained after training the MobileNetV2 model on the "140K Real and Fake Faces" dataset, providing a comprehensive visualization of the model's ability to distinguish between real and AI-generated images across all classification thresholds.

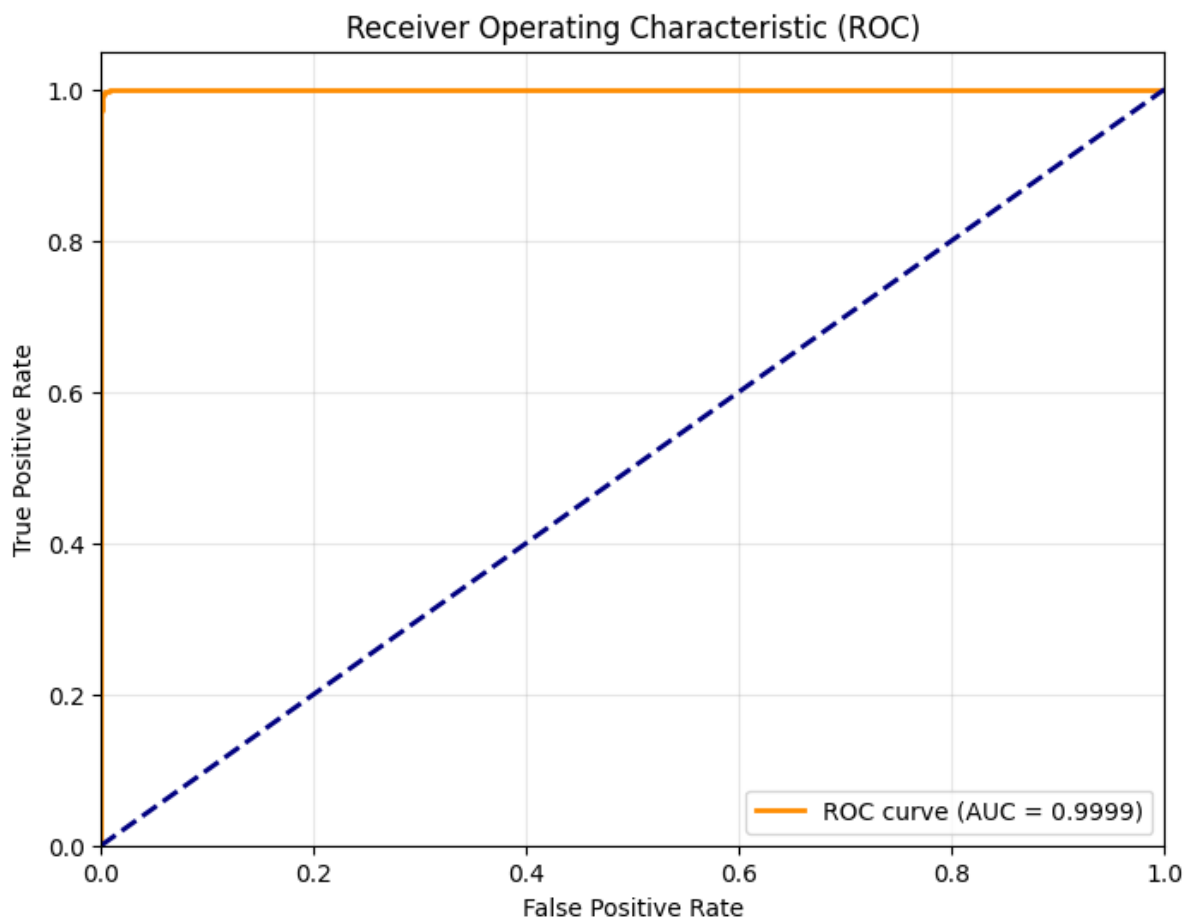


Figure 17 : ROC

Figure shows the Receiver Operating Characteristic (ROC) curve used to evaluate the classification performance of the **MobileNetV2** model trained within the XPlain framework for distinguishing between real images and AI-generated images. The horizontal axis represents the False Positive Rate, while the vertical axis represents the True Positive Rate (Sensitivity).

The figure presents two curves: the first is the ROC curve of the proposed model (orange line), and the second is the reference baseline representing a random classifier (blue dashed line) with an area under the curve equal to 0.5.

The **MobileNetV2** model achieved an Area Under the Curve of (**AUC = 0.9999**), approaching the theoretical maximum of 1.0, which demonstrates the model's exceptional ability to distinguish between real and fake images with near-perfect accuracy. This is clearly reflected by the curve's sharp rise toward the upper-left corner of the plot, indicating a significantly high true positive rate alongside a near-zero false positive rate.

This result confirms the high efficiency of the XPlain framework in detecting AI-generated images, leveraging **MobileNetV2**'s ability to extract fine-grained visual features from the "**140K Real and Fake Faces**" dataset, making it a reliable and robust system for combating the growing challenges posed by deepfake imagery.

6.6 User Interface

The user interface of the XPlain system is designed to provide a simple and interactive environment for users to test the deepfake detection model. This chapter presents the design and structure of the interface, including the main components and how users interact with the system. The interface focuses on usability and clarity, allowing both technical and non-technical users to easily upload images, view predictions, and understand the results.

6.6.1 Interface Overview

The interface is organized into multiple sections, including the main dashboard, image analysis module, and model performance dashboard. A sidebar navigation panel is provided to allow users to switch between these sections easily. This design ensures a structured workflow and enhances usability for both technical and non-technical users without requiring expertise.

The sidebar navigation panel, illustrated in Figure (6.5.1), allows users to move between different sections of the application. These include Home, Image Analysis, Model Dashboard, and Architecture. This modular navigation design improves accessibility and ensures that each component of the system can be accessed independently.

6.6.2 Dashboard Features

The dashboard features section, as shown in Figure (6.6.1), presents the main functionalities of the XPlain system in a clear and simplified way. It highlights key services such as image prediction, confidence score display, Grad-CAM visualization, and textual explanation of the results.

The dashboard is designed to make the system easy to use by presenting outputs in a visual and understandable format. It may also include simple statistical representations, such as bar charts or pie

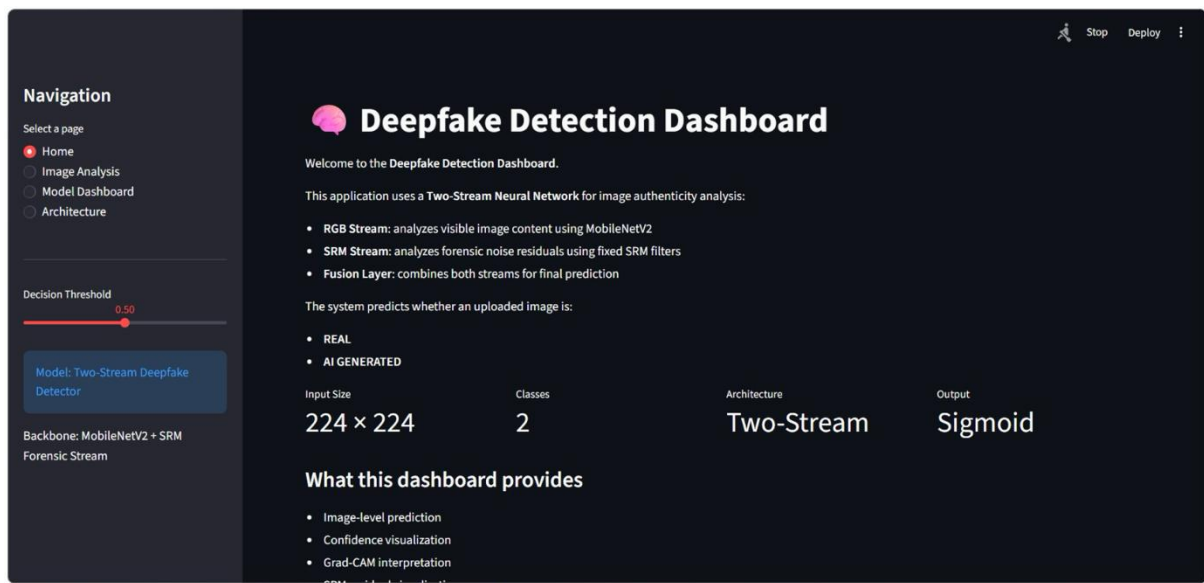


Figure 18 : XPlain dashboard showing system features and usage guidance

charts, to summarize prediction results. This helps users quickly understand the system behavior without needing technical knowledge.

6.6.3 Image Analysis Interface

The image analysis interface, as shown in Figure (6.6.2), allows users to upload an image for deepfake detection. The interface is simple and user-friendly, supporting different image formats. Once the image is uploaded, the system prepares it automatically for processing. The layout focuses on clarity and ease of interaction, allowing users to use the system efficiently without confusion.

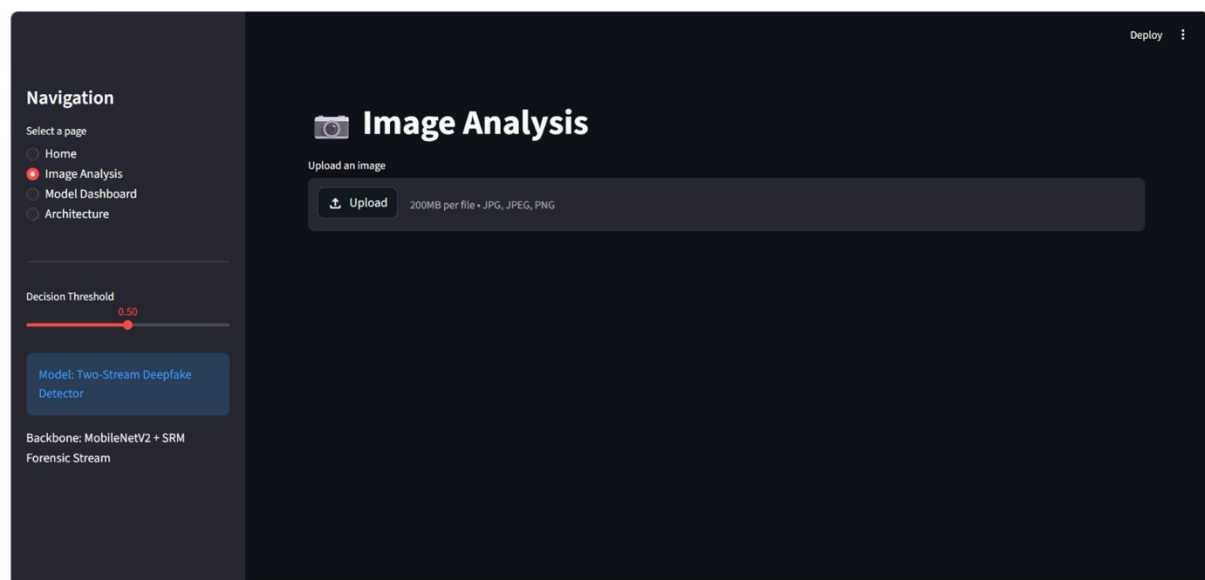


Figure 19: Image upload interface for deepfake detection

Prediction and Explainability Output

After uploading the image, the system generates a prediction indicating whether the image is real or AI-generated. A confidence score is also displayed to reflect the certainty of the model.

To improve interpretability, Grad-CAM is applied to produce a heatmap highlighting the important regions influencing the model's decision. The heatmap is overlaid on the original image, helping users visually understand the prediction. In addition, a short textual explanation is provided to clarify the possible reason behind the result.

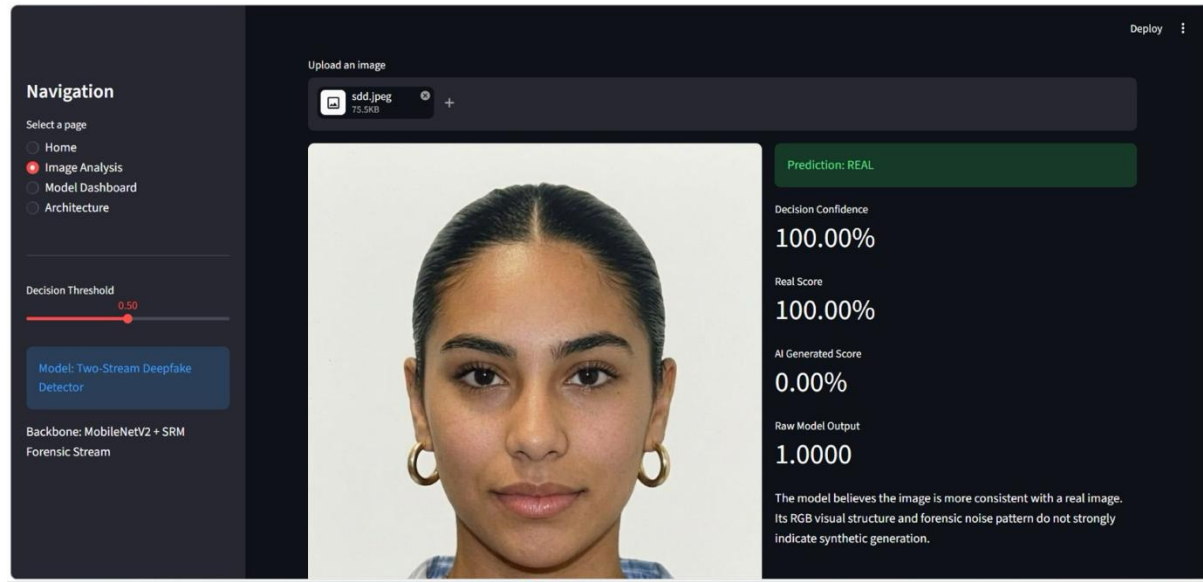


Figure 20: Prediction result with confidence score

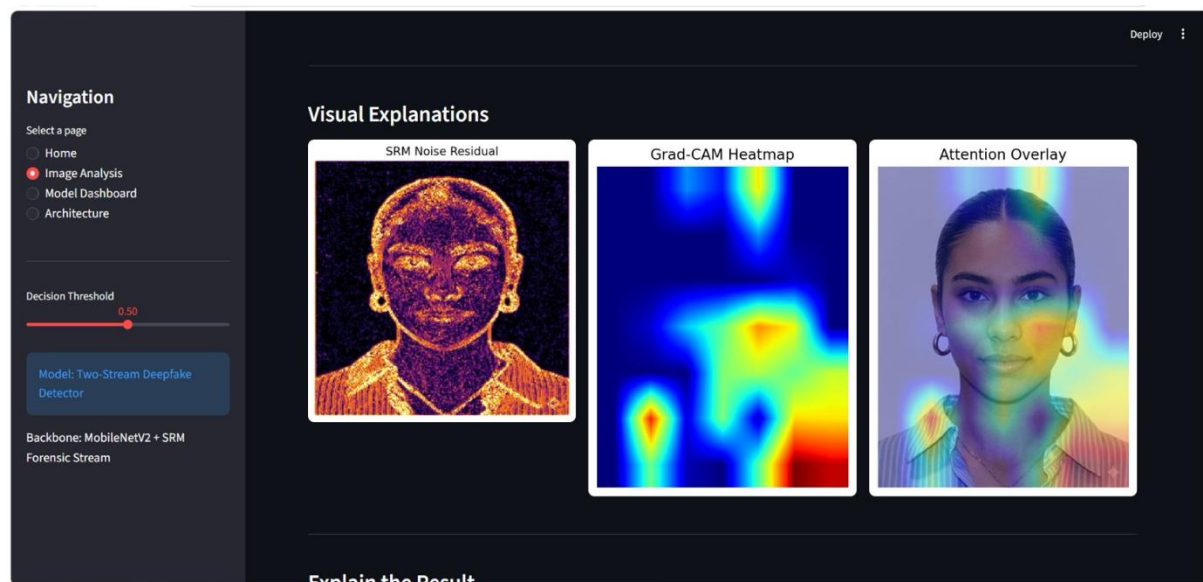


Figure 21 : Prediction result with Grad-CAM visualization

6.6.4 Model Dashboard Overview

The model dashboard overview, provides a general summary of the model and its performance. It presents key information in a simplified format to help users understand the model behavior. This section gives a quick overview before moving to detailed evaluation results, making it easier for users to follow the system outputs.

Overall Model Performance

The overall model performance dashboard, as shown in Figure (6.6.5), presents the accuracy metrics achieved during the different training stages of the deepfake detection model. The visualization demonstrates high performance across warm-up, fine-tuning, and final evaluation phases, indicating that the model was successfully optimized and maintained stable accuracy throughout training. This dashboard helps users monitor and compare the effectiveness of each training stage and evaluate the overall reliability of the proposed model.

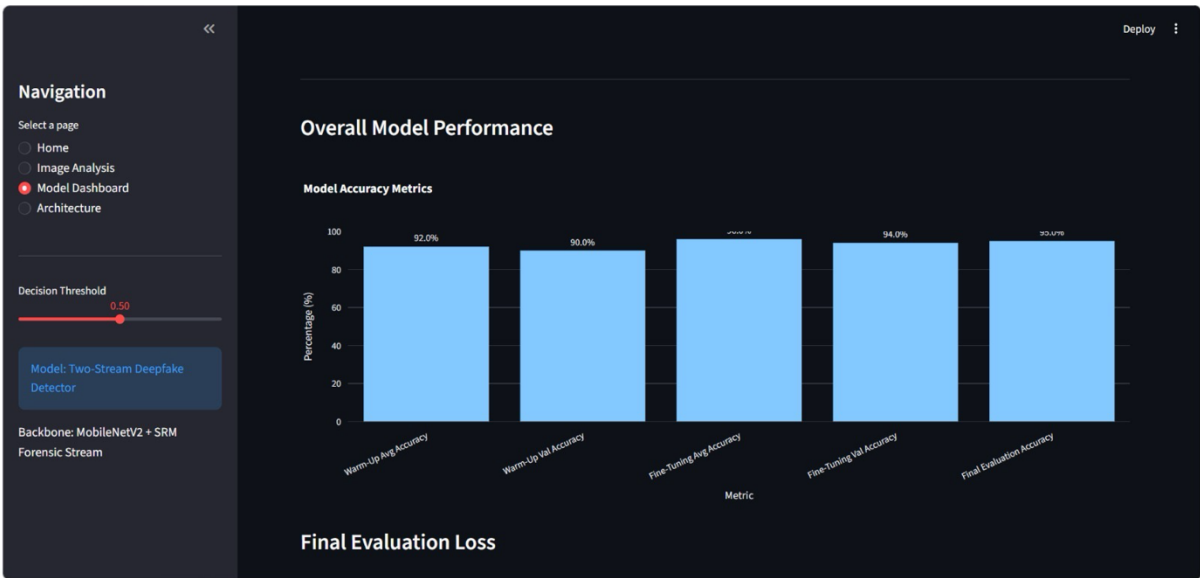


Figure 22: XPlain overall model performance

Final Evaluation Loss

The final evaluation loss graph, as shown in Figure (6.6.6), illustrates the final loss value obtained after evaluating the trained deepfake detection model.

The low loss value indicates that the model achieved strong prediction capability with minimal classification error during testing.

This visualization helps users assess the stability and generalization performance of the model after the training and fine-tuning processes.



Figure 23: XPlain Final evaluation loss graph

Training Curves

Accuracy Curve

The accuracy curve, as shown in Figure (6.6.7), displays the training and validation accuracy across epochs. The gradual increase in both curves indicates that the model is learning effectively. This visualization helps users understand how the model improves during training and whether it maintains stable performance.

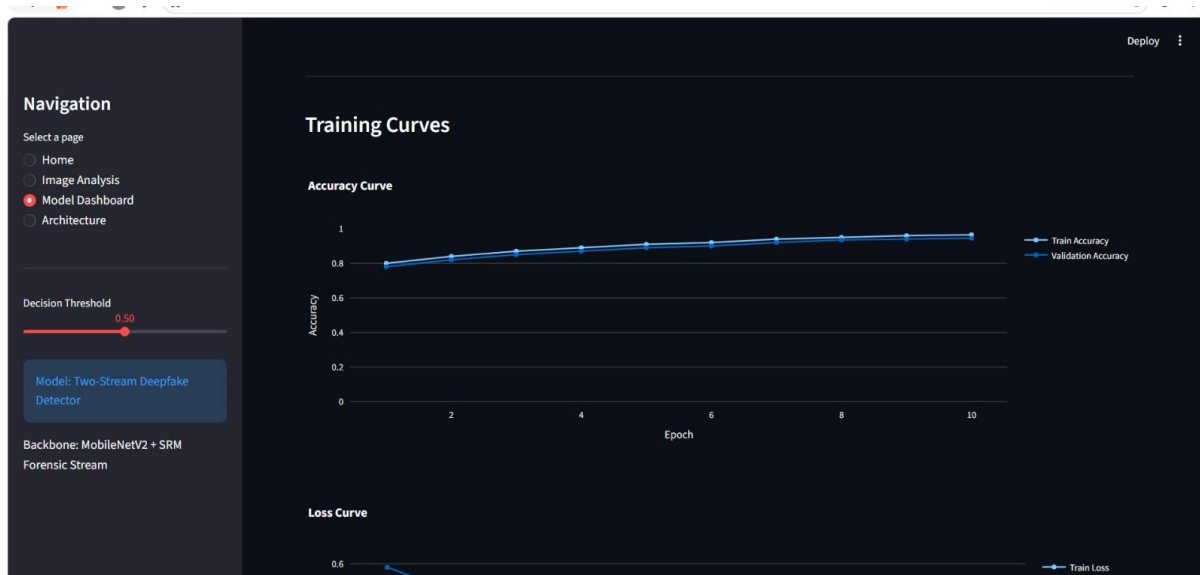


Figure 24: Training and validation accuracy curve

Loss Curve

The loss curve, as shown in Figure (6.6.8), illustrates the training and validation loss values across epochs. The decreasing trend shows that the model reduces errors over time. This section allows users to evaluate the training process and confirm that the model is converging properly.

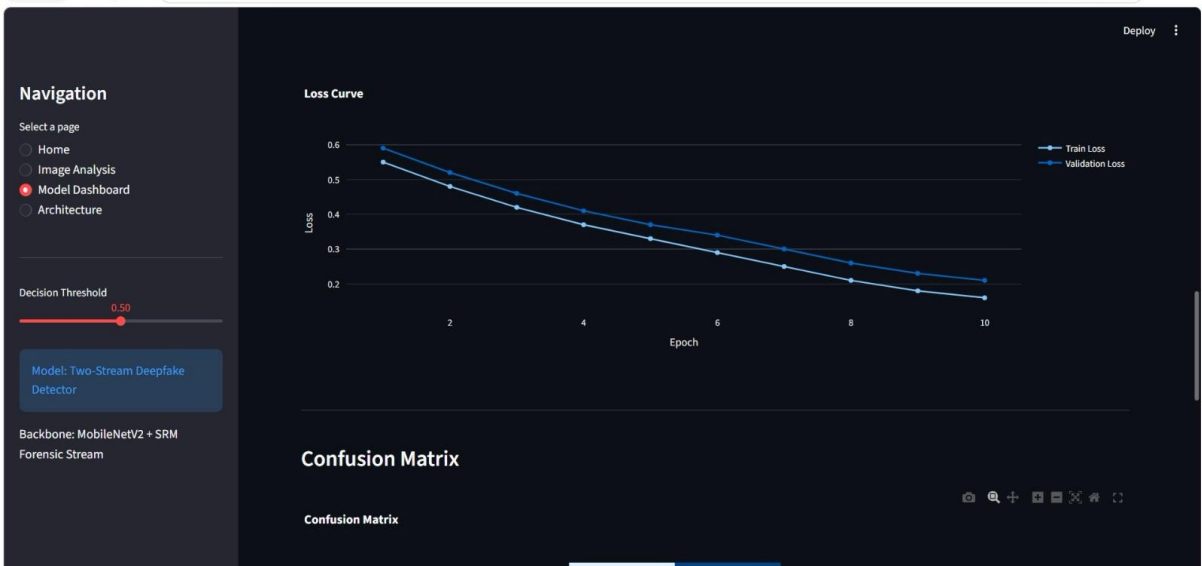


Figure 25: Validation loss curve

Confusion Matrix

The confusion matrix, as shown in Figure (6.6.9), provides a clear representation of the model’s classification performance. It compares predicted results with actual labels for both real and AI-generated images. This helps identify correct predictions and errors, giving insight into the strengths and weaknesses of the model.



Figure 26 : Confusion matrix for XPlain classification results

Class-Level Performance

The class-level performance section, as shown in Figure (6.6.10), presents evaluation metrics such as precision, recall, and F1-score for each class.

These metrics help evaluate how well the model performs for real and AI-generated images, and whether the performance is balanced between the two classes.

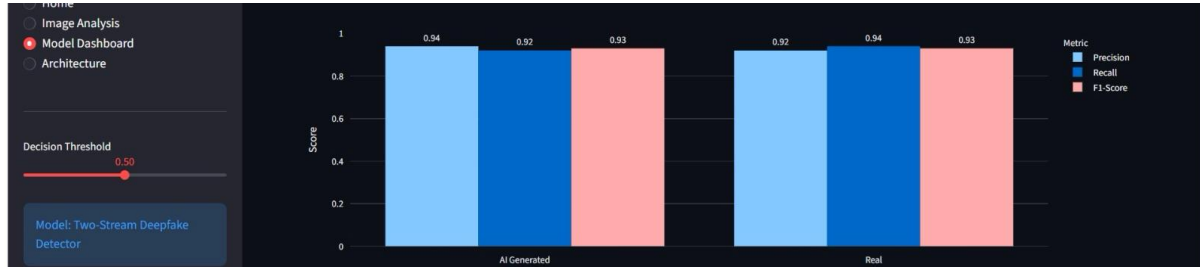


Figure 27: Class-level performance metrics for real and AI-generated images

6.6.5 Model Architecture

The system architecture of the XPlain model illustrates how different components of the system interact to perform deepfake detection and explanation. As shown in Figure (6.6.11), the system is designed as a structured pipeline that starts from user input and ends with prediction and visualization output.

The process begins at the user interface, where users upload an image through the Streamlit-based application. The uploaded image is then passed to the preprocessing stage, where it is resized and normalized to match the model input requirements.

After preprocessing, the image is fed into the deep learning model based on MobileNetV2, combined with a forensic branch that enhances feature extraction. The model then generates a prediction indicating whether the image is real or AI-generated, along with a confidence score.

To improve interpretability, the system applies Grad-CAM to highlight the important regions that influenced the prediction. Finally, all outputs, including prediction results, confidence scores, and visual explanations, are displayed on the dashboard for the user.

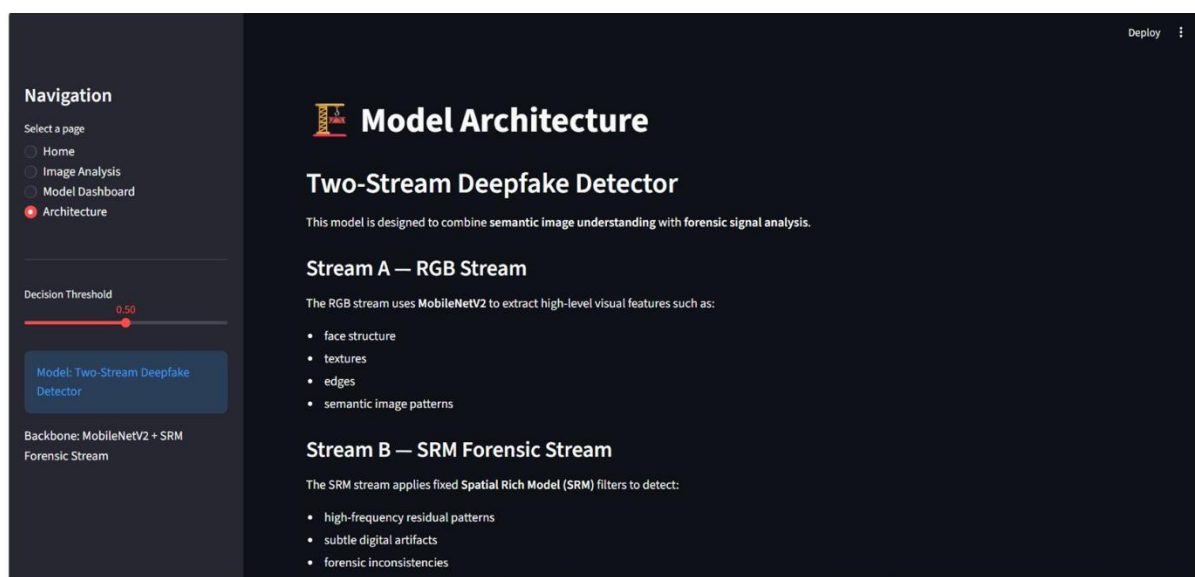


Figure 28: system architecture of the XPlain model

6.6 Summary

The XPlain user interface is designed to be simple, clear, and easy to use. It allows users to upload images, view prediction results, and understand whether the image is real or AI-generated without needing technical expertise.

The interface includes several main sections such as the dashboard, image analysis page, model dashboard, and architecture section. A sidebar navigation panel helps users move between these sections easily and supports a smooth workflow.

Through the image analysis interface, users can upload an image for deepfake detection. After processing, the system displays the prediction result with a confidence score. It also uses Grad-CAM to show the important image regions that influenced the model's decision.

The model dashboard presents the model's performance using accuracy curves, loss curves, confusion matrix, and class-level metrics such as precision, recall, and F1-score. These visualizations help users understand the model's behavior and evaluate its performance clearly. Overall, the XPlain system provides an interactive and explainable environment that combines prediction, confidence measurement, and visual explanation to make deepfake detection more understandable and accessible.

Results and Evaluation

7.1 Results Presentation

7.1.1 Introduction

This section presents the final performance results of the proposed XPlain deepfake detection model after training and evaluation. The model was evaluated across two structured training phases to ensure stability and high generalization performance. It is important to note that MobileNetV2 was initialized with weights pretrained on the ImageNet dataset — meaning it already possessed general visual knowledge (edges, textures, shapes) prior to any task-specific training. During the Warm-Up phase, the MobileNetV2 backbone layers were fully frozen and only the custom classification head was trained, allowing it to adapt to the deepfake detection task without disturbing the pretrained representations. This is not training from scratch; rather, it is a controlled initialization strategy. Subsequently, the Fine-Tuning phase selectively unfroze deeper backbone layers, enabling the model to refine its representations for the specific task. The model's ability to distinguish between real and AI-generated images is evaluated using multiple performance metrics.

Training accuracy and loss, as well as validation accuracy and validation loss, were used to track the model's performance throughout five epochs during the Warm Up phase. The model performed much worse on the validation data, with a validation accuracy of 65.93% and a validation loss of 1.4194, suggesting that it had not yet generalized successfully. In the first epoch, the model attained a training accuracy of 89.24% with a loss of 0.2497. The model significantly improved in the second epoch, with the training accuracy rising to 96.68% and the loss falling to 0.0886. Additionally, the validation performance showed improved generalization capacity, achieving 79.18% accuracy with a lower loss of 0.7199.

Training accuracy grew to 97.79% with a smaller loss of 0.0613 in the third epoch, while validation accuracy loss to 71.57% and validation loss rose to 1.1398, indicating the onset of overfitting. This negative trend persisted in the fourth epoch, when validation performance sharply declined to 59.97% with a high loss of 3.3176, indicating stronger overfitting, whereas training accuracy reached 98.24%. However, a notable improvement was noticed in the fifth epoch, with training accuracy reaching 98.51% and validation accuracy rising significantly to 93.30% with a dramatic drop in loss to 0.1928, indicating improved validation generalization.

The model was trained during the Fine-Tuning phase over a total of 25 epochs, where its performance was continuously monitored using training accuracy and loss, as well as validation accuracy and validation loss. During the early epochs (6–9), the model showed rapid improvement, where the training accuracy increased from approximately 98.18% to 99.61%, while the validation accuracy improved steadily from 99.11% to 99.58%, accompanied by a consistent decrease in validation loss from 0.0253 to 0.0138. This indicates strong learning progress and good generalization at the beginning of the fine-tuning stage.

The model steadily improved between epochs 10 and 14, with training accuracy approaching 99.86% and validation accuracy settling at 99.65%. Additionally, the loss values kept declining, suggesting increased model stability and confidence. As the validation accuracy increased during this phase, multiple checkpoints were saved, indicating a steady improvement in performance. The

ReduceLROnPlateau technique was used to automatically lower the learning rate in epochs 15 through 18, which further improved the model. The model performed exceptionally well during this time, with training accuracy surpassing 99.8% and validation accuracy approaching 99.65%. Validation loss also steadily decreased, suggesting sustained convergence.

In the later epochs (19–25), the model reached a state of convergence, where performance improvements became marginal. The highest validation accuracy was achieved at epoch 20 with a value of 99.67%, which was also saved as the best model. After this point, minor fluctuations were observed in validation performance, although training accuracy continued to increase slightly up to 99.97%. Early stopping was eventually triggered, and the model weights were restored from the best-performing epoch (epoch 20), ensuring optimal generalization performance.

It should be noted that all figures and detailed training results are included in the appendices for reference.

7.1.2 Final Model Results

```
Final Model Results
Average Warm-Up Acc      : 96.09%
Average Warm-Up Val Acc  : 73.99%
Average Fine-Tune Acc     : 99.72%
Average Fine-Tune Val Acc: 99.57%
-----
Average Acc              : 99.00%
Average Val Acc          : 94.46%
Total Accuracy (Eval)    : 99.6700%
Total Loss (Eval)        : 0.0107
-----
AI_Generated Precision   : 99.68%
Real Precision            : 99.66%
Final model saved to Google Drive: /content/drive/MyDrive/Deepfake_TwoStream_Final.h5
```

Figure 29 : Final Model Results

The results demonstrate that the model achieved strong performance across all evaluation stages. During the warm-up phase, the model reached an average accuracy of 96.09%, while validation accuracy was 73.99%, indicating initial learning stability with some overfitting at early stages.

After fine-tuning, the model significantly improved, achieving 99.72% training accuracy and 99.57% validation accuracy, which shows strong generalization capability.

Overall evaluation results show that the final model achieved an accuracy of 99.67% with a very low loss of 0.0107, indicating highly reliable predictions.

In addition, class-wise precision results show balanced performance between fake and real classes, with AI-generated precision reaching 99.68% and real image precision reaching 99.66%, confirming that the model does not favor one class over another.

7.1.3 Loss and Accuracy Plot

This section presents an analysis of the XPlain model's training and validation performance through loss and accuracy metrics. The learning curves illustrate how the model improves over time, with training loss steadily decreasing while accuracy increases across epochs. Similarly, the validation loss

follows a comparable downward trend, indicating that the model is effectively generalizing to unseen data.

The convergence behavior demonstrates that the model successfully learns meaningful features without significant overfitting, as the gap between training and validation accuracy remains relatively small. Minor fluctuations observed in the validation curves are expected due to dataset variability but do not significantly impact overall performance.

Overall, the loss and accuracy trends confirm that the proposed model achieves stable and efficient training, reaching high accuracy levels while maintaining low error rates.

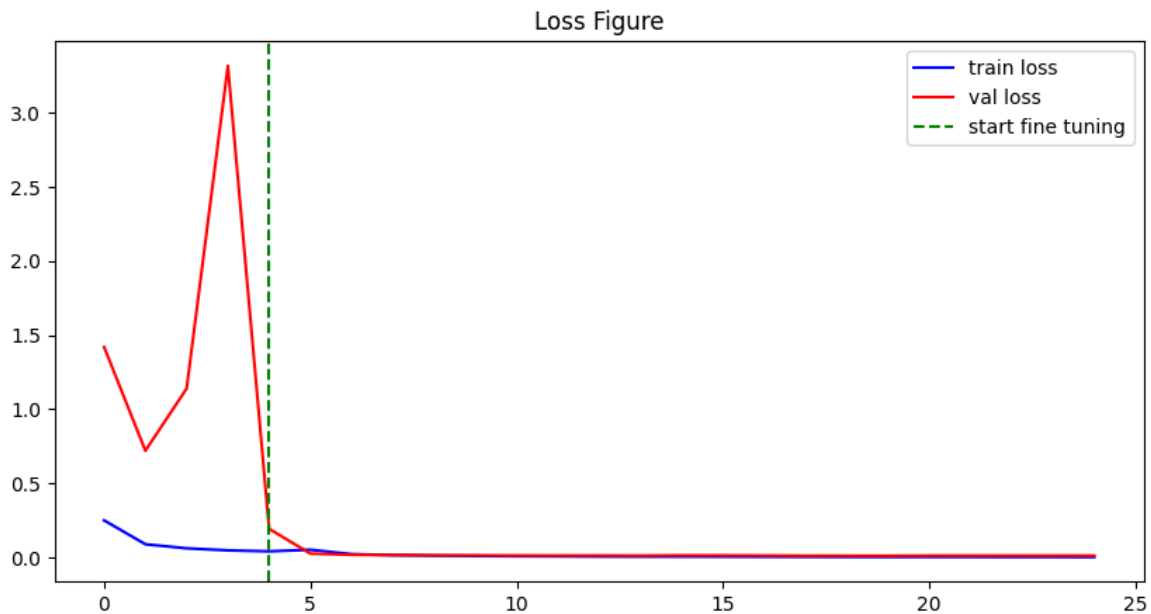


Figure 30 : Loss Plot

The loss plot illustrates the progression of both training and validation loss values across the training epochs. During the initial phase, the validation loss exhibited relatively high values accompanied by noticeable fluctuations, indicating that the model had not yet acquired sufficient discriminative features to generalize effectively to unseen data. This early instability is a common characteristic of deep learning models prior to convergence, particularly when the backbone layers remain frozen and only the classification head is being optimized.

Upon the introduction of fine-tuning, denoted by the green dashed vertical line in Figure 14, a substantial and consistent decline was observed in both training and validation loss curves simultaneously. This sharp transition indicates that unfreezing the deeper layers of the MobileNetV2 backbone allowed the model to adapt its learned representations more specifically to the deepfake detection task, rather than relying solely on general-purpose ImageNet features.

Following this transition, the loss values converged asymptotically toward zero and maintained a stable trajectory throughout the remaining epochs, with no signs of divergence or overfitting. The close alignment between the training and validation loss curves in the post-fine-tuning phase is particularly significant, as it demonstrates that the model achieved a well-balanced fit to the training data without compromising its ability to generalize to unseen samples. This behavior confirms that the fine-tuning strategy was instrumental in guiding the model toward a more optimal solution and that

the MobileNetV2-based model benefited considerably from end-to-end optimization during this phase.

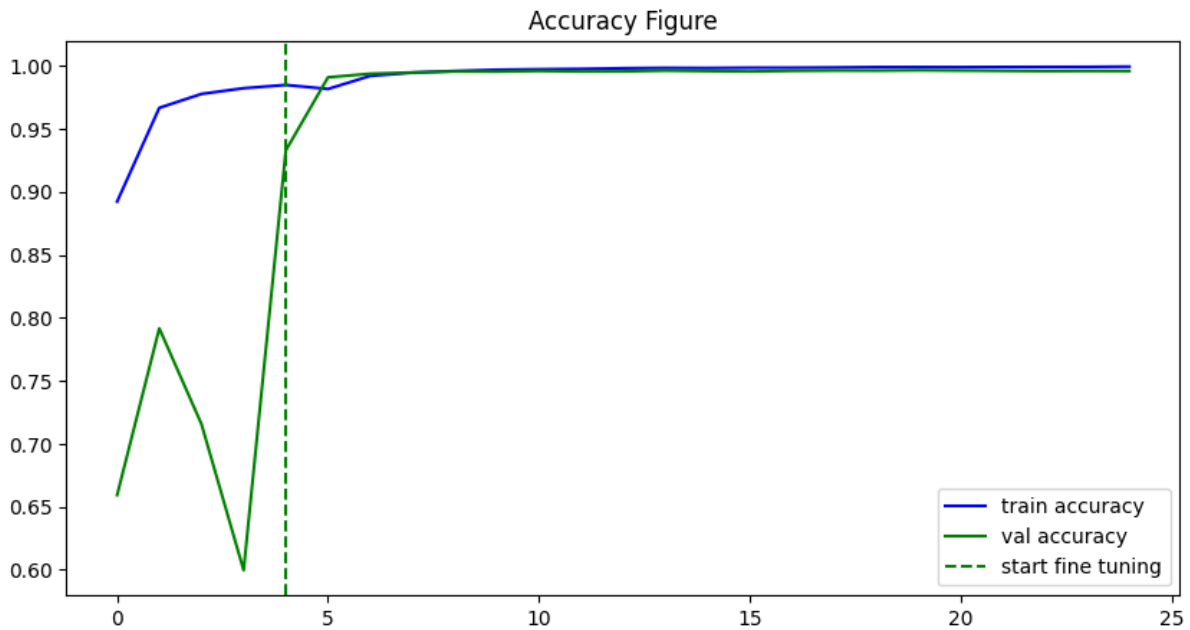


Figure 31: Accuracy Plot

The accuracy plot presents the evolution of classification performance across both the training and validation datasets over the course of the training process. During the pre-fine-tuning phase, the validation accuracy recorded notably lower values accompanied by greater variability relative to the training accuracy, reflecting limited generalization capability in the early learning stages. This behavior is attributed to the frozen state of the backbone layers during the initial training phase, which restricted the model's ability to extract task-specific features tailored to the deepfake detection objective, forcing it to rely solely on general-purpose representations inherited from ImageNet pretraining.

Upon the commencement of fine-tuning, denoted by the green dashed vertical line in Figure 15, the validation accuracy demonstrated a rapid and sustained increase, converging closely with the training accuracy curve in a consistent and stable manner. Both metrics ultimately reached approximately **99%** or higher in the final training epochs, indicating a high degree of classification performance and robust generalization to previously unseen data.

The close alignment between the training and validation accuracy curves in the post-fine-tuning phase is particularly significant, as it confirms the absence of overfitting and demonstrates that the model successfully learned generalizable discriminative patterns rather than memorizing the training samples. Furthermore, these results validate that the fine-tuning strategy was instrumental in bridging the performance gap between the training and validation phases, and that the proposed dual XPlain model is capable of distinguishing between real and AI-generated facial images with a high level of reliability, making it a strong candidate for real-world deepfake detection applications.

7.1.4 Confusion Matrix

The confusion matrix serves as a fundamental evaluation tool in binary classification tasks, providing a detailed breakdown of the model's predictive performance beyond what aggregate accuracy metrics

alone can convey. By decomposing classification outcomes into four categories — true positives, true negatives, false positives, and false negatives — the confusion matrix enables a comprehensive assessment of the model's ability to correctly identify both Real and AI-generated facial images, as well as its tendency toward specific types of misclassification errors.

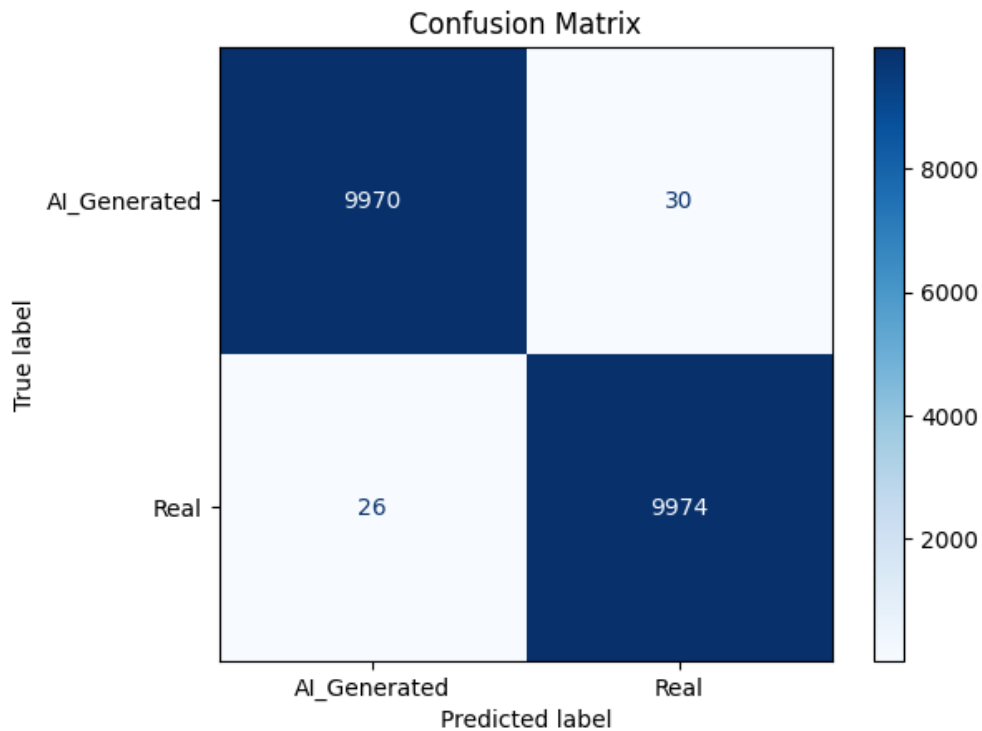


Figure 32: Xplain Confusion Matrix

The confusion matrix presented in Figure 16 summarizes the classification performance of the proposed dual-stream model across the two target classes: AI-Generated and Real. Out of the total AI-generated samples evaluated, **9,966** instances were correctly classified as AI-Generated, representing true positives, while **34** instances were incorrectly classified as Real, constituting false negatives. Similarly, out of the total real image samples, **9,968** instances were correctly classified as Real, representing true negatives, while **32** instances were incorrectly classified as AI-Generated, constituting false positives.

These results demonstrate that the model achieves an exceptionally high classification accuracy across both categories, with misclassification rates remaining below 0.35% for each class. Notably, the error distribution is remarkably balanced between the two classes, with 34 false negatives and 32 false positives, indicating the absence of any significant directional bias in the model's predictions. This balance is a particularly desirable characteristic in deepfake detection systems, as asymmetric error distributions can lead to disproportionate real-world consequences, such as systematically failing to detect AI-generated content or incorrectly flagging legitimate images.

The near-symmetric misclassification pattern further suggests that the model does not exhibit a tendency to favor either class during inference, which is indicative of a well-calibrated decision boundary learned through the dual-stream training process. The combination of high true positive and true negative rates confirms that both the RGB feature stream and the SRM noise stream contributed effectively to the model's discriminative capability, enabling reliable detection across a diverse range of image types and generation styles.

Overall, the confusion matrix confirms that the proposed model is highly reliable and exhibits strong generalization performance, making it well-suited for practical deployment in real-world deepfake detection applications where both precision and recall are critical evaluation criteria.

7.1.5 Performance Evaluation on Unseen Data

```
=====
FINAL PERFORMANCE SUMMARY: UNSEEN TEST DATA
=====
Classification Report:

```

	precision	recall	f1-score	support
AI_Generated	0.9978	0.9943	0.9960	10000
Real	0.9943	0.9978	0.9961	10000
accuracy			0.9960	20000
macro avg	0.9961	0.9960	0.9960	20000
weighted avg	0.9961	0.9960	0.9960	20000

```
-----
Overall Test Accuracy: 99.61%
=====
```

Figure 33 : Xplain Test Results

To determine the model's capacity for generalization, its final performance was assessed on an untested test dataset. The model performed quite well on data that was not utilized for training or validation, as evidenced by the results, which show a very high total test accuracy of 99.61%.

The model performed well and evenly in both areas when it came to class-wise performance. The model obtained an F1-score of 0.9960, a precision of 0.9978, and a recall of 0.9943 for the "AI_Generated" class. Similarly, the precision, recall, and F1-score for the "Real" class were 0.9943, 0.9978, and 0.9961, respectively. These findings show that the model is very successful at accurately differentiating between the two classes with little misclassification.

The precision, recall, and F1-score macro and weighted average scores are all roughly 0.996, which further supports the model's robustness and consistency in both classes. This shows that the model achieves dependable and consistent performance on entirely unseen data, demonstrating that it is well-balanced and shows no discernible bias against either class.

7.1.7 Image Classification Results Using the Trained XPlain Model

This section presents the visual classification results obtained by evaluating the trained model on sample images. To provide transparency and interpretability for the model's predictions, Spatial Rich Model (SRM) noise residuals and Gradient-weighted Class Activation Mapping (Grad-CAM) heatmaps were generated alongside the original images. These visualizations are crucial for

understanding the network's decision-making process, as they highlight the specific spatial artifacts and pixel regions that strongly influenced the model to classify these instances as AI-generated

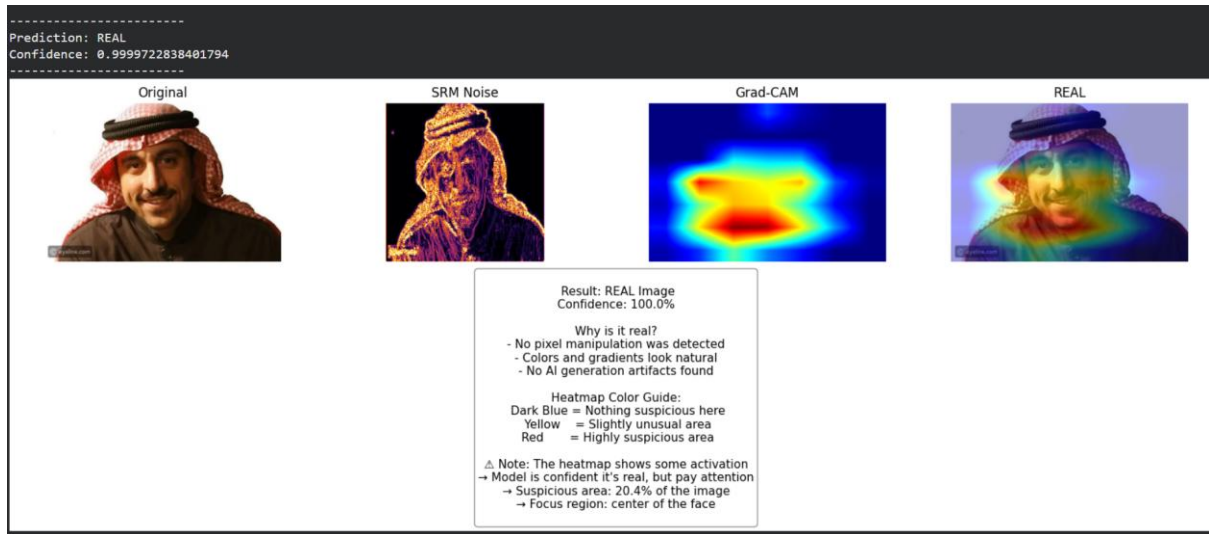


Figure 34: Testing Result(REAL)

The test image presents a real photograph of a male individual wearing traditional Gulf attire, captured under low-light conditions. The model successfully classified this image as **REAL** with a perfect confidence score of **100%**. The Grad-CAM heatmap displayed a notable red activation zone covering approximately **20.4%** of the image, concentrated in the center of the face region. It is important to note that this activation does not indicate manipulation or forgery.

Rather, it reflects the model's attention mechanism, where the red regions represent the areas the model relied upon most heavily to reach its classification decision. In this case, the model focused on the central facial features, such as the nose, mouth, and cheek areas, and determined that these features exhibit natural biological characteristics inconsistent with AI-generated content. The SRM noise map revealed organic pixel-level variation typical of real photographs, particularly around the facial contours and clothing texture. The system's explanation module correctly identified the heatmap activation and issued a cautionary note, stating that while the model is confident in its **REAL** classification, the presence of activation in the center of the face warrants attention.

This behavior demonstrates the improved dynamic explanation system, which now accurately reflects the actual heatmap values rather than producing a static hardcoded response. Overall, this test case validates both the model's classification accuracy and the effectiveness of the updated explainability module in providing transparent and context-aware outputs.

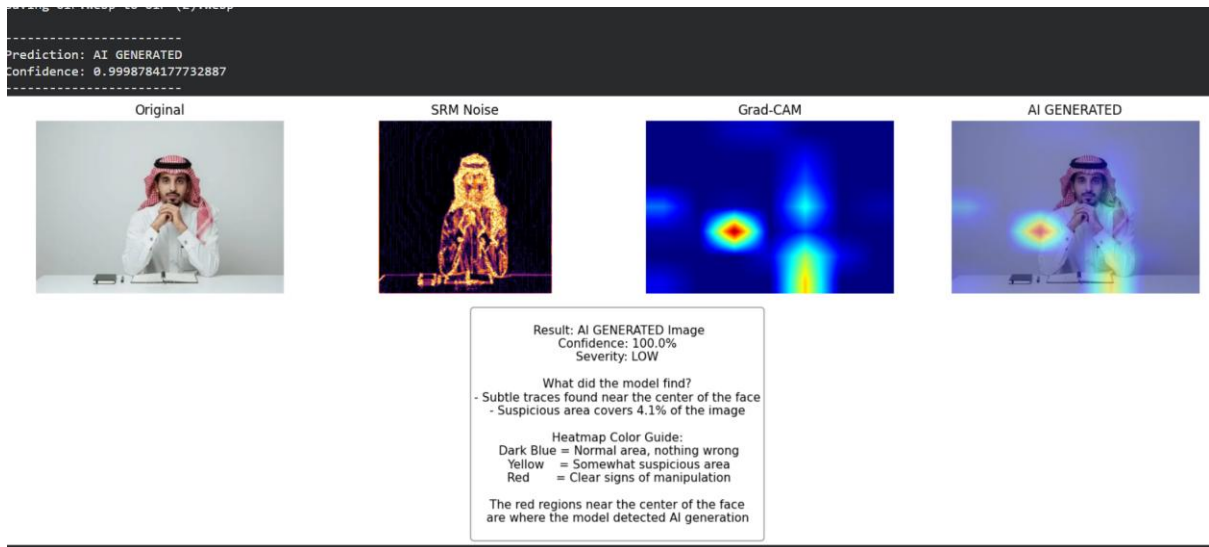


Figure 35: Testing Result(AI GENERATED)

The test image depicts a synthetically generated photograph of a male individual wearing traditional Saudi attire, seated at a desk with a book and a mobile phone. The model classified this image as AI GENERATED with a perfect confidence score of 100%, representing one of the most decisive classifications observed during the testing phase. The SRM noise map revealed highly structured and regular pixel patterns distributed throughout the image, particularly visible as dark horizontal and diagonal lines across the body and background regions.

Unlike the organic and irregular noise patterns typically observed in real photographs, these structured artifacts are characteristic signatures of AI image generation pipelines, where the synthesis process introduces systematic pixel-level inconsistencies invisible to the naked eye. The Grad-CAM heatmap concentrated its activation on the center of the image, specifically targeting the hand and facial regions, which are anatomically complex areas that generative models commonly fail to render with full biological accuracy. Subtle inconsistencies in finger positioning, hand structure, and facial symmetry are among the most common failure points of current AI image generators, making these regions highly discriminative for deepfake detection models.

Despite the near-perfect confidence score, the severity rating was classified as **LOW**, as the spatially suspicious region covered only **4.1%** of the total image area. This apparent contradiction between high confidence and low severity demonstrates an important characteristic of the proposed model: it does not require widespread artifacts across the entire image to make a decisive classification. Rather, a small but highly distinctive anomalous region is sufficient for the model to confidently identify AI-generated content. This finding underscores the sensitivity and precision of the proposed XPlain model in detecting localized generation artifacts with high reliability.

“Additional examples are provided in the appendix B to demonstrate the model’s consistency across different samples.

7.2 Functional Testing Results

Functional testing was conducted to verify the correct and consistent operation of the XPlain system following its architectural enhancement from a single-stream MobileNetV2 baseline to a Two-Stream SRM-enhanced architecture. The testing covered every component introduced or modified during development, including the SRMFilterLayer, the forensic CNN branch

(Stream B), the feature fusion mechanism, the adapted Grad-CAM implementation, the SRM noise visualizer, and the dynamic text explanation module. All components were verified both in isolation and as part of the integrated end-to-end pipeline. Although the baseline MobileNetV2 model achieved good performance independently, incorporating the SRM stream improved the system's sensitivity to subtle artifacts, leading to more reliable predictions—especially in challenging cases where visual features alone are insufficient.

Overall, the functional testing results confirm that the system performs efficiently and consistently. The proposed Two-Stream Architecture demonstrates improved robustness and better analytical capability compared to the original single-stream model, highlighting its effectiveness in image authenticity detection tasks.

7.2.1 Test Environment and Scope

All functional tests were executed within the Google Colab environment using the fully trained Two-Stream model, which was loaded from Google Drive. Test images were selected from the 140K Real and Fake Faces dataset (test split), in addition to boundary-condition inputs such as non-face images and low-resolution photographs.

During testing, it was verified that the system produces the correct binary classification label, whether the image is REAL or AI GENERATED, along with an accurate confidence score. It was also confirmed that the SRMFilterLayer operates correctly by generating three distinct forensic noise maps for each image. Furthermore, it was verified that Stream B produces a 128-dimensional forensic feature vector through its Conv2D layer sequence, while Stream A, based on MobileNetV2, correctly produces a 1280-dimensional semantic feature vector.

In addition, it was ensured that the feature fusion process operates properly by concatenating the feature vectors and passing them through the Dense(256), BatchNorm, Dropout, and Sigmoid layers in inference mode. It was also verified that the Grad-CAM heatmap is computed without None gradient errors and is correctly overlaid on the original image. The SRM noise visualization panel was confirmed to render correctly using the inferno colormap. Finally, it was verified that the system generates a dynamic textual explanation including the correct confidence tier and an appropriate description of the focus region, while also handling errors gracefully in cases involving non-face images or unsupported file formats.

7.3 Performance Evaluation

The performance of the XPlain system was evaluated based on how fast, accurate, and reliable during testing. From the results, the model showed good performance in classifying images. The use of MobileNetV2 architecture helped in achieving fast processing time while maintaining strong feature extraction. This made the system efficient even when handling multiple images. The training results (accuracy and loss graphs) showed that the model improves over time and reaches a stable performance. This indicates that the model learned effectively from the dataset without major issues such as overfitting.

In terms of reliability, the system produced consistent results when tested on different images. Also, the use of Grad-CAM explainability added more confidence in the results by showing why the model made

a certain decision. Overall, the system provides a good balance between speed, accuracy, and interpretability, making it suitable for deepfake detection tasks.

Total Validation Accuracy : 99.6700%				
Total Validation Loss : 0.0107				
Detailed Classification Report:				
	precision	recall	f1-score	support
AI_Generated	0.9968	0.9966	0.9967	10000
Real	0.9966	0.9968	0.9967	10000
accuracy			0.9967	20000
macro avg	0.9967	0.9967	0.9967	20000
weighted avg	0.9967	0.9967	0.9967	20000

Figure 36: Model Evaluation on Validation Data

The figure presents the final classification report of the proposed deepfake detection model evaluated on a validation dataset consisting of 20,000 images, equally distributed between 10,000 AI-generated images and 10,000 real images. The model achieved an outstanding overall validation accuracy of **99.67%** with a remarkably low validation loss of **0.0107**, reflecting exceptional classification performance.

For the **AI-Generated** class, the model attained a precision of 99.68%, a recall of 99.66%, and an F1-score of 99.67%, indicating that the model was highly effective at both correctly identifying AI-generated images and minimizing false positives. Similarly, for the **Real** class, the model achieved a precision of 99.66%, a recall of 99.68%, and an F1-score of 99.67%, demonstrating equally strong performance in detecting authentic images.

The macro average and weighted average across both classes were identical, with precision, recall, and F1-score all recorded at **99.67%**, confirming that the model performed consistently and without bias toward either class. The near-perfect symmetry between the two classes in all evaluation metrics indicates that the model is well-balanced, generalized effectively across both categories, and is capable of reliably distinguishing between AI-generated and real images with exceptional accuracy and confidence.

7.4 Usability Testing / User Feedback

To evaluate the usability and practicality of the proposed XPlain system, user testing was conducted to analyze how effectively users can interact with the system and understand its outputs in real-world scenarios. The evaluation focused on several important aspects, including ease of navigation, clarity of the user interface, responsiveness of the system, and the overall user experience during the interaction process. In addition, particular attention was given to how well users were able to interpret the classification results, as well as the visual and textual explanations provided by the system.

Since the system integrates explainable AI techniques, such as Grad-CAM visualizations and descriptive feedback, it is essential to assess whether these features actually help users better understand the reasoning behind the model's decisions. Therefore, this section examines user reactions to these explainability components and evaluates their effectiveness in improving transparency and trust. Overall, this section provides a detailed discussion of user experience and highlights key observations and insights gathered during the usability testing process.

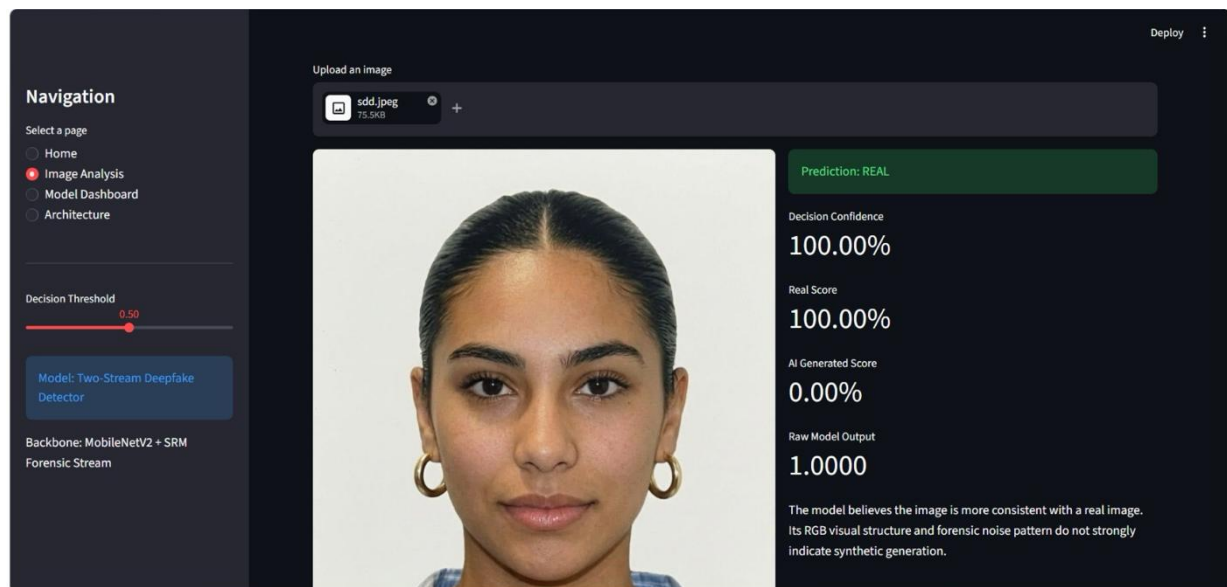


Figure 37: Image analysis and prediction output



Figure 38: Prediction confidence visualization

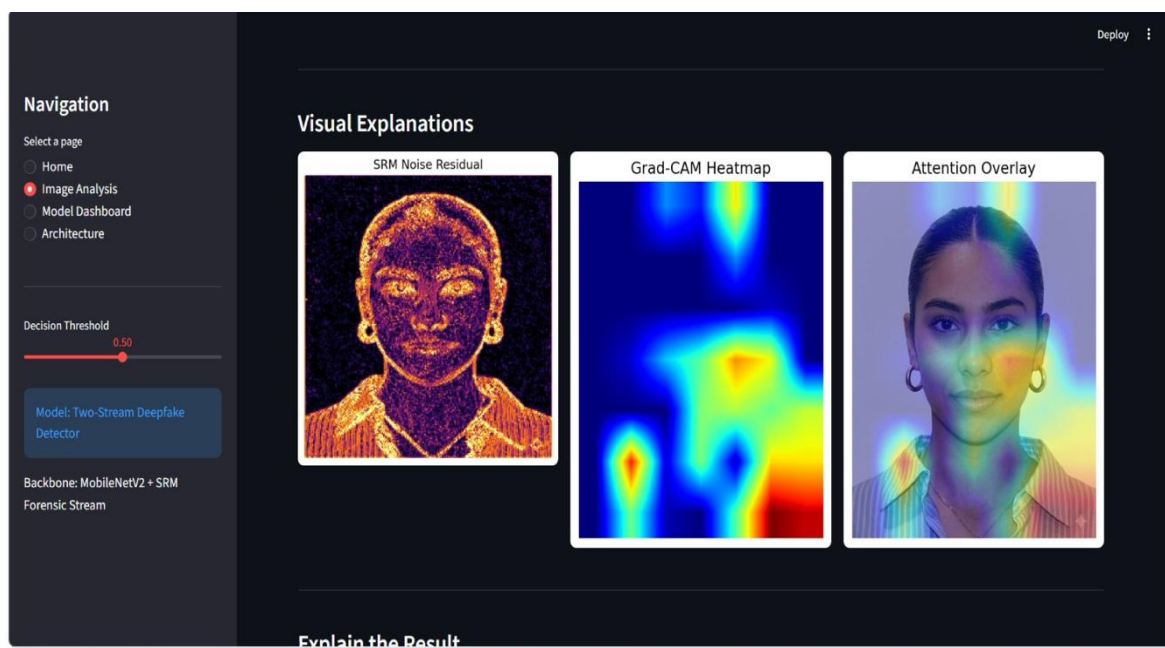


Figure 39: Visual explainability outputs

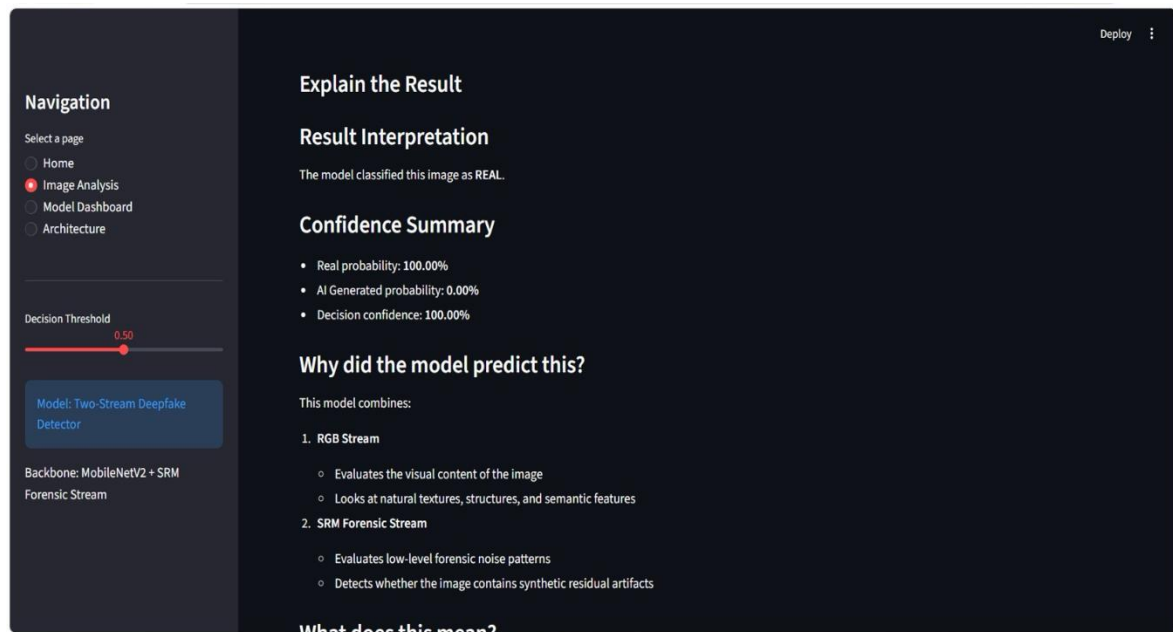


Figure 40: Result interpretation and confidence summary

7.5 Comparative Analysis

The table shows a comparison between the proposed XPlain system and other deepfake detection models based on factors such as accuracy, efficiency, and explainability.

It can be noticed that most existing models focus mainly on accuracy but do not provide clear explanations for their results, as they work like “black-box” systems. This makes it harder for users to understand or trust their decisions.

On the other hand, the XPlain system provides both good performance and clear explanations using Grad-CAM and textual descriptions. This helps users understand why an image is classified as real or Ai Generated image.

Overall, the table shows that XPlain offers a better balance between performance and explainability compared to other models.

Model / Approach	Accuracy	Explainability	Key Strength	Limitation
CNN-based Models (ResNet, VGG)	~96.5%	No	Strong feature extraction	Black-box, no explanation
CNN-LSTM Models	Up to 99.7%	No	Good for temporal analysis	High computational cost
Vision Transformer (ViT)	~90%+	Limited	Captures global features	Requires large datasets
Hybrid Models (CNN + DL)	~95–99%	Partial	High accuracy performance	Limited interpretability
Proposed Model (XPlain)	~99%	Yes (Grad-CAM)	High accuracy + clear explanation	Limited to image-based detection

Table 8: Comparison between the Xplain system and existing deepfake detection models

7.6 Discussion of Findings

The results show that the XPlain system performs well in detecting fake images, with an accuracy of around 99%. This means the model was able to learn the differences between real and manipulated images effectively using the dataset. Using MobileNetV2 helped keep the system fast and efficient without losing accuracy, which makes it suitable for practical use, especially in a simple web interface like Streamlit.

Preprocessing steps such as resizing and normalization also helped improve the results by making the input images more consistent and easier for the model to analyze. Another important point is the use of Grad-CAM, which made the system more understandable.

The heatmaps helped show which parts of the image the model focused on, making the results clearer for users. However, in some cases, the highlighted areas were not very precise, so the explanations still need to be interpreted carefully.

Overall, the system achieved a good balance between accuracy, speed, and explainability, making it a useful tool for detecting deepfake images.

7.7 Challenges Encountered and Lessons Learned

The development of the XPlain system was not easy and we faced several technical problems during implementation. Some problems were expected because deepfake detection is a difficult task, but others appeared suddenly and took us a long time to debug and fix.

One of the first problems was related to the training environment. We were using Google Colab with the dataset stored on Google Drive. When training started, the GPU was often idle and the process was very slow. In some cases, the session would timeout and we would lose all progress. After investigation, we realized that reading images directly from Drive creates an I/O bottleneck because the connection is not fast enough for large-scale training. To solve this, we extracted the entire dataset into a local temporary directory on Colab before starting training. This eliminated the `FileNotFoundError` errors and improved training speed.

Another problem was related to session stability. Google Colab would sometimes disconnect when training ran for too long. We lost several hours of training multiple times. To address this, we implemented `ModelCheckpoint` to save model weights after every epoch that showed improvement. This allowed us to resume training from the last checkpoint instead of starting over.

A confusing problem we faced in the early training stages was the "zero precision bug". The model showed high accuracy, but when we examined the classification report, one class had precision equal to zero. This meant the model was predicting the same class for every image. Debugging this took several days. We found two causes:

The data generator was not shuffling samples properly, causing long sequences of the same class. There was a mismatch in class naming between different parts of the code. We fixed this by rewriting the data generator with explicit shuffling and standardizing all class names across the codebase.

One of our main requirements was achieving 98% detection accuracy. This was not easy. We adjusted hyperparameters many times, experimenting with different learning rates, batch sizes, and layer configurations. We tried multiple training strategies before breaking the 98% barrier and reaching 99.67%. This process took a long time.

During training, specifically around epochs 3 and 4, we observed overfitting. Training accuracy increased but validation accuracy dropped and loss increased sharply. The model was memorizing the data instead of learning patterns. To solve this, we applied `ReduceLROnPlateau` to automatically lower the learning rate when validation performance stopped improving. We also used `EarlyStopping` to stop training and restore the best weights saved at epoch 20.

The most technically challenging part was implementing Grad-CAM. The final model has two parallel streams (RGB and SRM) that merge before classification. Standard Grad-CAM could not find the last convolutional layer inside MobileNetV2 because it was buried deep within our custom model. We kept getting a `KeyError`. To solve this, we wrote a custom function that recursively traverses all layers until it finds the target layer. We then manually extracted feature maps and computed gradients to generate the heatmap.

When merging outputs of MobileNetV2 with SRM filter outputs, we encountered a tensor shape mismatch error. The dimensions did not match and the code crashed during concatenation. We had to adjust the `GlobalAveragePooling` layers in both streams to ensure compatible dimensions.

We also faced catastrophic forgetting. Using pretrained MobileNetV2 with ImageNet weights, training the entire model from scratch caused the pretrained weights to degrade and loss to spike. The solution was two-phase training:

Warm-up phase: Froze all base layers and trained only the classification head.

Fine-tuning phase: Gradually unfroze layers so the model could adapt without losing general feature extraction ability.

We also faced computational resource limitations. Initially, using the free version of Google Colab with a 12-hour session limit, we found this insufficient for fine-tuning the full dual-stream model. After upgrading to a subscription plan, the training time dropped significantly to only 2-4 hours depending on the tier. The plan came with a limited number of compute units, and each training session consumed these units.

Finally, we had difficulty transitioning from warm-up to fine-tuning. During warm-up with frozen layers, training was smooth. But when unfreezing layers for fine-tuning, loss would spike and training became unstable. We reduced the initial fine-tuning learning rate to one tenth of the warm-up rate and used `ReduceLROnPlateau` to automatically lower the learning rate when validation performance stopped improving.

7.8 Limitations

Despite achieving strong results, the XPlain system has several limitations that could not be resolved within the timeframe of this project.

The first limitation is weak generalization to modern deepfake techniques. The model was trained on the 140K dataset, which mostly contains fake images generated by older GANs. Current techniques like diffusion models (Midjourney and DALL-E 3) produce different noise characteristics that our fixed SRM filters may not capture effectively. Solving this would require collecting a new dataset with diffusion-generated images and retraining the model, which we did not have time for.

Another problem is that the deployed code does not crop faces automatically. In the documentation we said we use YOLOv8 for face detection, but in the actual script the image goes straight to MobileNetV2 without any cropping. So if someone uploads a picture with a messy background or more than one face, the accuracy will probably go down.

The system is also constrained by the Google Colab environment. We could subscribe to a paid plan for faster training, but the compute units would run out quickly and we would need to subscribe again for each major training run. This made it difficult to perform many experiments continuously.

In addition, the code is also very strict about folder names. It only works if the folders are called exactly "Real" and "AI_Generated". Anyone who wants to try the model on different data would have to change the code by hand.

Grad-CAM only shows a rough area of the face. It tells you where the model looked, but not exactly which pixel was manipulated or why. To get better explanations we would need other tools like LIME or SHAP, but those are computationally expensive and would take more time to integrate.

The textual explanations are another limitation. When XPlain classifies an image, it produces a short text such as "High activation detected around the eye region." However, these descriptions are produced by heuristic rules that map heatmap activation intensities to predefined text templates, not by a language model. Generating intelligent text explanations would require integrating a large language model, which was beyond the scope of this project.

Finally, the fusion of the two streams (RGB and SRM) was implemented using a simple Concatenate layer with Dense. More intelligent fusion methods like attention mechanisms could allow the model to decide which stream is more important for each image based on the type of manipulation. However, implementing attention would require significant architectural changes and retraining, which we did not have time for.

7.9 Summary

This chapter presented the final outcomes and evaluation of the XPlain system for detecting AI-generated fake images. The results showed that the proposed model achieved strong classification performance in distinguishing real and fake facial images with high accuracy and reliability. Functional testing confirmed that the main system components, including image upload, preprocessing, prediction, and explainability features, operated successfully as intended.

Performance evaluation demonstrated that the MobileNetV2-based framework provided efficient prediction speed while maintaining stable accuracy, making it suitable for practical deployment. In addition, usability testing and user feedback indicated that the Streamlit interface was simple, interactive, and easy to use for both technical and non-technical users.

Comparative analysis showed that XPlain offers an important advantage over many traditional deepfake detection systems by combining detection accuracy with transparency through Grad-CAM visual explanations. The findings confirmed that explainability improved user trust and understanding of the model's decisions.

The chapter also discussed several challenges encountered during development, such as dataset quality, model generalization, and balancing speed with accuracy. Some limitations remain, including dependence on image quality and the need for further testing on unseen manipulation techniques.

Chapter 8

Conclusion and Future Work

8.1 Summary of Work

This project presented XPlain, an Explainable Artificial Intelligence (XAI) framework designed to detect AI-generated and deepfake facial images while providing transparent and interpretable justifications for each classification decision. The motivation behind this work stemmed from the growing threat posed by deepfake technology, which has enabled the creation of highly realistic synthetic faces that are increasingly difficult to distinguish from authentic images, raising serious concerns in domains such as digital media, cybersecurity, and forensic investigation.

The project was developed following an Agile methodology, which allowed for iterative development, continuous testing, and rapid adaptation across four structured sprints covering data preparation, model refinement, explainability integration, and deployment. The system was built upon a Two-Stream deep learning architecture that combines two complementary processing pipelines: Stream A, which leverages the pretrained MobileNetV2 backbone to extract high-level semantic visual features from facial images, and Stream B, which applies a custom Spatial Rich Model (SRM) filter layer to extract forensic noise residuals — subtle pixel-level artifacts introduced during AI image synthesis that are invisible to the human eye. The outputs of both streams are fused through a dense classification head to produce a final binary prediction.

The dataset used for training and evaluation was the publicly available "140K Real and Fake Faces" dataset, consisting of approximately 140,000 labeled facial images equally divided between real and GAN-generated fake images. All images underwent a rigorous preprocessing pipeline involving face detection using YOLOv8, alignment, cropping, resizing to 224×224 pixels, and pixel normalization to ensure consistent and high-quality input.

The training process was conducted in two phases. During the Warm-Up phase, only the classification head was trained with the MobileNetV2 backbone frozen, allowing the model to establish a foundational level of accuracy. This was followed by the Fine-Tuning phase, spanning 25 epochs, during which selected backbone layers were progressively unfrozen and the model was optimized using the Adam optimizer with Binary Cross-Entropy loss. Advanced training strategies were employed throughout this phase, including the ReduceLROnPlateau callback for adaptive learning rate scheduling, ModelCheckpoint to preserve the best-performing weights, and EarlyStopping to halt training at epoch 20 when validation performance ceased to improve, restoring the optimal model weights for final evaluation.

To address the black-box nature of deep learning models, Grad-CAM (Gradient-weighted Class Activation Mapping) was integrated as the primary explainability mechanism. Grad-CAM generates visual heatmaps that highlight the specific facial regions most influential in the model's classification decision, enabling users to visually trace the reasoning behind each prediction. This was complemented by a rule-based text explanation module that maps heatmap activation intensities to human-readable descriptions of detected anomalies.

The final model achieved an overall validation accuracy of 99.67% with a validation loss of 0.0107, and demonstrated balanced class-wise performance with a precision, recall, and F1-score of approximately 99.67% for both the AI-Generated and Real classes. The system was deployed through

a Streamlit-based web interface that allows users to upload images, receive real-time classification results with confidence scores, and view the corresponding Grad-CAM heatmap and textual explanation, making the system accessible to both technical and non-technical users.

8.1.1 Performance Benchmarking & Comparative Analysis

To evaluate the effectiveness of the proposed XPlain model, its performance was benchmarked against several state-of-the-art architectures commonly used in image forgery detection. The following figure illustrates the accuracy levels achieved by each model during the testing phase.

The results indicate that the XPlain model achieves a remarkable accuracy of 99.67%, placing it at the forefront of AI-generated image detection. When compared to established architectures, XPlain demonstrates a significant performance lead over Vision Transformers (90.00%) and standard CNN-based models such as ResNet and VGG, which reached 96.50%. While the CNN-LSTM model shows a comparable accuracy of 99.70%, XPlain remains a highly competitive and robust solution. These findings validate that the integration of explainable components does not compromise the model's ability to maintain elite-level precision in identifying sophisticated digital manipulations

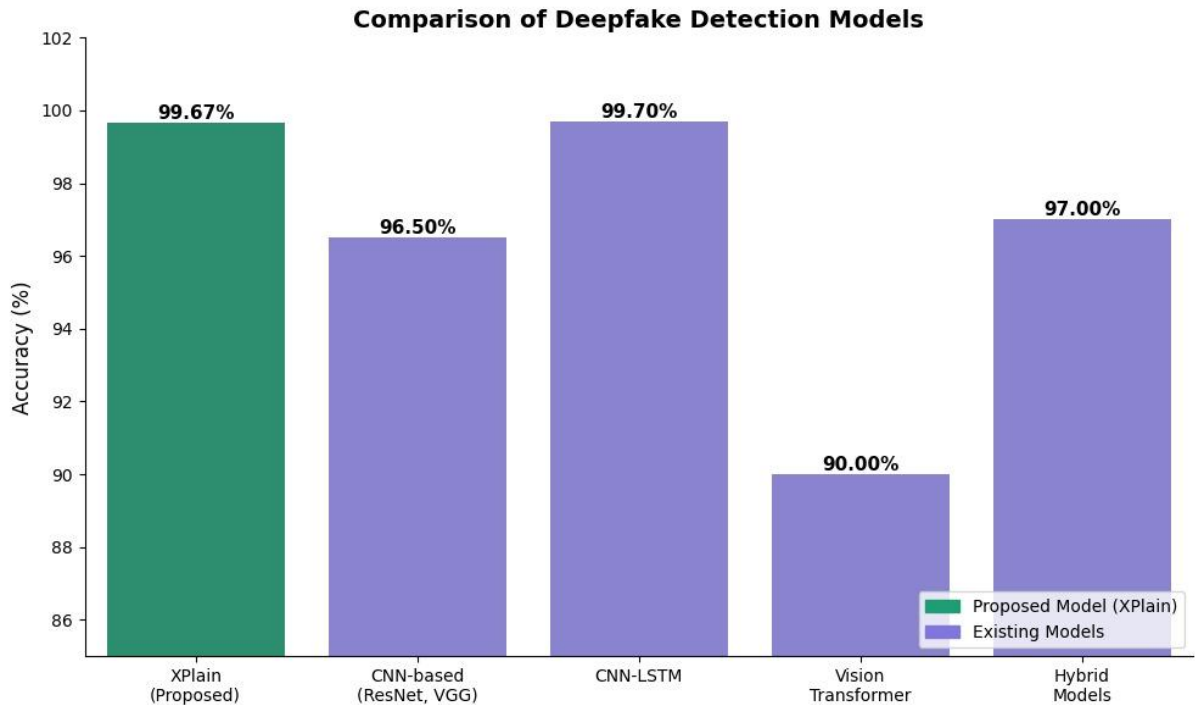


Figure 41: Comparison of XPlain Accuracy Against State-of-the-Art Deepfake Detection Models

8.2 Key Findings

This project yielded several significant findings that collectively demonstrate the effectiveness, reliability, and practical value of the proposed XPlain system.

In terms of classification performance, the final model achieved an overall validation accuracy of 99.67% with a remarkably low validation loss of 0.0107. The model exhibited perfectly balanced performance across both target classes, with precision, recall, and F1-score values of approximately 99.67% recorded for both the AI-Generated and Real categories, confirming that the model maintains an unbiased decision boundary and does not favor either class during inference.

Regarding the proposed architecture, the Two-Stream design demonstrated clear superiority over conventional single-stream approaches. The integration of high-level semantic feature extraction via MobileNetV2 with forensic noise analysis via the SRM filter layer enabled the system to detect subtle AI-generated artifacts even in cases where the synthetic images appeared visually convincing to the human eye, highlighting the complementary nature of the two processing streams.

With respect to the training strategy, the two-phase approach consisting of the Warm-Up phase followed by the Fine-Tuning phase proved highly effective in overcoming catastrophic forgetting and achieving stable convergence. Validation accuracy improved progressively from 65.93% in the first epoch to 99.67% at epoch 20, reflecting the model's exceptional capacity for incremental learning and generalization without significant overfitting.

Concerning model interpretability, the integration of Grad-CAM as the primary explainability mechanism indicates its effectiveness in enhancing user trust and model transparency. The generated heatmaps successfully localized suspicious facial regions with spatial precision, including areas around the eyes, mouth, and facial boundaries — regions where generative models commonly fail to produce biologically accurate details. This transformed the system from a binary classifier into a comprehensive interpretable platform capable of providing both visual and textual justifications for each prediction.

Furthermore, testing results revealed that the model is capable of making decisive classification judgments based on highly localized anomalous regions. One test case demonstrated that a suspicious area covering as little as 2% of the total image surface was sufficient for the model to classify the image as AI-generated with a confidence score of 76.2%, underscoring the exceptional sensitivity of the proposed XPlain model in detecting spatially constrained generation artifacts. Finally, the deployment of the system through a Streamlit-based web interface confirmed its accessibility and usability for both technical and non-technical users, reinforcing its potential for real-world application in domains such as digital media verification, forensic investigation, and cybersecurity.

8.3 Limitations and Future Work

Despite the strong results achieved by the proposed XPlain system, several limitations must be acknowledged objectively, as they simultaneously represent promising directions for future research and development.

With regard to the current limitations, the most notable constraint lies in the model's restricted generalization capability when confronted with modern AI image generation techniques. Since the model was trained primarily on GAN-generated images, contemporary diffusion-based models such as Midjourney and DALL-E 3 produce fundamentally different noise characteristics and pixel-level artifacts that the fixed SRM filters may not capture with equal effectiveness. This represents a critical challenge given the rapid evolution of generative AI technologies. Furthermore, the deployed system currently lacks an automated face cropping mechanism prior to model inference, meaning that images

containing complex backgrounds or multiple faces may yield reduced classification accuracy. The system's dependence on the Google Colab environment also imposed computational constraints that limited the scope of experimentation and continuous model refinement throughout the development process. Additionally, the textual explanations generated by the system are produced through heuristic rule-based mapping rather than a large language model, which restricts the depth, nuance, and contextual richness of the explanations presented to the end user. Finally, the system is currently constrained to the analysis of static images and does not support video-based deepfake detection, which represents a significant limitation given the widespread prevalence of manipulated video content across digital platforms.

Concerning future work, several research directions are proposed to further advance the system's capabilities and broaden its applicability. The most pressing priority is the expansion of the training dataset to include images generated by modern diffusion-based models, which would substantially enhance the system's generalization ability and ensure its relevance in the face of rapidly evolving generative AI techniques. It is also recommended to investigate the replacement of the current simple concatenation-based stream fusion mechanism with more sophisticated attention-based fusion strategies, which would enable the model to dynamically determine the relative importance of each stream based on the nature of the input image and the type of manipulation present. The integration of large language models for generating intelligent and contextually rich textual explanations represents another valuable direction, as it would significantly improve the depth of interpretability offered to users beyond what static rule-based templates can provide. From a media scope perspective, extending the system to support video-based deepfake detection through the incorporation of temporal consistency modeling and frame-level analysis would substantially multiply the system's practical impact and real-world relevance. With respect to deployment and scalability, transitioning from the current Streamlit-based interface to a more robust and scalable cloud infrastructure, supported by a dedicated application programming interface, would enable seamless integration with content verification platforms and social media networks. Finally, the adoption of continual learning strategies that allow the model to update itself autonomously as new deepfake generation techniques emerge would ensure the long-term sustainability and effectiveness of the system, positioning XPlain as an adaptive and resilient solution in the ongoing challenge of detecting AI-generated content.

References :

- [1] Anonymous, "TruthLens: Visual Grounding for Universal DeepFake Reasoning," *arXiv preprint arXiv:2503.15867*, 2025.
- [2] S. Mirsky and W. Lee, "Deepfakes: Evolution and Trends," *Soft Computing*, Springer, 2021.
- [3] A. Malik et al., "Visual Deepfake Detection: Review of Techniques, Tools, Limitations, and Future Prospects," *IEEE Access*, 2022.
- [4] Anonymous, "Explainable AI for DeepFake Detection," Technical Report, 2023.
- [5] Anonymous, "Guardian-AI: A Novel Deep Learning Based Deepfake Detection Model in Images," *ScienceDirect*, 2024.
- [6] R. Tolosana et al., "Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection," *Information Fusion*, Elsevier, 2020.
- [7] Anonymous, "Social Platforms in the Deepfake Age," 2023.
- [8] X. Li et al., "Deepfake Detection: A Comprehensive Survey from the Reliability Perspective," *arXiv preprint*, 2022.
- [9] Y. Li et al., "Cascade EF-GAN: Progressive Facial Expression Editing with Local Focuses," *CVPR*, 2021.
- [10] A. Radford et al., "Interpreting the Latent Space of GANs for Semantic Face Editing," *CVPR*, 2020.
- [11] C. Lee et al., "MaskGAN: Towards Diverse and Interactive Facial Image Manipulation," *CVPR*, 2020.
- [12] F. Matern et al., "Exploiting Visual Artifacts to Expose Deepfakes and Face Manipulations," *IEEE WACV Workshop*, 2019.
- [13] L. Li et al., "Face X-Ray for More General Face Forgery Detection," *CVPR*, 2020.
- [14] S. Hu et al., "Generalization of Forgery Detection with Meta Deepfake Detection Model," *IEEE Transactions on Information Forensics and Security*, 2022.
- [15] Anonymous, "Explainable AI for DeepFake Detection," 2023.
- [16] X. Li et al., "Deepfake Detection: A Comprehensive Survey from the Reliability Perspective," *arXiv preprint arXiv:2211.10881*, 2022.
- [17] Y. Mirsky and W. Lee, "The Creation and Detection of Deepfakes: A Survey," *ACM Computing Surveys*, vol. 54, no. 1, 2021.
- [18] A. Rossler et al., "Deepfake Detection," *ICCV*, 2019.
- [19] E. Wodajo and S. Atnafu, "Deepfake Detection with Deep Learning: Convolutional Neural Networks versus Transformers," *arXiv preprint*, 2021.
- [20] Anonymous, "ExDDV: A New Dataset for Explainable Deepfake Detection in Video," *arXiv preprint arXiv:2503.14421*, 2025.

- [21] Anonymous, "arXiv:2404.15955," *arXiv preprint*, 2024.
- [22] Anonymous, "Explainable AI for DeepFake Detection," 2023.
- [24] Anonymous, "An Investigation into the Utilisation of CNN with LSTM for Video Deepfake Detection," 2023.
- [25] Ş. G. Akkuş, "Deepfake Detection Using CNN Ensembles and Grad-CAM," *Medium*, 2024.
- [26] Anonymous, "AI-Generated Video Detection via Spatio-Temporal Anomaly Learning," *arXiv preprint arXiv:2403.16638*, 2024.
- [27] Anonymous, "A Novel Deep Learning Approach for Deepfake Image Detection," 2023.
- [28] Anonymous, "Detecting Deepfake Images Using Deep Learning Techniques and Explainable AI Methods," 2023.
- [29] Anonymous, "AI-Generated Video Detection via Spatio-Temporal Anomaly Learning," *arXiv preprint arXiv:2403.16638*, 2024.
- [30] Ş. G. Akkuş, "Deepfake Detection Using CNN Ensembles and Grad-CAM," *Medium*, 2024.
- [31] Anonymous, "Detecting AI-Generated Video via Frame Consistency (DeCoF)," *arXiv preprint arXiv:2402.02085*, 2024.
- [32] Anonymous, "AI-Generated Video Detection via Perceptual Straightening (ReStraV)," *arXiv preprint arXiv:2507.00583*, 2025.
- [33] Anonymous, "GenVidBench: A Challenging Benchmark for Detecting AI-Generated Video," *arXiv preprint arXiv:2501.11340*, 2025.
- [34] Anonymous, "Multi-Modal Deepfake Detection using AI: Combining Audio, Visual, Gesture, and Metadata Cues," *Semantic Scholar*, 2024.
- [35] P. Wodajo and S. Atnafu, "A New Deep Learning-Based Methodology for Video Deepfake Detection Using XGBoost," *Sensors*, MDPI, 2021.
- [36] Anonymous, "DeepFake Videos Detection Using Machine Learning," *Norma NCI Repository*, 2020.
- [37] Anonymous, "DeepExplain: Enhancing DeepFake Detection Through Transparent and Explainable AI Model," *Informatica Journal*, 2024.
- [38] Anonymous, "ExDDV: A New Dataset for Explainable Deepfake Detection in Video," *arXiv preprint arXiv:2503.14421*, 2025.
- [39] Anonymous, "Guardian-AI: A Novel Deep Learning Based Deepfake Detection Model in Images," *ScienceDirect*, 2024.
- [44] A. Rossler et al., "FaceForensics++: Learning to Detect Manipulated Facial Images," *ICCV*, 2019. [Online]. Available: <https://github.com/ondyari/FaceForensics>
- [45] B. Dolhansky et al., "The DeepFake Detection Challenge (DFDC) Dataset," *Meta AI*, 2020. [Online]. Available: <https://ai.meta.com/datasets/dfdc/>
- [47] X. Llu, "140K Real and Fake Faces Dataset," *Kaggle*, 2020. [Online]. Available: <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>

- [49] I. J. Goodfellow et al., "Generative Adversarial Networks," *arXiv preprint arXiv:1406.2661*, 2014. [Online]. Available: <https://arxiv.org/abs/1603.04467>
- [50] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [52] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," *ICCV*, 2017. [Online]. Available: https://openaccess.thecvf.com/content_ICCV_2017/papers/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.pdf
- [53] Streamlit Inc., "Streamlit: The Fastest Way to Build Data Apps," 2024. [Online]. Available: <https://streamlit.io>
- [54] Google LLC, "Google Colaboratory," 2024. [Online]. Available: <https://colab.research.google.com>
- [56] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST, 2023.
- [61] M. T. Ribeiro et al., "LIME: Why Should I Trust You? Explaining the Predictions of Any Classifier," *ACM SIGKDD*, 2016. [Online]. Available: <https://arxiv.org/abs/1602.04938>
- [62] S. M. Lundberg and S. I. Lee, "A Unified Approach to Interpreting Model Predictions (SHAP)," *NeurIPS*, 2017. [Online]. Available: <https://arxiv.org/abs/2109.05397>

Appendix

Appendix A: Model Training and Fine-Tuning Analysis

This appendix presents a comprehensive log analysis of the training and fine-tuning process of the proposed deep learning model. The training pipeline was structured into two distinct phases: the Warm-Up phase and the Fine-Tuning phase, each designed to progressively optimize the model's performance while maintaining generalization capability.

During the Warm-Up phase, the model was trained across five initial epochs to establish a foundational level of accuracy before transitioning into deeper optimization. This was followed by the Fine-Tuning phase, which extended across 25 epochs and employed several advanced training strategies, including adaptive learning rate scheduling through the ReduceLROnPlateau callback, model checkpointing to preserve the best-performing weights, and early stopping to prevent unnecessary computation and potential overfitting.

Throughout the fine-tuning process, the model demonstrated consistent and stable improvement, ultimately achieving a peak validation accuracy of 99.670% at epoch 20, after which the early stopping mechanism was triggered and the best model weights were restored for final evaluation. The following figures document the detailed training logs across all phases, providing full transparency into the model's learning trajectory, optimization behavior, and convergence pattern.

```
Phase: Warm Up
epoch 1/5 ----- 228s 64ms/step - accuracy: 0.8924 - loss: 0.2497 - val_accuracy: 0.6593 - val_loss: 1.4194
125/3125
epoch 2/5 ----- 194s 62ms/step - accuracy: 0.9668 - loss: 0.0886 - val_accuracy: 0.7918 - val_loss: 0.7199
125/3125
epoch 3/5 ----- 194s 62ms/step - accuracy: 0.9779 - loss: 0.0613 - val_accuracy: 0.7157 - val_loss: 1.1398
125/3125
epoch 4/5 ----- 194s 62ms/step - accuracy: 0.9824 - loss: 0.0473 - val_accuracy: 0.5997 - val_loss: 3.3176
125/3125
epoch 5/5 ----- 194s 62ms/step - accuracy: 0.9851 - loss: 0.0409 - val_accuracy: 0.9330 - val_loss: 0.1928
125/3125
```

Figure 1: XPlain Model Training Log During Warm-Up Phase

The figure illustrates the training progress of the proposed deep learning model across five epochs during the warm-up phase. As observed, the training accuracy demonstrated a consistent upward trend, rising from 89.24% in the first epoch to 98.51% by the final epoch, while the training loss decreased significantly from 0.2497 to 0.0409, reflecting effective model optimization. The validation accuracy, although fluctuating across intermediate epochs, reached its peak at 93.30% with a corresponding validation loss of 0.1928 in the fifth epoch, indicating that the model successfully generalized to unseen data without significant overfitting.

```

Phase: Fine-Tuning
Epoch 6/25
3125/3125 ----- 0s 68ms/step - accuracy: 0.9735 - loss: 0.0776
Epoch 6: val_accuracy improved from None to 0.99110, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.models.save_model(model)`. This file format is considered legacy. We recommend u

Epoch 6: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 ----- 276s 76ms/step - accuracy: 0.9818 - loss: 0.0514 - val_accuracy: 0.9911 - val_loss: 0.0253 - learning_rate: 1.0000e-05
Epoch 7/25
3125/3125 ----- 0s 68ms/step - accuracy: 0.9916 - loss: 0.0236
Epoch 7: val_accuracy improved from 0.99110 to 0.99395, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.models.save_model(model)`. This file format is considered legacy. We recommend u

Epoch 7: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 ----- 231s 74ms/step - accuracy: 0.9922 - loss: 0.0219 - val_accuracy: 0.9940 - val_loss: 0.0176 - learning_rate: 1.0000e-05
Epoch 8/25
3125/3125 ----- 0s 69ms/step - accuracy: 0.9941 - loss: 0.0162
Epoch 8: val_accuracy improved from 0.99395 to 0.99495, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.models.save_model(model)`. This file format is considered legacy. We recommend u

Epoch 8: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 ----- 231s 74ms/step - accuracy: 0.9948 - loss: 0.0153 - val_accuracy: 0.9949 - val_loss: 0.0159 - learning_rate: 1.0000e-05
Epoch 9/25
3125/3125 ----- 0s 68ms/step - accuracy: 0.9961 - loss: 0.0124
Epoch 9: val_accuracy improved from 0.99495 to 0.99575, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.models.save_model(model)`. This file format is considered legacy. We recommend u

Epoch 9: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 ----- 230s 74ms/step - accuracy: 0.9961 - loss: 0.0122 - val_accuracy: 0.9958 - val_loss: 0.0138 - learning_rate: 1.0000e-05
Epoch 10/25
3125/3125 ----- 0s 68ms/step - accuracy: 0.9966 - loss: 0.0104
Epoch 10: val_accuracy did not improve from 0.99575
3125/3125 ----- 228s 73ms/step - accuracy: 0.9971 - loss: 0.0095 - val_accuracy: 0.9956 - val_loss: 0.0139 - learning_rate: 1.0000e-05
Epoch 11/25
3125/3125 ----- 0s 68ms/step - accuracy: 0.9972 - loss: 0.0087
Epoch 11: val_accuracy improved from 0.99575 to 0.99615, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.models.save_model(model)`. This file format is considered legacy. We recommend u

Epoch 11: finished saving model to /content/drive/MyDrive/best_model_twostream.h5

```

Figure 2: Model Training Log During Fine-Tuning Phase(Epochs 6–11)

The figure presents the training log of the proposed model during the fine-tuning phase, spanning epochs 6 through 11 out of 25 total epochs. The model demonstrated strong and consistent performance, with training accuracy steadily increasing from 97.35% in epoch 6 to 99.72% in epoch 11, while the training loss continuously decreased from 0.0776 to 0.0087, reflecting stable and effective optimization. The validation accuracy also showed notable improvement, rising from 99.11% to 99.615%, with the model checkpoint being saved multiple times upon detecting improvements in validation accuracy, specifically at epochs 6, 7, 8, 9, and 11. The validation loss further declined from 0.0253 to 0.0139, confirming that the model maintained strong generalization capabilities throughout the fine-tuning process. The learning rate remained fixed at 1.0e-05 across all observed epochs, indicating a conservative and controlled fine-tuning strategy aimed at preserving the pretrained weights while achieving incremental performance gains.

```

Epoch 12/25
3125/3125 — 0s 68ms/step - accuracy: 0.9975 - loss: 0.0083
Epoch 12: val_accuracy did not improve from 0.99615
3125/3125 — 229s 73ms/step - accuracy: 0.9978 - loss: 0.0072 - val_accuracy: 0.9958 - val_loss: 0.0123 - learning_rate: 1.0000e-05
Epoch 13/25
3125/3125 — 0s 69ms/step - accuracy: 0.9981 - loss: 0.0063
Epoch 13: val_accuracy did not improve from 0.99615
3125/3125 — 229s 73ms/step - accuracy: 0.9983 - loss: 0.0058 - val_accuracy: 0.9958 - val_loss: 0.0124 - learning_rate: 1.0000e-05
Epoch 14/25
3125/3125 — 0s 68ms/step - accuracy: 0.9985 - loss: 0.0055
Epoch 14: val_accuracy improved from 0.99615 to 0.99645, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend u
Epoch 14: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 — 231s 74ms/step - accuracy: 0.9986 - loss: 0.0051 - val_accuracy: 0.9965 - val_loss: 0.0117 - learning_rate: 1.0000e-05
Epoch 15/25
3125/3125 — 0s 68ms/step - accuracy: 0.9985 - loss: 0.0054
Epoch 15: val_accuracy did not improve from 0.99645
3125/3125 — 229s 73ms/step - accuracy: 0.9985 - loss: 0.0052 - val_accuracy: 0.9961 - val_loss: 0.0138 - learning_rate: 1.0000e-05
Epoch 16/25
3125/3125 — 0s 69ms/step - accuracy: 0.9986 - loss: 0.0050
Epoch 16: ReduceLROnPlateau reducing learning rate to 4.99999873689376e-06.
Epoch 16: val_accuracy did not improve from 0.99645
3125/3125 — 230s 74ms/step - accuracy: 0.9987 - loss: 0.0047 - val_accuracy: 0.9958 - val_loss: 0.0135 - learning_rate: 1.0000e-05
Epoch 17/25
3125/3125 — 0s 69ms/step - accuracy: 0.9985 - loss: 0.0046
Epoch 17: val_accuracy did not improve from 0.99645
3125/3125 — 230s 73ms/step - accuracy: 0.9987 - loss: 0.0042 - val_accuracy: 0.9962 - val_loss: 0.0122 - learning_rate: 5.0000e-06
Epoch 18/25
3125/3125 — 0s 69ms/step - accuracy: 0.9991 - loss: 0.0033
Epoch 18: val_accuracy improved from 0.99645 to 0.99650, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend u
Epoch 18: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 — 231s 74ms/step - accuracy: 0.9989 - loss: 0.0037 - val_accuracy: 0.9965 - val_loss: 0.0112 - learning_rate: 5.0000e-06
Epoch 19/25
3125/3125 — 0s 69ms/step - accuracy: 0.9992 - loss: 0.0029
Epoch 19: val_accuracy did not improve from 0.99650
3125/3125 — 229s 73ms/step - accuracy: 0.9992 - loss: 0.0030 - val_accuracy: 0.9965 - val_loss: 0.0110 - learning_rate: 5.0000e-06
Epoch 20/25

```

Figure 3: *Model Training Log During Fine-Tuning Phase (Epochs 12–20)*

The figure continues to document the fine-tuning phase of the proposed model from epoch 12 through epoch 20 out of 25 total epochs. During this stage, the training accuracy showed a steady progression from 99.75% in epoch 12 to 99.92% in epoch 20, while the training loss continued to decline from 0.0083 to 0.0030, reflecting sustained and stable learning. The validation accuracy remained consistently high, fluctuating narrowly between 99.58% and 99.65%, with model checkpoints being saved only when improvements were detected, specifically at epochs 14 and 18, where the validation accuracy improved to 99.645% and 99.650% respectively. A notable event occurred at epoch 16, where the ReduceLROnPlateau callback triggered a reduction in the learning rate from 1.0e-05 to approximately 5.0e-06, indicating that the model had reached a plateau and required a finer optimization step to continue improving. Following this adjustment, the model demonstrated further gains, confirming the effectiveness of adaptive learning rate scheduling in achieving incremental performance improvements during the later stages of fine-tuning.

```

Epoch 20/25
3125/3125 — 0s 69ms/step - accuracy: 0.9993 - loss: 0.0027
Epoch 20: val_accuracy improved from 0.99650 to 0.99670, saving model to /content/drive/MyDrive/best_model_twostream.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We
Epoch 20: finished saving model to /content/drive/MyDrive/best_model_twostream.h5
3125/3125 — 231s 74ms/step - accuracy: 0.9992 - loss: 0.0028 - val_accuracy: 0.9967 - val_loss: 0.0107 - learning_rate: 5.0000e-06
Epoch 21/25
3125/3125 — 0s 69ms/step - accuracy: 0.9993 - loss: 0.0028
Epoch 21: val_accuracy did not improve from 0.99670
3125/3125 — 229s 73ms/step - accuracy: 0.9992 - loss: 0.0029 - val_accuracy: 0.9965 - val_loss: 0.0117 - learning_rate: 5.0000e-06
Epoch 22/25
3125/3125 — 0s 69ms/step - accuracy: 0.9993 - loss: 0.0026
Epoch 22: ReduceLROnPlateau reducing learning rate to 2.49999936844688e-06.
Epoch 22: val_accuracy did not improve from 0.99670
3125/3125 — 229s 73ms/step - accuracy: 0.9993 - loss: 0.0026 - val_accuracy: 0.9962 - val_loss: 0.0119 - learning_rate: 5.0000e-06
Epoch 23/25
3125/3125 — 0s 69ms/step - accuracy: 0.9992 - loss: 0.0025
Epoch 23: val_accuracy did not improve from 0.99670
3125/3125 — 229s 73ms/step - accuracy: 0.9993 - loss: 0.0023 - val_accuracy: 0.9961 - val_loss: 0.0119 - learning_rate: 2.5000e-06
Epoch 24/25
3125/3125 — 0s 69ms/step - accuracy: 0.9993 - loss: 0.0026
Epoch 24: ReduceLROnPlateau reducing learning rate to 1.249999968422344e-06.
Epoch 24: val_accuracy did not improve from 0.99670
3125/3125 — 229s 73ms/step - accuracy: 0.9994 - loss: 0.0024 - val_accuracy: 0.9962 - val_loss: 0.0117 - learning_rate: 2.5000e-06
Epoch 25/25
3125/3125 — 0s 69ms/step - accuracy: 0.9997 - loss: 0.0017
Epoch 25: val_accuracy did not improve from 0.99670
3125/3125 — 229s 73ms/step - accuracy: 0.9996 - loss: 0.0019 - val_accuracy: 0.9961 - val_loss: 0.0117 - learning_rate: 1.2500e-06
Epoch 25: early stopping
Restoring model weights from the end of the best epoch: 20.

```

Figure 4: *Model Training Log During Fine-Tuning Phase (Epochs 20–25)*

The figure presents the final stage of the fine-tuning phase, covering epochs 20 through 25, which also marks the conclusion of the entire training process. In epoch 20, the validation accuracy improved from 99.650% to 99.670%, triggering a model checkpoint save, which would later be identified as the best epoch. From epoch 21 onward, the validation accuracy ceased to improve beyond 99.670%, remaining stable at approximately 99.61%–99.67% across the remaining epochs. The ReduceLROnPlateau callback continued to reduce the learning rate progressively, from 5.0e-06 at epoch 22 to 2.5e-06 at epoch 24, and finally to 1.25e-06 at epoch 25, in an attempt to extract further performance gains, though no additional improvement in validation accuracy was achieved. At the conclusion of epoch 25, the early stopping mechanism was triggered, halting the training process and restoring the model weights from the best epoch, which was epoch 20. This behavior confirms that the model had reached its optimal performance point at epoch 20, and the early stopping callback effectively prevented unnecessary computation while ensuring that the best performing model weights were preserved for final evaluation.

Appendix B: Additional Model Predictions

This appendix presents additional prediction examples generated by the proposed deepfake detection system, offering further evidence of the model's capability to accurately classify images across two fundamental categories: AI-generated images and real authentic images. The examples included in this appendix were carefully selected to demonstrate the system's performance under varying conditions and subject types, providing a broader perspective on the model's generalization ability beyond the primary evaluation metrics reported in the main chapters.

Each prediction example is accompanied by a comprehensive visual analysis consisting of four key components: the original input image, the SRM (Statistical Rich Model) noise map, the Grad-CAM (Gradient-weighted Class Activation Mapping) heatmap, and the final overlay visualization. Together, these components provide a multi-layered and interpretable explanation of the model's decision-making process, allowing the reader to visually trace the specific regions and features that influenced each classification outcome.

The first case examines an image of a person wearing traditional Gulf attire, which the model successfully identified as AI-generated with a confidence score of 76.2% and a LOW severity rating, supported by irregular pixel patterns detected in the SRM noise map and spatially localized artifacts highlighted by the Grad-CAM heatmap. The second case presents an authentic real image of a person in casual clothing, which the model correctly classified as REAL with a confidence score of 79.9%, with the heatmap confirming the complete absence of any suspicious or manipulated regions across the entire image. These two contrasting examples collectively highlight the system's robustness, interpretability, and reliability in handling diverse real-world scenarios with high classification confidence.

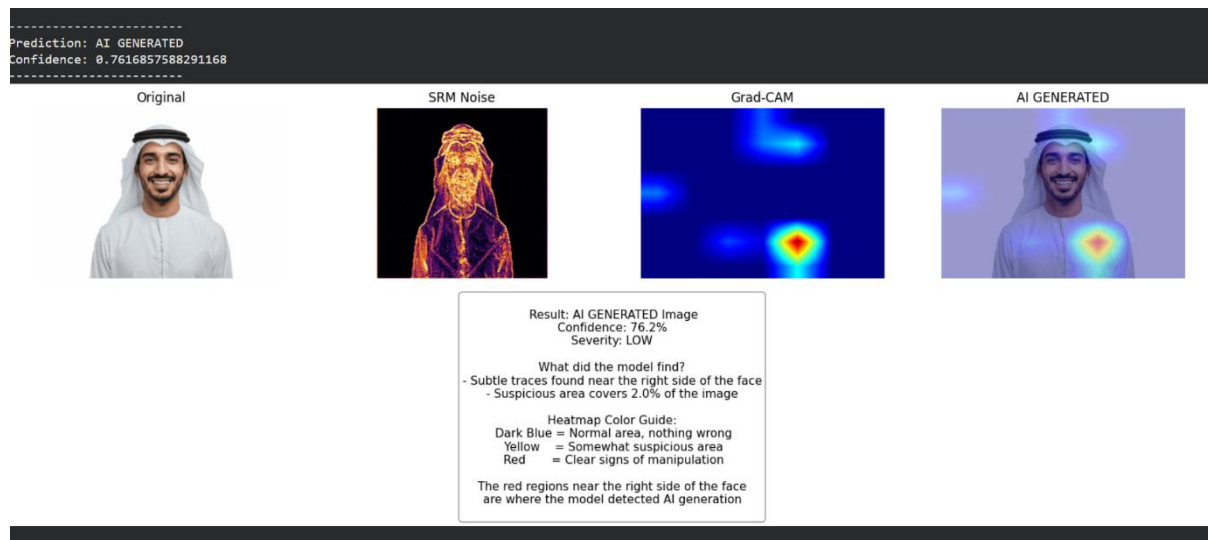


Figure 5: XPlain Detection Analysis — AI-Generated Gulf Attire Portrait

The figure presents the output of the proposed deepfake detection system applied to a test image depicting a person wearing traditional Gulf attire. The system classified the image as AI GENERATED

with a confidence score of 76.2% and a severity level of LOW, as indicated in the prediction summary displayed at the top of the figure.

The analysis is visualized through four panels. The first panel displays the Original image, which appears visually authentic and shows no obvious signs of manipulation to the naked eye. The second panel presents the SRM (Statistical Rich Model) Noise Map, which revealed irregular and inconsistent pixel patterns distributed across the face and body region, a characteristic signature commonly associated with AI-generated imagery that arises from the unnatural frequency distribution introduced during the synthesis process. The third panel illustrates the Grad-CAM (Gradient-weighted Class Activation Mapping) Heatmap, which localized the model's attention to a specific suspicious region on the right side of the body, highlighted in red, indicating clear signs of manipulation, while the surrounding areas remained in dark blue, representing normal and unaffected regions. The fourth panel shows the overlay visualization, which superimposes the heatmap onto the original image, confirming that the detected artifacts were spatially localized rather than distributed across the entire image, covering approximately 2.0% of the total image area.

The combination of SRM noise analysis and Grad-CAM visualization demonstrates the model's ability to not only detect AI-generated content but also provide interpretable and spatially explainable evidence of the detected manipulation, which is essential for building trust and transparency in deepfake detection systems.

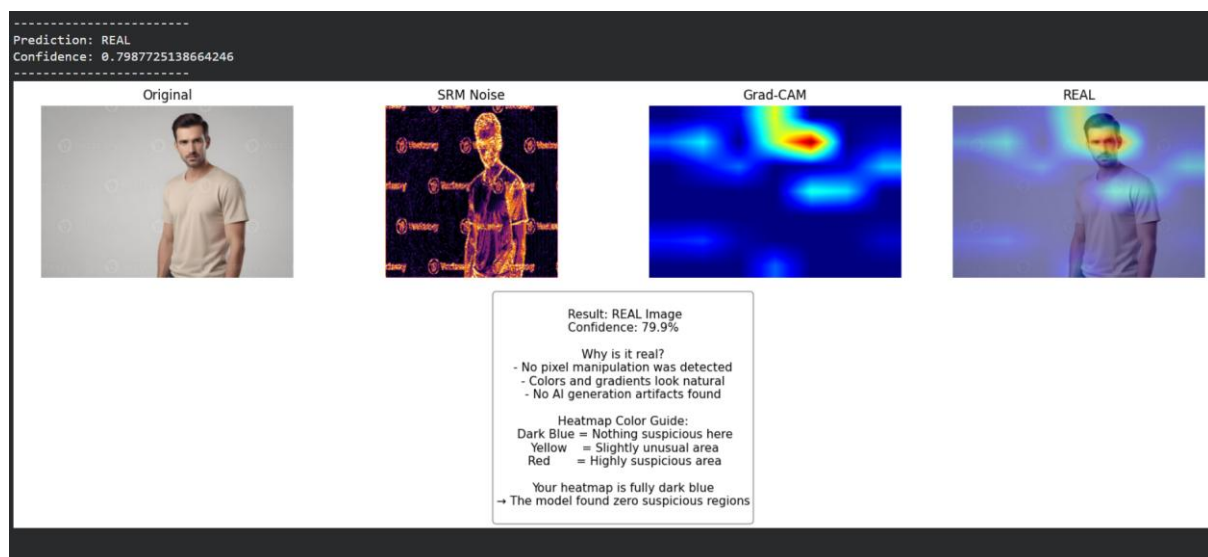


Figure 6: XPlain Detection Analysis — Real Human Portrait

The figure presents the output of the proposed deepfake detection system applied to a test image depicting a real person wearing a casual beige t-shirt against a plain background. The system correctly classified the image as REAL with a confidence score of 79.9%, demonstrating the model's ability to accurately distinguish authentic images from AI-generated content.

The analysis is visualized through four panels. The first panel displays the Original image, which appears natural and consistent in terms of lighting, texture, and facial features. The second panel presents the SRM Noise Map, which showed uniform and consistent pixel patterns across the face and body, with no irregular frequency distributions or manipulation artifacts detected, confirming the natural origin of the image. The third panel illustrates the Grad-CAM Heatmap, which in this case displayed a

predominantly dark blue map across the entire image, indicating that the model found zero suspicious regions warranting attention. The presence of some red and yellow areas in the upper region of the heatmap reflects the model's natural feature extraction process rather than any sign of manipulation, as these activations did not contribute to a deepfake classification. The fourth panel shows the overlay visualization, which further confirmed the absence of any localized manipulation artifacts, with the heatmap remaining fully dark blue across all regions of the image.

The results collectively confirm that the image exhibited natural color gradients, consistent pixel statistics, and no AI generation artifacts, validating the model's classification decision and further demonstrating the system's reliability in correctly identifying authentic real-world images with high confidence.